ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC & KỸ THUẬT MÁY TÍNH



ĐỒ ÁN TỐT NGHIỆP

Tích hợp AI trên thiết bị biên dựa trên RISC-V cho các ứng dụng IoT

Ngành: Kỹ thuật máy tính

HỘI ĐỒNG: Đồ án 4 Kỹ thuật Máy tính
GVHD: Trần Ngọc Thịnh
Huỳnh Phúc Nghị
—o0o—
SVTH 1: Ngô Minh Hồng Thái (1912046)

LỜI CAM ĐOAN

Tôi cam kết toàn bộ sản phẩm này là kết quả của quá trình nghiên cứu và hiện thực dưới sự hướng dẫn và giúp đỡ của PGS. Trần Ngọc Thịnh và ThS. Huỳnh Phúc Nghị, ngoài trừ những cơ sở lí thuyết và công trình nghiên cứu được trình bày cụ thể ở phần Tài liệu Tham khảo. Những kết quả, số liệu thu nhập được là kết quả rõ ràng, trung thực và không sao chép ở bất kỳ nguồn nào. Nếu có bất kỳ sai phạm nào được phát hiện, tôi sẵn sàng chịu mọi hình thức kỷ luật và trách nhiệm trước pháp luật.

Thành phố Hồ Chí Minh, 05/2023

LỜI CẢM ƠN

Để hoàn thành đề tài đồ án này, tôi xin chân thành gửi lời cảm ơn và bày tỏ lòng biết ơn sâu sắc tới:

Quý thầy, cô trong khoa Khoa học và Kỹ thuật Máy tính, bộ môn Kỹ thuật Máy tính, trường Đại học Bách Khoa Tp. Hồ Chí Minh, những người luôn hết lòng tận tụy cống hiến tài năng và tri thức của mình cho sự nghiệp giáo dục. Đặc biệt, tôi xin gửi lời cảm ơn chân thành và trân trọng nhất đến thầy PGS. Trần Ngọc Thịnh và ThS. Huỳnh Phúc Nghị đã tận tình hướng dẫn, hỗ trợ và giúp đỡ trong suốt quá trình nghiên cứu và thực hiện đề tài.

Các anh/chị, các bạn, các em cùng khoa cũng như khác khoa, đặc biệt là tất cả thành viên của tập thể TickLab đã hỗ trợ, giúp đỡ và góp phần định hướng cả về sự nghiệp và xây dựng về nhân cách trong suốt quá trình học tập và sinh hoạt tại trường Đại học Bách Khoa Tp. Hồ Chí Minh.

Gia đình luôn là điểm tựa, là động lực và là nơi để trở về của con. Con cảm ơn Cha vì luôn yêu thương, mạnh mẽ, thấu hiểu, tin tưởng cũng như cho con những bài học quý báu nhất. Con cảm ơn Bà ngoại vì đã thay Mẹ chăm sóc, giúp đỡ con trong suốt quá trình học tập. Con cảm ơn Mẹ đã sinh con ra, luôn tin tưởng, thương yêu và chia sẻ với con mọi tâm tư tình cảm. Dù Mẹ đã đi xa nhưng con hứa con sẽ luôn cố gắng từng ngày, từng giờ để luôn đi về phía trước theo con đường mình đã chọn.

Tôi cảm thấy rất may mắn khi có được những sự tin tưởng và giúp đỡ này!!

TÓM TẮT

Trong những năm gần đây, sự phát triển của Internet of Things (IoT) đã mang tới sự thay đổi đột phá trong việc phục vụ đời sống con người. Nhu cầu về tích hợp Trí tuệ nhân tạo (AI) trên các thiết bị IoT đã trở nên ngày càng quan trọng và cần thiết cho các ứng dụng phục vụ nhu cầu của con người. Tuy nhiên, các thiết bị này có yêu cầu phần cứng nghiêm ngặt về năng lượng, diện tích và đồng thời nó phải đảm bảo khả năng tính toán hiệu quả. Trong đề tài này, tôi đề xuất hệ thống SoC IoT dựa trên RISC-V cho thiết bị biên kết hợp bộ tăng tốc cho mạng Deep Neural Network bao gồm 2 thành phần chính: phần cứng là hệ thống dựa trên RISC-V và phần mềm bao gồm các chương trình C phục vụ huấn luyện và suy luận mô hình Deep Learning trên phần cứng tương ứng. Bộ tăng tốc được thiết kế tăng tốc quá trình tính toán cho mô hình Deep Learning dựa trên việc tái sử dụng dữ liệu, sử dụng mô hình bit-serial cho phép song song hóa tính toán, sử dụng hiệu quả dữ liệu được đọc từ bộ nhớ và sử dụng cơ chế Pipeline gồm 3 giai đoạn. Hệ thống được triển khai và đánh giá dựa trên công nghệ FPGA với thử nghiệm được thực hiện trên bo mạch Ultra96-V2. Với việc kiểm thử ở các Layer cơ bản, tốc độ tính toán tăng gấp $272 \sim 2422$ lần ở Convolution Layer, $34 \sim 114$ lần ở Fully-Connected layer và $38 \sim 146$ lần ở Pooling Layer. Thêm nữa, với việc kiểm thử ở mô hình CNN MNIST, tốc độ cải thiện tới 474 lần. Hệ thống cải thiện hiệu quả tính toán rõ rệt với mức năng lượng tiêu thụ là 328 mW.

ABSTRACT

In recent years, the development of the Internet of Things (IoT) has led to a breakthrough change in the task of serving people's lives. The need to integrate Artificial Intelligence (AI) on IoT devices is becoming more and more important, and necessary for applications serving human needs. However, these devices have strict hardware requirements in terms of energy, area and also it must ensure efficient computing capacity. In this project, I would like to propose a RISC-V-based IoT SoC for edge devices with a Deep Neural Network accelerator that consists of 2 main components: Hardware is a RISC-V-based SoC system and software includes C program for training and inference Deep Learning models on respective RISC-V hardware. Accelerator is designed to accelerate the computation of Deep Learning models based on data reuse, using bit-serial models to enable parallel computation, efficient use of data read from memory and uses a 3-Stage Pipeline mechanism. The system is implemented and evaluated based on FPGA technology with testing performed on an Ultra96-V2 board. When testing the model in the basic Deep Learning Layer, the computation speed increased by $272 \sim 2422$ times in the Convolution Layer, $34 \sim 114$ times in the Fully-Connected Layer, and $38 \sim 146$ times in the Pooling Layer. In addition, with the test in the CNN MNIST model, the speed has been improved by 474 times. The system significantly improves computational efficiency with an energy consumption of 328 mW.

Mục lục

1 GIỚI THIỆU ĐỀ TÀI **1**

1.1 Giới thiệu vấn đề 1

1.2 Phương pháp giải quyết 2

1.2.1 Vấn đề cần giải quyết 2

1.2.2 Phương pháp đề xuất 3

1.2.3 Cấu trúc của đồ án 4

2 CƠ SỞ LÝ THUYẾT **5**

2.1 Trí tuệ nhân tạo – Artificial Intelligence 5

2.1.1 Học sâu – Deep Learning 5

2.1.2 Kiến thức nền tảng về Học sâu 6

2.1.3 Huấn luyện – Training và Suy luận – Inference 12

2.2 RISC-V 14

2.2.1 Giới thiệu 14

2.2.2 Lợi ích 14

2.2.3 Giới thiệu các ISA của RISC-V 15

2.3 Nguyên lý lượng tử hóa – Quantization Specification 19

2.3.1 Phương pháp lượng tử hóa số nguyên 19

2.3.2 Tính toán trên các giá trị lượng tử 20

3 CÔNG TRÌNH NGHIÊN CỨU LIÊN QUAN **22**

3.1 RISC-V 22

3.1.1 Lõi tính toán RISC-V 22

3.1.2 Hệ thống trên chip – SoC RISC-V 24

3.2 Hiện thực Deep Learning trên phần cứng 25

3.2.1 Về kiến trúc 25

3.2.2 Về MAC 27

3.2.3 Về Dataflow 28

3.2.4 Công trình khác 32

4 THIẾT KẾ BỘ TĂNG TỐC **33**

4.1 Giới thiệu tổng quát . . . 33
4.2 Thiết kế chức năng . . . 34
4.2.1 Kiến trúc tổng quát . . . 34
4.2.2 Thiết kế các khối chức năng . . . 36
4.2.3 Khả năng tính toán . . . 47
4.3 Thiết kế điều khiển . . . 49
4.3.1 Cấu hình cho bộ tăng tốc . . . 49
4.3.2 Pipeline ở Alpha Accelerator . . . 52
4.3.3 Máy trạng thái . . . 56
4.3.4 Quy trình tính toán . . . 70

5 THIẾT KẾ HỆ THỐNG 81
5.1 Kiến trúc phần cứng . . . 81
5.1.1 Khối Alpha Accelerator Pico Controller . . . 82
5.1.2 Không gian địa chỉ . . . 83
5.2 Nền tảng phần mềm . . . 85
5.2.1 Công cụ đào tạo mô hình Deep Learning . . . 85
5.2.2 Công cụ biên dịch chương trình . . . 86
5.2.3 Tập thư viện suy luận trên mô hình Deep Learning . . . 86
5.2.4 Tập thư viện đặc thù của AlPicoSoC . . . 87
5.3 Thiết kế phần cứng–phần mềm . . . 88

6 KẾT QUẢ VÀ ĐÁNH GIÁ 90
6.1 Phần cứng triển khai . . . 90
6.2 Hiện thực bộ tăng tốc Alpha Accelerator . . . 92
6.2.1 Kết quả mô phỏng . . . 92
6.2.2 Kết quả hiện thực . . . 99
6.2.3 Ràng buộc sử dụng . . . 102
6.3 Hiện thực hệ thống PicoSoC và AlPicoSoC . . . 104
6.3.1 Hiện thực và tinh chỉnh PicoSoC . . . 104
6.3.2 Hiện thực AlPicoSoC . . . 106
6.4 Triển khai và đánh giá trên Ultra96-V2 FPGA . . . 109
6.4.1 Kết quả kiểm thử tốc độ tính toán cấp Layer . . . 110
6.4.2 Kết quả kiểm thử tốc độ tính toán cấp Model . . . 113
6.5 Đánh giá . . . 118

7 KẾT LUẬN 122
7.1 Tổng kết . . . 122
7.2 Tiềm năng phát triển . . . 123
7.2.1 Mở rộng về thiết bị . . . 123
7.2.2 Đa lõi – đa nhân . . . 123

7.2.3 Huấn luyện mô hình 123
7.2.4 Chuyển đổi nền tảng 123

Tài liệu tham khảo **124**

Danh sách hình vẽ

2.1 Mô hình của neuron sinh học [3] . . . 6
2.2 Mô hình của neuron nhân tạo [3] . . . 7
2.3 Đồ thị của Feedforward Neural Network (bên trái) và Recurrent Neural Network (bên phải) . . . 7
2.4 Mô hình tính toán của Fully-Connected Layer . . . 9
2.5 Mô hình tính toán của Convolution Layer . . . 10
2.6 Mô hình tính toán của Pooling Layer . . . 11

3.1 Kiến trúc của khối AI [15] . . . 25
3.2 Kiến trúc tổng quát của hệ thống [16] . . . 26
3.3 Kiến trúc tổng quát của ShiDianNao [17] . . . 27
3.4 Thực hiện tính toán đối với FCL và RL của LBPE [16] . . . 28
3.5 Thực hiện tính toán đối với CL của LBPE [16] . . . 29
3.6 Ánh xạ 1-D convolution primitive [21] . . . 30
3.7 Ánh xạ 2-D Convolution PE Set [21] . . . 30
3.8 Xử lý kích thước ngoài 2-D trong mỗi PE [21] . . . 31

4.1 Sơ đồ khối về kiến trúc của bộ tăng tốc . . . 34
4.2 Sơ đồi khối về kiến trúc của khối Processing Matrix . . . 37
4.3 Sơ đồ khối về kiến trúc của khối Processing Array . . . 38
4.4 Sơ đồ khối về kiến trúc của khối Processing Unit . . . 39
4.5 Ví dụ về phép nhân giữa hai số nguyên nhị phân 6-bit với nhau . . . 40
4.6 Tổng quát về phép nhân giữa hai số nguyên bất kỳ với nhau . . . 40
4.7 Tổng quát về phép nhân và cộng tích lũy giữa ba cặp số nguyên bất kỳ với nhau . . . 41
4.8 Sơ đồ nguyên lý đơn vị tính toán MAC với giá trị $N_0 = 3$ . . . 42
4.9 Sơ đồ khối về kiến trúc của khối Accumulate Matrix . . . 43
4.10 Sơ đồ khối về kiến trúc của khối Element-Wise Unit . . . 44
4.11 Tóm tắt về Pipeline của bộ tăng tốc . . . 52
4.12 Phân chia tài nguyên Pipeline của Alpha Accelerator . . . 55
4.13 Máy trạng thái ở giai đoạn RDATA của Convolution Layer . . . 56
4.14 Máy trạng thái ở giai đoạn RDATA của Fully-Connected Layer . . . 60
4.15 Máy trạng thái ở giai đoạn RDATA của Pooling Layer . . . 63

4.16 Máy trạng thái ở giai đoạn COMPS của Convolutionn và Fully-Connected Layer . . . 65
4.17 Máy trạng thái ở giai đoạn WBACK của Convolution và Fully-Connected Layer . . . 66
4.18 Máy trạng thái ở giai đoạn WBACK của Pooling Layer 68
4.19 Quá trình tính toán với tập Kernel là $(0, 1, 2)$ ở mức độ sâu là $(0, 1, 2)$ 70
4.20 Quá trình tính toán với tập Kernel là $(3, 4, 5)$ ở mức độ sâu là $(0, 1, 2)$ 71
4.21 Quá trình tính toán với tập Kernel là $(k_N - 3, k_N - 2, k_N - 1)$ ở mức độ sâu là $(0, 1, 2)$ 72
4.22 Quá trình tính toán với tập Kernel là $(k_N - 3, k_N - 2, k_N - 1)$ ở mức độ sâu là $(k_D - 3, k_D - 2, k_D - 1)$ 72
4.23 Ví dụ về thuật toán thay đổi giá trị tính toán trên Input Feature Map ở Convolution Layer 74
4.24 Quá trình tính toán với ở tập Input Activation đầu tiên 75
4.25 Quá trình tính toán với ở tập Input Activation thứ hai 76
4.26 Quá trình tính toán với ở tập Input Activation cuối cùng 77
4.27 Quá trình tính toán với ở mức độ sâu tính toán đầu tiên 78
4.28 Quá trình tính toán với ở mức độ sâu tính toán tiếp theo 78
4.29 Quá trình tính toán với ở mức độ sâu tính toán cuối cùng 79
4.30 Ví dụ về thuật toán thay đổi giá trị tính toán trên Input Feature Map ở Pooling Layer . 80

5.1 Sơ đồ khối về kiến trúc tổng quát của hệ thống AlPicoSoC 81
5.2 Máy trạng thái hoạt động của khối Alpha Accelerator Pico Controller 82
5.3 Quy trình đồng thiết kế phần cứng và phần mềm 88

6.1 Ngoại hình của Board FPGA Ultra96-V2 90
6.2 Waveform của Input Buffer 92
6.3 Waveform của Weight Buffer 93
6.4 Waveform của Bias and Partial-Sum Buffer 94
6.5 Waveform thứ nhất của Processing Matrix 95
6.6 Waveform thứ hai của Processing Matrix 95
6.7 Waveform của Accumulate Matrix 96
6.8 Waveform của Elment-Wise Unit 97
6.9 Waveform của Output Buffer 97
6.10 Waveform của Flow Controller 98
6.11 Waveform của Alpha Accelerator 99
6.12 Kết quả báo cáo về thời gian khi tổng hợp ở tần số 100 MHz 100
6.13 Kết quả báo cáo về thời gian khi tổng hợp ở tần số 150 MHz 100
6.14 Kết quả báo cáo về năng lượng tổng quát được thực hiện bởi Vivado 101
6.15 Waveform của PicoSoC 104
6.16 Kết quả báo cáo về thời gian khi tổng hợp PicoSoC ở tần số 100 MHz 104
6.17 Kết quả báo cáo về năng lượng tổng quát của PicoSoC được thực hiện bởi Vivado . . 105
6.18 Waveform của AlPicoSoC 106
6.19 Kết quả báo cáo về thời gian khi tổng hợp AlPicoSoC ở tần số 100 MHz 107
6.20 Kết quả báo cáo về năng lượng tổng quát của AlPicoSoC được thực hiện bởi Vivado . 108

6.21 Mô hình kiểm thử trên Ultra96-V2 FPGA 109
6.22 Quy trình triển khai trên Ultra96-V2 FPGA 110
6.23 Biểu diễn Model kiểm thử dạng sơ đồ 115

Danh sách bảng

4.1 Bảng mô tả trạng thái RDATA ở Convolution Layer 59
4.2 Bảng mô tả trạng thái RDATA ở Fully-Connected Layer 62
4.3 Bảng mô tả trạng thái RDATA ở Pooling Layer 64
4.4 Bảng mô tả trạng thái COMPS ở Convolution Layer và Fully-Connected Layer 66
4.5 Bảng mô tả trạng thái WBACK ở Convolution Layer và Fully-Connected Layer . . . 68
4.6 Bảng mô tả trạng thái WBACK ở Convolution Layer và Fully-Connected Layer . . . 69

5.1 Bảng mô tả trạng thái của khối Alpha Accelerator Pico Controller 83
5.2 Bảng mô tả không gian địa chỉ được sử dụng trong AlPicoSoC 84

6.1 Bảng tổng kết tổng quát tài nguyên sử dụng của Alpha Accelerator 100
6.2 Bảng tổng kết chi tiết tài nguyên sử dụng của Alpha Accelerator 101
6.3 Kết quả cụ thể về năng lượng tiêu thụ của Alpha Accelerator 102
6.4 Bảng tổng kết tổng quát tài nguyên sử dụng của PicoSoC 105
6.5 Bảng tổng kết chi tiết tài nguyên sử dụng của PicoSoC 105
6.6 Kết quả cụ thể về năng lượng tiêu thụ của PicoSoC 106
6.7 Bảng tổng kết tổng quát tài nguyên sử dụng của AlPicoSoC 107
6.8 Bảng tổng kết chi tiết tài nguyên sử dụng của AlPicoSoC 107
6.9 Kết quả cụ thể về năng lượng tiêu thụ của AlPicoSoC 108
6.10 Bảng mô tả về test case ở Convolution Layer 111
6.11 Bảng kết quả về tốc độ hoàn thành test case của PicoSoC ở Convolution Layer 111
6.12 Bảng kết quả về tốc độ hoàn thành test case của AlPicoSoC ở Convolution Layer . . 111
6.13 Bảng mô tả về test case ở Fully-Connected Layer 112
6.14 Bảng kết quả về tốc độ hoàn thành test case của PicoSoC ở Fully-Connected Layer . 112
6.15 Bảng kết quả về tốc độ hoàn thành test case của AlPicoSoC ở Fully-Connected Layer 112
6.16 Bảng mô tả về test case ở Pooling Layer 113
6.17 Bảng kết quả về tốc độ hoàn thành test case của PicoSoC ở Pooling Layer 113
6.18 Bảng kết quả về tốc độ hoàn thành test case của AlPicoSoC ở Pooling Layer 113
6.19 Bảng tóm tắt thông tin của Model kiểm thử 114
6.20 Bảng tổng kết về tốc độ hoàn thành 10 lần suy luận của PicoSoC đối với mô hình MNIST 116

6.21 Bảng tổng kết về tốc độ hoàn thành 10 lần suy luận của AlPicoSoC đối với mô hình MNIST 116
6.22 Bảng tổng kết về tốc độ hoàn thành 10 lần suy luận của Ultra96-V2 Processor System đối với mô hình MNIST 117
6.23 Bảng so sánh tài nguyên giữa PicoSoC và AlPicoSoC 118
6.24 Bảng so sánh năng lượng giữa PicoSoC và AlPicoSoC 118
6.25 Bảng so sánh tốc độ tính toán giữa PicoSoC và AlPicoSoC ở Convolution Layer . . . 119
6.26 Bảng so sánh tốc độ tính toán giữa PicoSoC và AlPicoSoC ở Fully-Connected Layer . 119
6.27 Bảng so sánh tốc độ tính toán giữa PicoSoC và AlPicoSoC ở Pooling Layer 120
6.28 Bảng so sánh tốc độ tính toán mô hình CNN MNIST ở Ultra96-V2 Processor System, PicoSoC và AlPicoSoC 121

Chương 1

GIỚI THIỆU ĐỀ TÀI

1.1 Giới thiệu vấn đề

Trong suốt hai thập kỷ, vạn vật kết nối – Internet of Things đã có bước phát triển vượt bậc và được ứng dụng rộng rãi trong công nghiệp. Công nghệ này trở nên ổn định, phát triển, trưởng thành một cách đáng kể và dần dần trở thành một công nghệ chủ đạo trong xu hướng phát triển khoa học công nghệ nói chung. Thêm nữa, sự phát triển của IoT không có dấu hiệu dừng lại mà ngày càng trở nên bùng nổ hơn nữa.

Một trong những trào lưu phát triển của IoT là sự kết hợp giữa trí tuệ nhân tạo – Artifical Intelligence (AI) vào phân tích dữ liệu trên các thiết bị biên. Dựa theo báo cáo của ResearchAndMarkets, thị trường IoT và AI được dự đoán đạt tới 26.79 tỷ đô vào năm 2025 với tỷ lệ tăng trưởng kép hằng năm lên tới 29.7% từ năm 2020 tới 2025 [1].

Theo đó, số lượng thiết bị IoT sẽ tăng mạnh và là một phần quan trọng trong các ứng dụng tiên tiến khác nhau như Augmented Reality, Virtual Reality, xe tự hành,... trong những năm tới. Hầu hết các ứng dụng này sẽ được phát triển dựa trên nền tảng AI sử dụng học sâu – Deep Learning để xử lý hiệu quả các thao tác như thu thập, xử lý và phân loại dữ liệu từ môi trường xung quanh để hoạt động hay truyền thông tin này lên server.

Tuy nhiên, các thuật toán Deep Learning yêu cầu rất nhiều tính toán cần phải được thực hiện. Thật vậy, cả quá trình suy luận và huấn luyện mô hình Deep Learning có thể yêu cầu hàng tỷ cho tới hàng chục tỷ tác vụ nhân và cộng tích lũy – Multiply-and-ACcumulate (MAC) khiến cho các mô hình này cực kỳ phụ thuộc vào khả năng tính toán của hệ thống được dùng cho nó.

Một trong những giải pháp đầu tiên được đề xuất là việc thực thi các mô hình Deep Learning trên các máy chủ đám mây với khả năng tính toán khổng lồ. Cách thức này, giải quyết được yêu cầu cơ bản của các mô hình Deep Learning về việc tính toán trong hoạt động suy luận và huấn luyện. Tuy nhiên, việc sử dụng giải pháp dùng máy chủ đám mây này có nghĩa là các thiết bị AIoT cần gửi một lượng lớn dữ liệu đến máy chủ để xử lý, chờ xử lý ở đám mây và gửi lại kết quả thiết bị AIoT. Do đó, cách tiếp cận này làm tăng độ trễ trong việc xử lý dữ liệu, không phù hợp cho các ứng dụng mang tính thời

gian thực và mang tới trải nghiệm với độ trễ lớn cho người dùng.

Cách thứ hai cũng là cách được lựa chọn trong đồ án là việc thực hiện tính toán ngay trên biên. Khi đó, khối lượng tính toán cần thiết cho các mô hình Deep Learning sẽ được đảm nhận bởi các thiết bị AIoT này. Tuy nhiên, System-on-Chip (SoC) trong các thiết bị này thường bị ràng buộc cực kỳ nặng về yếu tố hình dạng, chi phí, năng lượng và bảo mật. Cụ thể là các thiết bị AIoT thường là những thiết bị nhỏ, nhẹ với khả năng tính toán hạn chế và dung lượng bộ nhớ thấp, đồng thời yêu cầu mức tiêu thụ điện năng cực thấp do chúng chạy bằng pin. Do đó, việc triển khai AI vào các thiết bị biên IoT là một thách thức cực kỳ lớn.

1.2 Phương pháp giải quyết

1.2.1 Vấn đề cần giải quyết

Các vấn đề liên quan tới phần cứng hiện thực Deep Learning ở biên (các thiết bị AIOT) [2]:

- *Giới hạn về năng lượng và bộ nhớ*: Thực hiện tác vụ suy luận ở mô hình Deep Learning trên các thiết bị biên là một nhiệm vụ đầy thách thức khi chúng thường bị hạn chế về tài nguyên và năng lượng. Ví dụ: ResNet-50 yêu cầu nhiều hơn 95 MB bộ nhớ để lưu trữ các trọng số và hơn 3,8 tỷ phép nhân để xử lý một hình ảnh. Lượng xử lý như vậy là gần như không khả thi để triển khai trong các thiết bị biên mà kết quả trả về theo thời gian thực.
- *Độ trễ*: Đối với các ứng dụng thời gian thực như xe tự hành, máy bay không người lái,.. việc xử lý ở biên là cần thiết để nhận được phản hồi nhanh từ mô hình Deep Learning, cũng như vì các lý do về quyền bảo mật và riêng tư. Hơn nữa, không chỉ độ trễ mà cả các vấn đề về bảo mật và quyền riêng tư cũng thúc đẩy quá trình xử lý ở biên. Do đó, cần có phần cứng chuyên dụng để thực hiện hiệu quả tác vụ suy luận mô hình Deep Learning ở biên nhằm đáp ứng các yêu cầu về độ trễ, bảo mật và quyền riêng tư.
- *Độ chính xác, tốc độ và độ hiệu quả*: Các mô hình Deep Learning có độ chính xác cao đòi hỏi rất nhiều tác vụ liên quan tới tính toán và bộ nhớ. Mặc dù một số xu hướng gần đây gợi ý về việc thiết kế mô hình với dung lượng bộ nhớ nhỏ, phương pháp hứa hẹn nhất là nén mô hình Deep Learning bằng cách cắt tỉa mô hình hoặc lượng tử hóa tham số.
- *Độ chính xác và năng lượng*: Việc tính toán với độ chính xác cao đòi hỏi năng lượng tiêu thụ lớn hơn và ngược lại việc tính toán với độ chính xác thấp thì năng lượng tiêu thụ ít hơn. Tuy nhiên, các mô hình Deep Learning cũng đòi hỏi một mức độ chính xác nhất định cho mô hình. Do đó, việc xác định đánh đổi giữa độ chính xác và năng lượng tiêu thụ cũng là một vấn đề đáng quan tâm.
- *Tác vụ dư thường*: Mô hình Deep Learning thường chứa một số tác vụ dư thừa như phép nhân với số 0 hoặc các đầu vào tương đương nhau được tải đi tải lại nhiều lần. Do đó, không nhất thiết phải luôn hoàn thành một tác vụ hoàn chỉnh ở mọi thời điểm. Nhiệm vụ đặt ra là tìm ra những điểm dư thừa này và loại bỏ chúng để việc hệ thống hoạt động một cách hiệu quả

1.2.2 Phương pháp đề xuất

Để giải quyết các vấn đề ràng buộc trong việc triển khai mô hình Deep Learning ở thiết bị biên AIoT, có những giải pháp như sau [2]:

- *Tối ưu cấp phần mềm – Software-Level Optimization*. Mục đích của tối ưu cấp phần mềm là để giảm bớt độ phức tạp của các mô hình Deep Learning. Tối ưu cấp phần mềm chủ yếu gồm 2 phương thức chính là cắt tỉa mô hình – Network Pruning và làm tròn – Quantization các thông số của mô hình. Tuy nhiên, chỉ tối ưu cấp phần mềm thì vẫn có một số hạn chế nhất định khi hiện thực trên các phần cứng thông thường và chưa đạt được yêu cầu về năng lượng.
- *Tối ưu cấp phần cứng – Hardware-level Optimization.* Kiến trúc phần cứng đặc thù hiệu quả có thể xử lý mô hình Deep Learning một cách tiết kiệm năng lượng với hiệu suất cao. Thay vì sử dụng phần cứng thông thường, các mô hình Deep Learning nên được tăng tốc với công nghệ khác như FPGA tiết kiệm năng lượng (linh hoạt và có thể được tái lập trình) hoặc ASIC chuyên dụng (tính chuyên hóa cao và phù hợp với từng trường hợp của ứng dụng).
- *Kết hợp của hai phương pháp trên.* Đây là phương pháp mà tôi chọn trong đồ án khi xây dựng kiến trúc phần cứng đặc thù chạy các mô hình Deep Learning được rút gọn. Cách này khai thác được điểm mạnh của hai cách trên và phù hợp để tiếp tục phát triển về sau.

Tôi đề xuất một kiến trúc hệ thống cho phần cứng Deep Learning hỗ trợ cho việc suy luận các mô hình Deep Learning. Hệ thống dựa trên nền tảng là PicoSoC – một SoC đơn giản khi chỉ có lõi RISC-V chính là PicoRV32, một ngoại vi đơn giản là UART và mã lệnh được chạy trực tiếp trên SPI Flash – được tinh chỉnh cho phù hợp kết hợp với một bộ tăng tốc AI được nhúng vào. Về mặt phần mềm, mô hình được dùng trên hệ thống được lượng tử hóa dữ liệu về 8-bit để giảm kích thước mô hình và đơn giản hơn việc tính toán trên dữ liệu.

Nhiệm vụ của bộ tăng tốc AI là để thực hiện các tính toán đặc thù cho mô hình Deep Learning với độ hiệu quả cao trong khi duy trì năng lượng tiêu hao ở mức thấp. Khi đó, trong lúc bộ tăng tốc AI tính toán, lõi RISC-V PicoRV32 có thể dùng cho các tác vụ khác.

Nhìn nhận về phương pháp này, có thể nhận thấy một vài ưu và nhược điểm như sau:

- Về ưu điểm, việc tạo ra bộ tăng tốc riêng lẻ sẽ cải thiện đáng kể tốc độ làm việc cũng như hiệu suất của hệ thống ở tác vụ được tăng tốc. Thêm nữa, các bộ tăng tốc rất linh hoạt trong việc thay đổi hệ thống SoC nền tảng vì chỉ cần thay đổi phương thức giao tiếp phù hợp thì hoàn toàn có thể được tích hợp vào bất kỳ một SoC nào khác.
- Về nhược điểm, việc sử dụng bộ tăng tốc làm tăng một lượng tài nguyên tương đối đáng kể nên cần phải tính toán về việc đánh đổi giữa tài nguyên tiêu hao và hiệu suất hệ thống. Thêm nữa, việc dùng bộ tăng tốc chỉ có tác dụng với tác vụ được tăng tốc nên nếu tác vụ này không đóng góp nhiều trong hoạt động của hệ thống thì việc thêm bộ tăng tốc không có tác dụng gì nhiều trong việc cải thiện hiệu suất của toàn bộ hệ thống.

1.2.3 Cấu trúc của đồ án

Trong đồ án này, tôi chia thành 6 phần chính. Chương 2 cung cấp kiến thức nền tảng về Deep Learning, về kiến trúc RISC-V và về lý thuyết lượng tử hóa. Chương 3 cung cấp thông tin về các công trình nghiên cứu liên quan tới đồ án. Chương 4 cung cấp thông tin chi tiết về thiết kế bộ tăng tốc Alpha Accelerator. Ở chương 5, thông tin về thiết kế hệ thống AlPicoSoC được cung cấp. Kết quả mô phỏng, hiện thực và đánh giá của bộ tăng tốc và hệ thống được thể hiện ở chương 6. Cuối cùng, chương 7 đưa ra kết luận và các hướng phát triển của hệ thống.

Chương 2

CƠ SỞ LÝ THUYẾT

2.1 Trí tuệ nhân tạo – Artificial Intelligence

Trí tuệ nhân tạo – Artificial Intelligence là sự mô phỏng các quá trình trí tuệ của con người bằng máy móc, đặc biệt là hệ thống máy tính. Các quá trình này bao gồm học tập – learning (thu thập thông tin và các quy tắc sử dụng thông tin), lập luận – reasoning (sử dụng các quy tắc để đạt được kết luận gần đúng hoặc xác định) và tự điều chỉnh – self-correction.

Trí tuệ nhân tạo đã trở thành trụ cột cơ bản trong nhiều ứng dụng và hệ thống trong những năm gần đây. Nó đang thay đổi cách chúng ta tương tác với công nghệ, đến mức chúng ta thường xuyên sử dụng nó mà không hề nhận ra. Nhiều kỹ thuật thuộc lĩnh vực của AI, trong khi một kỹ thuật đặc biệt được nêu ra trong số tất cả, Học máy – Machine Learning. Trong hai thập kỷ qua, học máy đã được sử dụng rộng rãi trong các lĩnh vực ứng dụng khác nhau, nhờ có sự phát triển của các mẫu thống kê linh hoạt và dễ học.

Những mô hình phổ biến nhất là các mô hình lấy cảm hứng từ não bộ của người như Neural Network (NN), bao gồm Spiking Neural Network (SNN) và Học sâu – Deep Learning (DL) với Deep Neural Network (DNN).

2.1.1 Học sâu – Deep Learning

Học sâu – Deep Learning là một kỹ thuật học máy được lấy cảm hứng từ cách bộ não con người lọc thông tin, về cơ bản nó là cách học từ các ví dụ cụ thể. Nó giúp mô hình máy tính lọc dữ liệu đầu vào qua các lớp – layer để dự đoán và phân loại thông tin. Vì học sâu xử lý thông tin theo cách tương tự như bộ não con người, nên nó chủ yếu được sử dụng trong các ứng dụng mà mọi người thường làm.

Có nhiều ứng dụng đã dựa trên phương pháp học sâu và bao trùm nhiều lĩnh vực chính:

- *Thị giác máy tính*: Các mô hình Deep Learning đóng vai trò quan trọng để trích xuất các thông tin, tính năng có ý nghĩa từ video và hình ảnh. Các tác vụ này bao gồm định vị đối tượng, phân loại hình ảnh và phân đoạn hình ảnh.
- *Kinh doanh và Tài chính*: Các công ty tài chính triển khai các mô hình Deep Learning để dự

báo hành vi thị trường, bao gồm cả bảo hiểm và tín dụng.

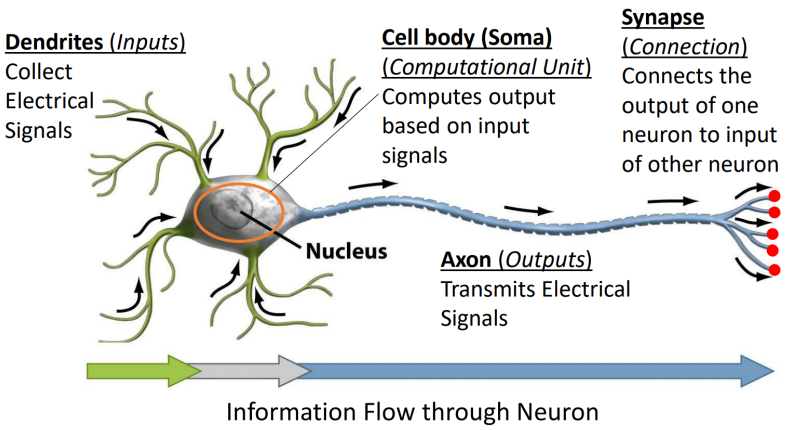
- *Chăm sóc sức khỏe*: Các mô hình Deep Learning được sử dụng rộng rãi trong phát hiện ung thư như ung thư phổi , ung thư não, ung thư da và nhiều loại ung thư khác đang không ngừng gia tăng. Ngoài ra, cũng có khả năng ứng dụng rộng rãi các kỹ thuật Deep Learning trong các trường hợp sử dụng IoT trong chăm sóc sức khỏe dựa trên các thiết bị được gắn trên người bệnh nhân, để đưa ra các dự đoán về sức khỏe ngắn hạn và dài hạn.
- *Robotic*: Các mô hình Deep Learning phục vụ trong nhiều trường hợp như xe tự hành, người máy hình người, người máy hỗ trợ, swarm và hệ thống điều khiển máy bay không người lái – drone.
- *Quản lý năng lượng thông minh*: Deep Learning cũng có thể được sử dụng để bảo tồn các nguồn tài nguyên quý giá như điện năng. Thật vậy, cả việc quản lý và dự báo lượng tiêu thụ năng lượng cần thiết đều có thể giúp tiết kiệm đáng kể.

Deep Learning về cơ bản là bắt chước bộ não con người. Do đó, nó cũng có thể được định nghĩa là một kiến trúc mạng đa thần kinh chứa một số lượng lớn các tham số và lớp.

2.1.2 Kiến thức nền tảng về Học sâu

2.1.2.1 Perceptron

Thành phần cấu thành của Neural Network là neuron hay còn gọi là perceptron, một khối tính toán cố gắng mô hình hóa hành vi của một tế bào thần kinh sinh học.

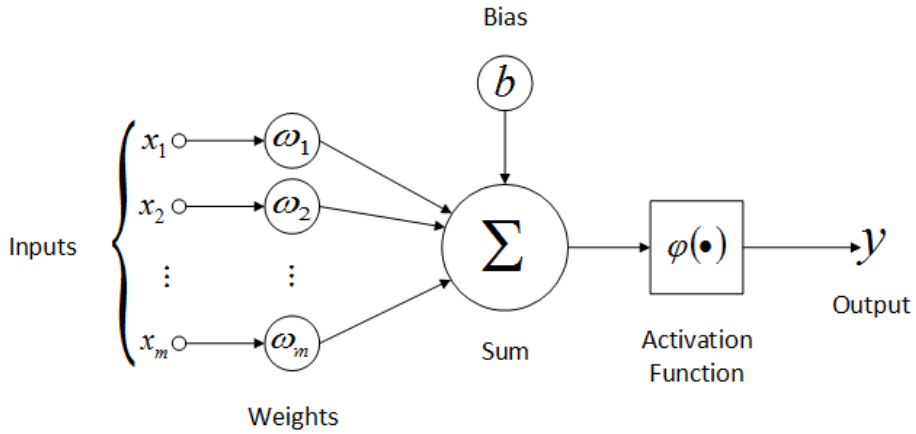


Hình 2.1: Mô hình của neuron sinh học [3]

Một tế bào thần kinh sinh học bao gồm thân tế bào – Soma, đuôi gai – Dendrite và sợi trục – Axon. Dendrite và Axon đóng vai trò là điểm tiếp nối; Dendrite nhận các kích thích, sau đó được Soma xử lý, trong khi Axon lấy tín hiệu đầu ra của neuron đến các neuron khác. Các Neuron bị kích thích bởi tín hiệu điện; khi điện áp đầu vào vượt quá một ngưỡng nhất định, một xung, được gọi là điện thế hoạt động, được tạo ra trên Axon. Phản hồi của neuron hoặc là hoàn toàn hoặc là không có gì, nghĩa là neuron chỉ có thể không có phản hồi hoặc phản hồi đầy đủ tùy thuộc vào giá trị điện áp đầu vào.

Mô hình tính toán được áp dụng trong các Neural Network đã được sửa đổi nhiều lần cho đến khi đạt

đến cấu hình hiện được áp dụng như minh họa trong hình dưới đây.



Hình 2.2: Mô hình của neuron nhân tạo [3]

Phương trình 1:

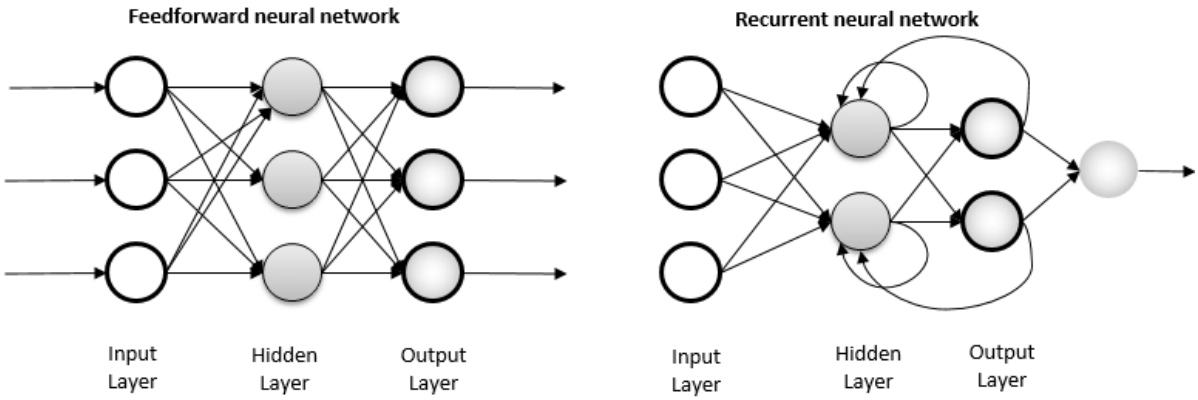
$$g(x) = \sum_{n=0}^{N-1} x[n]w[n]$$

Phương trình 2:

$$y = \varphi(g(x) + b)$$

Về bản chất, neuron nhân tạo thực hiện tính toán tổng trọng số của tất cả các đầu vào của nó (Phương trình 1), và độ lệch – bias b được thêm vào như là phần bù cho tổng. Cuối cùng, đầu ra của neuron thu được khi áp dụng hàm phi tuyến tính $\varphi()$ (Phương trình 2) vào tổng trên.

2.1.2.2 Neural Network



Hình 2.3: Đồ thị của Feedforward Neural Network (bên trái) và Recurrent Neural Network (bên phải)

Neural Network nhân tạo thường được biểu diễn dưới dạng đồ thị có hướng trong đó các nút đại diện cho các neuron. Nếu đồ thị là đồ thị không chu trình, mạng là Feedforward Neural Network. Nếu đồ thị là đồ thị có chu trình, mạng là hồi quy Recurrent Neural Network và có thuộc tính hành vi động theo thời gian.

Các nút được tổ chức theo lớp: trong Feedforward Neural Network, mỗi neuron của lớp l nhận đầu vào của nó từ lớp $l-1$ và gửi đầu ra của nó tới các neuron của lớp $l+1$. Các đầu vào của Neural Network hình thành lớp đầu vào – input layer và thông thường, có ít nhất một lớp xử lý các giá trị đầu vào này, được gọi là lớp đầu ra – output layer. Tất cả các lớp được chèn giữa các input layer và output layer được định nghĩa là các lớp ẩn – hidden layer. Số lượng các hidden layer xác định độ sâu của Neural Network. Nếu có nhiều hơn ba hidden layer, Neural Network thường được gọi là Deep Neural Network. Neural Network học cách giải quyết các vấn đề khác nhau bằng cách tìm các giá trị tối ưu cho trọng số – weight và độ lệch – bias của các neuron của nó, các neuron này có thể được tổ chức và kết nối theo những cách khác nhau.

2.1.2.3 Thành phần của Neural Network

A. Fully-Connected Layer

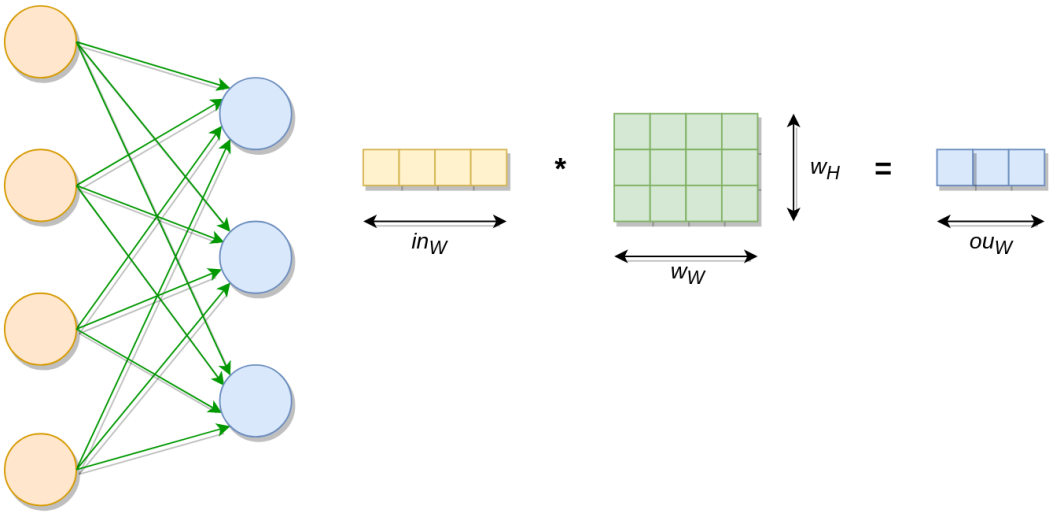
Trong Fully-Connected Layer, mỗi neuron của lớp này nhận đầu vào là tất cả các kích hoạt đầu ra của lớp trước đó. Do đó, mỗi neuron đầu ra thực hiện một tổng trọng số của tất cả các neuron đầu vào.

Các thông số liên quan trong Fully-Connected Layer:

- Về kích thước:
  - Input Activation (**I**) có kích thước là in_W (1D)
  - Weight (**W**) có kích thước là $w_W \times w_H$ (2D)
  - Output Activation (**O**) có kích thước là ou_W (1D)
  - Bias (**b**) có kích thước là b_W (1D)
- Về ràng buộc:
  - Kích thước theo chiều thứ nhất của Weight phải bằng với kích thước của Input Activation. Nghĩa là $w_W == in_W$.
  - Kích thước theo chiều thứ hai của Weight phải bằng với kích thước của Output Activation. Nghĩa là $w_H == ou_W$.
  - Kích thước của Bias phải bằng với kích thước của Output Activation. Nghĩa là $b_W == ou_W$.

Phương trình tính toán:

$$\mathbf{O}[w_o] = \sum_{w_i=0}^{in_W-1} \mathbf{W}[w_i, w_o]\mathbf{I}[w_i] + \mathbf{b}[w_o], 0 \leq w_o < ou_W, 0 \leq w_i < in_W$$



Hình 2.4: Mô hình tính toán của Fully-Connected Layer

B. Convolution Layer

Fully-Connected Layer thường không phù hợp với các tác vụ như phát hiện và nhận dạng đối tượng vì mức độ kết nối cao của chúng dẫn đến việc bùng nổ về số lượng tham số cần thiết để xử lý hình ảnh có độ phân giải cao. Hơn nữa, Fully-Connected Layer xem việc xử lý các đầu vào gần nhau hoặc xa nhau là tương tự nhau, bỏ qua cấu trúc không gian có trong hình ảnh. Để khắc phục hai vấn đề này, vào năm 1998, một kiến trúc mới đã được đề xuất, được gọi là Convolutional Neural Network (CNN), bao gồm các Convolution Layer và khai thác các ý tưởng về tiếp nhận cục bộ (local receptive) và chia sẻ trọng số. Các neuron được tổ chức theo dạng 2D, tức là bản đồ đặc trưng – feature map, và một neuron của lớp này không nhận tất cả các kích hoạt của lớp trước đó, mà thay vào đó, nó được kết nối với một vùng tiếp nhận nhỏ có kích thước $[k_W \times k_H]$. Kích thước của vùng tiếp nhận và của ma trận trọng số – weight matrix, thông thường được gọi là kích thước kernel và khoảng cách giữa các trường tiếp nhận liền kề được xác định bởi tham số bước – stride. Áp dụng ý tưởng chia sẻ trọng số, tất cả neuron của lớp này có cùng một weight matrix, phát hiện cùng một feature ở các vị trí khác nhau của lớp trước đó. Để phát hiện nhiều feature, Convolution Layer có nhiều kênh, tức là có nhiều feature map.

Các thông số liên quan trong Convolution Layer:

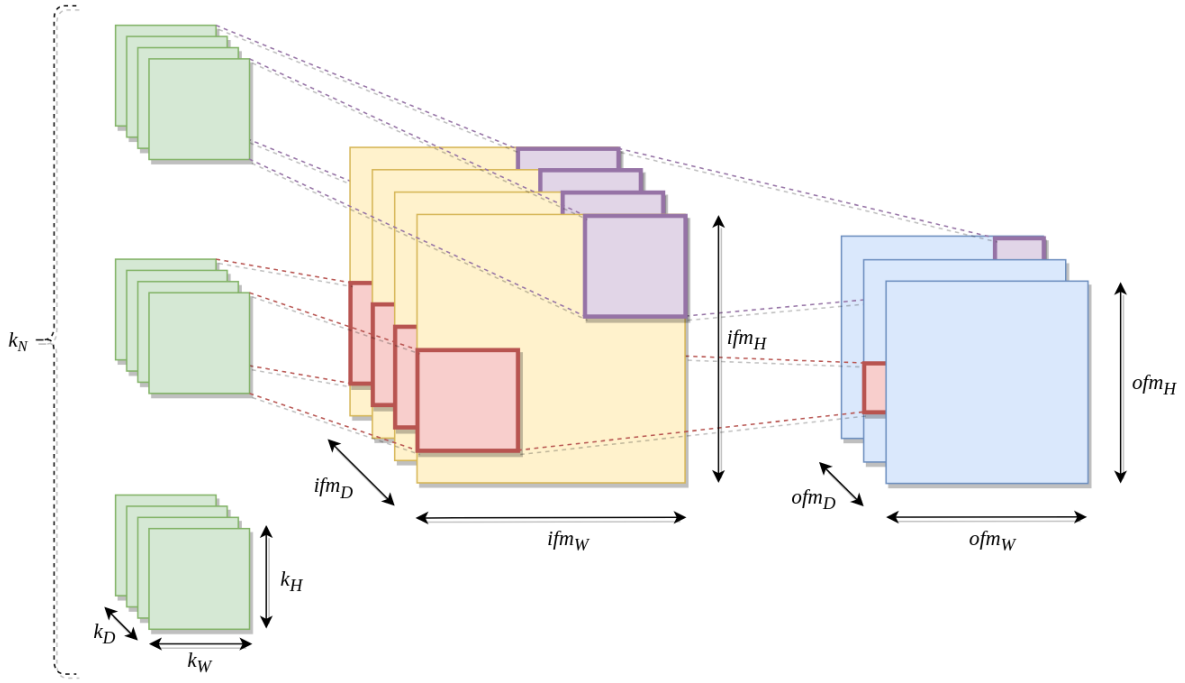
- Về kích thước:
  - Input Feature Map (**Ifm**) có kích thước là $ifm_W \times ifm_H \times ifm_D$ (3D)
  - Kernel (**K**) có kích thước là $k_W \times k_H \times k_D \times k_N$ (4D)
  - Output Feature Map (**Ofm**) có kích thước là $ofm_W \times ofm_H \times ofm_D$ (3D)
  - Bias (**b**) có kích thước là b_W (1D)
  - Stride có hai giá trị là S_W (giá trị bước theo chiều thứ nhất) và S_H (giá trị bước theo chiều thứ hai).
- Về ràng buộc:

- Số lượng Kernel phải bằng với kích thước theo chiều thứ ba của Output Feature Map
  Nghĩa là $k_N == ofm_D$.
- Kích thước theo chiều thứ ba của Kernel phải bằng với kích thước theo chiều thứ ba của Input Feature Map.
  Nghĩa là $k_D == ifm_D$.
- Kích thước của Bias phải bằng với số lượng Kernel.
  Nghĩa là $b_W == k_N$.

Phương trình tính toán:

$$\mathbf{Ofm}[w_o, h_o, d_o] = \sum_{d_i=0}^{ifm_D-1} \sum_{h_k=0}^{k_H-1} \sum_{w_k=0}^{k_W-1} (\mathbf{K}[w_k, h_k, d_i, d_o]\mathbf{Ifm}[S_w w_o + w_k, S_h h_o + h_k, d_i] + \mathbf{b}[d_o])$$

với $0 \leq w_o < ofm_W, 0 \leq h_o < ofm_H, 0 \leq d_o < ofm_D, 0 \leq h_k < k_H, 0 \leq w_k < k_W$



Hình 2.5: Mô hình tính toán của Convolution Layer

C. Pooling Layer

Pooling Layer thường được đặt sau Convolution Layer. Pooling Layer có các trường tiếp nhận tương tự như Convolution Layer. Đối với nhóm các neuron trong mỗi trường tiếp nhận, chúng trả về một giá trị duy nhất chứa thông tin liên quan tới thống kê của nhóm như giá trị tối đa hoặc giá trị trung bình. Thông số stride thường được đặt bằng với kích thước của vùng tiếp nhận để các vùng này không chồng lên nhau.

Pooling Layer làm giảm số lượng activation của một lớp và do đó làm giảm bộ nhớ yêu cầu và số lượng tính toán cần được thực hiện sau đó. Hơn nữa, Pooling Layer đạt được sự bất biến đối với việc dịch cục bộ tương đối nhỏ. Đầu ra của các Convolution Layer phụ thuộc rất nhiều vào vị trí của đầu vào, do đó, ngay cả đối với những biến đổi nhỏ của đầu vào, vẫn có những ảnh hưởng biến đổi đáng

kể đối với đầu ra. Pooling Layer giảm số lượng mẫu của đầu ra làm cho chúng ít bị ảnh hưởng bởi sự thay đổi nhỏ của đầu vào.

Các thông số liên quan trong Pooling Layer:

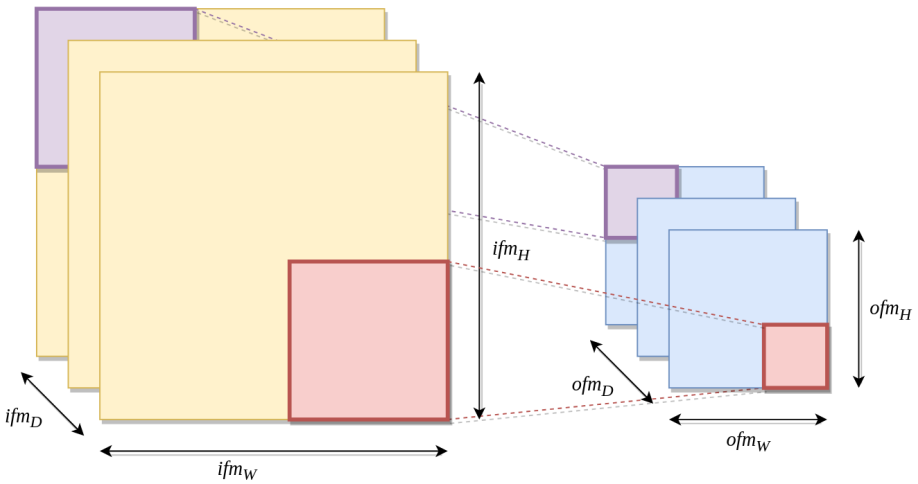
- Về kích thước:
  - Input Feature Map (**Ifm**) có kích thước là $ifm_W \times ifm_H \times ifm_D$ (3D)
  - Output Feature Map (**Ofm**) có kích thước là $ofm_W \times ofm_H \times ofm_D$ (3D)
  - Patch có hai giá trị là P_W (giá trị bước theo chiều thứ nhất) và P_H (giá trị bước theo chiều thứ hai).
  - Stride có hai giá trị là S_W (giá trị bước theo chiều thứ nhất) và S_H (giá trị bước theo chiều thứ hai).
- Về ràng buộc:
  - Kích thước theo chiều thứ ba của Input Feature Map phải bằng với kích thước theo chiều thứ ba của Output Feature Map.
    Nghĩa là $ifm_D == ofm_D$.

Phương trình tính toán:

$$\textbf{Ofm}[w_o, h_o, d_o] = Func_{w_p=0 \to P_W-1;\ h_p=0 \to P_H-1}(\textbf{Ifm}[S_w w_o + w_p, S_h h_o + h_p, d_o])$$

với $0 \leq w_o < ofm_W, 0 \leq h_o < ofm_H, 0 \leq d_o < ofm_D$

Thêm nữa, $Func$ ở đây có thể là hàm Max (tương ứng với Max Pooling) hoặc $Average$ (tương ứng với Average Pooling).

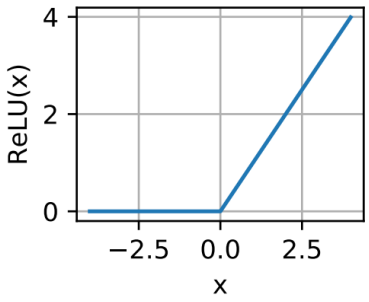


Hình 2.6: Mô hình tính toán của Pooling Layer

D. Hàm kích hoạt phi tuyến – Non-linear Activation Function

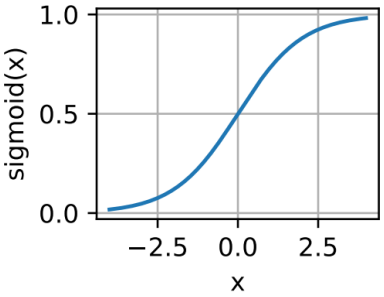
- Rectified Linear Unit (ReLU). Hàm ReLU buộc activation phải lớn hơn hoặc bằng không. Nó phổ biến vì nó hiệu quả về mặt tính toán vì nó yêu cầu một phép so sánh đơn giản giữa x và 0.

$$y = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases}$$



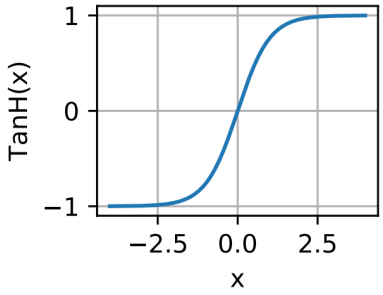
- Sigmoid. Hàm Sigmoid chuẩn hóa đầu ra trong phạm vi $(0, 1)$. Trái ngược với chức năng ReLU, Hàm Sigmoid rất tốn kém về mặt tính toán, như có thể thấy từ phương trình của nó.

$$y = \frac{1}{1 + e^{-x}}$$



- Hyperbolic Tangent (TanH). Hàm TanH tương đương với hàm Sigmoid về việc ràng buộc activation trong phạm vi $(-1, 1)$, để mô hình hóa các kết quả đầu ra cũng có thể nhận các giá trị âm.

$$y = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



- Softmax. Hàm Softmax còn được gọi là hàm số mũ chuẩn hóa – normalized exponential function. Nó nhận một vectơ đầu vào gồm N số: mỗi số được chuẩn hóa trong phạm vi $(0, 1)$ và tổng của tất cả N số bằng 1. Chức năng này được sử dụng chủ yếu trong các lớp đầu ra nếu đầu ra đại diện cho xác suất phân loại.

$$y_i = \frac{e^{x_i}}{\sum_{j=0}^{N-1} e^{x_j}} \text{ for } i = 0, 1, ..., N-1$$

2.1.3 Huấn luyện – Training và Suy luận – Inference

Neural Network có thể học cách giải quyết vấn đề bằng cách xác định giá trị chính xác của weight và bias của các lớp của nó, quá trình này được gọi là huấn luyện – training. Sử dụng Neural Network được huấn luyện, với các weight và bias đã học trước, được gọi là suy luận – inference.

Có nhiều cách để huấn luyện một Neural Network:

- *Học có giám sát – Supervised Learning*. Quá trình này yêu cầu một tập hợp các cặp đầu vào - đầu ra được dán nhãn, tức là, một tập hợp đầu vào (dữ liệu) với đầu ra tương ứng (nhãn). Tập

hợp các cặp này được gọi là tập huấn luyện. Trong quá trình Supervised Learning, mô hình nhận được đầu vào được gắn nhãn và cập nhật các tham số của nó dựa trên sự khác biệt giữa đầu ra mong đợi và đầu ra thực tế.

- *Học không giám sát – Unsupervised Learning*. Quá trình này được thực hiện khi chỉ có dữ liệu không được gắn nhãn. Quá trình này tập trung trong việc tìm kiếm các mẫu chung trong dữ liệu. Một ví dụ Unsupervised Learning là phân cụm dữ liệu dựa trên các thuộc tính của chúng.
- *Học tăng cường – Reinforcement Learning*. Đây là kiểu học chính thứ ba và tương tự như Unsupervised Learning, nó không cần dữ liệu được gán nhãn. Mục đích của Reinforcement Learning là tạo ra tác nhân tự trị có thể đưa ra quyết định trong một môi trường nhất định. Quá trình huấn luyện bao gồm các tác nhân thực hiện hành động trong một môi trường. Sau đó, có một nhân tố khác đánh giá các hành động của tác nhân dưới dạng phần thưởng, phần thưởng này sau đó sẽ được phản hồi lại cho tác nhân. Mục tiêu cuối cùng là tối đa hóa phần thưởng nhận được.

Thuật toán Supervised Learning thường được sử dụng để đào tạo Deep Neural Network là lan truyền ngược độ dốc – gradient backpropagation, trong đó các mẫu đầu vào được đưa vào mạng và đầu ra được tính toán bằng trọng số. Đầu ra thực tế và đầu ra dự kiến được so sánh và giá trị tổn thất – loss được tính toán bằng hàm mất mát – loss function. Để thực hiện quá trình học, weight được cập nhật theo một đại lượng tỷ lệ với đạo hàm riêng của tổn thất đối với chính các trọng số, tức là độ dốc – gradient. Gradient được tính toán hiệu quả bằng thuật toán lan truyền ngược, dựa trên quy tắc chuỗi – chain rule của phép tính được áp dụng để tính đạo hàm của tổn thất bắt đầu lớp đầu ra và vòng ngược lại lớp đầu vào của mạng.

Quá trình học thực sự diễn ra bằng cách cập nhật weight và bias của mạng, điều này có thể được thực hiện bằng các thuật toán tối ưu khác nhau. Thuật toán tối ưu đơn giản nhất là giảm dần độ dốc – gradient descent.

$$\theta^{t+1} = \theta^t - \eta \frac{\delta L}{\delta \theta^t}$$

với θ là tham số của mạng và η là hệ số tỷ lệ được gọi là tỷ lệ học – learning rate.

2.2 RISC-V

2.2.1 Giới thiệu

RISC-V là một kiến trúc tập lệnh – Instruction-Set Architecture (ISA) mã nguồn mở ban đầu được thiết kế để hỗ trợ nghiên cứu và giáo dục kiến trúc máy tính. Nó được xây dựng và cải thiện dựa trên Reduced Instruction Set (RISC) Architecture [4].

RISC-V là một kiến trúc load-store. Điều này có nghĩa là:

1. Các lệnh số học của nó chỉ hoạt động trên các thanh ghi
2. Chỉ có các lệnh tải – load và lưu – store là vận chuyển dữ liệu giữa thanh ghi và bộ nhớ.
3. Dữ liệu trước phải được tải vào thanh ghi trước khi có thể thực hiện các tác vụ trên nó.

RISC-V sử dụng thứ tự byte là little-endian cho hệ thống bộ nhớ. Little-endian nghĩa là byte trọng số thấp – Least Significant Byte (LSB) được lưu trữ tại địa chỉ bộ nhớ thấp nhất.

RISC-V sử dụng kỹ thuật RVC (RISC-V code Compression) để cải thiện kích thước chương trình và cũng như giảm số chu kỳ CPU trên mỗi lệnh với chi phí tăng số lệnh cho mỗi chương trình. Nó đánh đổ kích thước chương trình để đơn giản hóa các triển khai phần cứng.

Mục tiêu của RISC-V là trở thành một kiến trúc tập lệnh phổ quát:

- Trở thành một ISA hoàn toàn mở được cung cấp miễn phí trong học tập và công nghiệp.
- Phù hợp với các vi xử lý ở mọi kích cỡ
- Hoạt động tốt với nhiều loại phần mềm và ngôn ngữ lập trình phổ biến.
- Phù hợp với tất cả các công nghệ triển khai: Field-Programmable Gate Array (FPGA), Application-Specific Integrated Circuit (ASIC), full-custom chip và thậm chí cả các công nghệ thiết bị trong tương lai.
- Hiệu quả cho tất cả các kiểu kiến trúc vi mô – microarchitecture như microcoded, in-order, decoupled, out-of-order,...
- Hỗ trợ rộng rãi việc chuyên môn hóa để hoạt động như một cơ sở cho các máy tăng tốc tùy chỉnh
- Ổn định, trong đó ISA cơ bản không thay đổi.

2.2.2 Lợi ích

- *Phổ quát – Universal*. Đây là một trong những mục tiêu được đề cập ở trên, RISC-V phải phù hợp với vi xử lý ở mọi kích cỡ, với mọi kiểu triển khai (FPGA/ASIC/SoC), với các loại phần mềm khác nhau và với các ngôn ngữ lập trình khác nhau.
- *Modular*:

  Cách tiếp cận thông thường đối với kiến trúc máy tính là các kiến trúc tập lệnh gia tăng –

Incremental ISA, trong đó các bộ xử lý mới phải triển khai không chỉ các phần mở rộng ISA mới mà còn tất cả các phần mở rộng trước đây. Mục đích là duy trì khả năng tương thích ngược để các phiên bản của chương trình hàng chục năm tuổi để chúng vẫn có thể chạy chính xác trên bộ xử lý mới nhất.

Trái ngược lại, RISC-V được thiết kế theo kiểu module. Nghĩa là, RISC-V có tập lệnh cơ bản cộng với các tập lệnh mở rộng tùy chọn. Tập lệnh cơ bản luôn ổn định và không thay đổi. Điều này có nghĩa là các nhà thiết kế có thể loại bỏ những thứ họ không cần cho ứng dụng của mình.

- *Có khả năng mở rộng – Extensible*. Người thiết kế có thể thêm lệnh tùy chỉnh cho các chức năng chuyên biệt như học máy hoặc bảo mật. Điều này đặc biệt quan trọng khi Định luật Moore sắp tới kết thúc.
- *Tự do – Freedom*. Nhà thiết kế có quyền tự do làm việc trên các triển khai tối ưu của riêng mình và giữ nguyên lựa chọn của mình.
- *Đông cứng – Frozen*. Bằng cách đóng băng tập lệnh, có thể chắc chắn rằng phần mềm và công cụ ngày nay sẽ hoạt động trên các hệ thống RISC-V trong nhiều thập kỷ kể từ bây giờ.
- *Adoption & Reuse*. Bởi vì là mã nguồn mở, RISC-V sẽ khuyến khích việc áp dụng rộng rãi hơn vì khả năng tương thích của nó. Điều này cũng cho phép tăng tái sử dụng của nó.

2.2.3 Giới thiệu các ISA của RISC-V

Trong RISC-V có các tập lệnh cơ bản – Base ISA và tập lệnh mở rộng – Extension ISA. Tập cơ bản mang tính ổn định, không thay đổi và là nên tảng cho hệ thống. Các tập mở rộng được thêm vào phải phù hợp với tập cơ bản và theo mong muốn của người thiết kế.

Trước khi đi vào các tập lệnh, cách đọc các tập lệnh nên giới thiệu phục vụ cho các phần tiếp theo. Tập lệnh bất kỳ bao gồm nhiều dòng, mỗi dòng tương ứng với một lệnh (trừ các dòng chữ in nghiêng nghĩa là tên của loại lệnh). Mỗi lệnh được xem bằng cách nối các chữ cái được gạch dưới từ trái sang phải ở mỗi dòng. Ký hiệu tập hợp {} nghĩa là liệt kê các biến thể có thể có của lệnh. Cách đọc trong tập hợp cũng tương tự phần không có dấu ngoặc khi nối các chữ cái được gạch dưới từ trái sang phải. Tuy nhiên, có một ngoại lệ là dấu gạch dưới _ nghĩa là không có chữ cái nào cho biến thể này. Ví dụ như sau:

$$\underline{\mathbf{b}}\text{ranch}\left\{\begin{array}{c}\underline{\mathbf{g}}\text{reater than or }\underline{\mathbf{e}}\text{qual}\\ \underline{\mathbf{l}}\text{ess }\underline{\mathbf{t}}\text{han}\end{array}\right\}\left\{\begin{array}{c}_\\ \underline{\mathbf{u}}\text{nsigned}\end{array}\right\}$$

tương ứng với 4 lệnh là bge, blt, bgeu, bltu.

2.2.3.1 Các tập lệnh cơ bản – Base ISA

A. RV32I: RISC-V Base Integer ISA

RV32I được thiết kế đủ để biên dịch và hỗ trợ các môi trường hệ điều hành hiện đại. Đây là ISA được thiết kế để giảm phần cứng cần thiết trong một triển khai phần cứng tối giản. Các phần cứng hỗ trợ RV32I có thể thực thi các ISA mở rộng khác mà dựa trên tập lệnh cơ bản là RV32I (ngoại trừ tập lệnh mở rộng A vì yêu cầu hỗ trợ phần cứng bổ sung cho lệnh nguyên tử).

Nội dung của RV32I ISA được thể hiện như bên dưới [5].

RV32I

Integer Computation

add { – / **i**mmediate }

subtract

{ **and** / **or** / e**x**clusive **or** } { – / **i**mmediate }

{ **s**hift **l**eft **l**ogical / **s**hift **r**ight **a**rithmetic / **s**hift **r**ight **l**ogical } { – / **i**mmediate }

load **u**pper **i**mmediate

add **u**pper **i**mmediate to **pc**

set **l**ess **t**han { – / **i**mmediate } { – / **u**nsigned }

Control Transfer

branch { **e**qual / **n**ot **e**qual }

branch { **g**reater than or **e**qual / **l**ess **t**han } { – / **u**nsigned }

jump **a**nd **l**ink { – / **r**egister }

Load and Store

{ **l**oad / **s**tore } { **b**yte / **h**alfword / **w**ord }

load { **b**yte / **h**alfword } **u**nsigned

Miscellaneous Instructions

fence loads & stores

fence.instruction & data

environment { **break** / **call** }

control **s**tatus **r**egister { **r**ead & **c**lear bit / **r**ead & **s**et bit / **r**ead & **w**rite } { – / **i**mmediate }

Trong RV32I, tất cả 32 thanh ghi đều có kích thước theo chiều rộng là 32 bit. Thanh ghi x0 được cài đặt cứng với tất cả 32 bit đều bằng 0 nên thanh ghi này còn có tên gọi khác là thanh ghi zero. 31 thanh ghi đa dụng x1 − x31 còn lại giữ các giá trị được thực hiện bởi các lệnh khác nhau dưới dạng tập hợp các giá trị Boolean hoặc dưới dạng số nguyên nhị phân có dấu theo biểu diễn bù hai – two's complement hoặc số nguyên nhị phân không dấu. Ngoài ra, còn có một thanh ghi đặc biệt: thanh ghi program counter pc giữ địa chỉ của lệnh hiện tại.

Những tên bên phải dấu / ở mỗi hàng đại diện cho tên được xác định bởi RISC-V Application Binary Interface (ABI) từ Assembler. Lưu ý rằng, các tên này chỉ mang tính tham khảo chứ không phải bắt buộc.

Tập con của tập lệnh cơ sở RV32I có thể hữu ích cho mục đích mang tính sư phạm, nhưng tập cơ sở đã được xác định sao cho sẽ có rất ít động lực để một phần các triển khai phần cứng thực ngoài việc bỏ qua hỗ trợ cho các truy cập bộ nhớ bị sai lệch và coi tất cả các lệnh cấp hệ thống dưới dạng trap.

B. RV32E: RISC-V Base Integer ISA for Embedded

RV32E được thiết kế để trở thành tập lệnh cơ bản cho các vi điều khiển được dùng trong hệ thống nhúng. Nội dung của RV32E ISA tương tự như RV31I. Điểm khác biệt duy nhất so với RV32I là việc RV32E giảm số lượng thanh ghi từ 32 xuống còn 16 thanh ghi, gồm 15 thanh ghi đa dụng x1 − x15 và 1 thanh ghi chuyên dụng x0 chứa giá trị cứng là 0.

C. RV64I: RISC-V Base Integer ISA

RV64I được xây dựng dựa trên biến thể RV32I ISA. RV64I mở rộng kích thước thanh ghi và không gian địa chỉ cho người dùng lên tới 64 bit. Do đó, các thanh ghi trong RV64I cũng tương tự so với RV32I chỉ khác duy nhất là kích thước thanh ghi tăng từ 32 bit lên 64 bit.

2.2.3.2 Các tập lệnh mở rộng – Extend ISA

A. RV32M: Multiply and Divide

Tập lệnh mở rộng RV32M thêm lệnh nhân và chia số nguyên cho RV32I (hoặc RV32E). Để cung cấp bộ xử lý RISC-V nhỏ nhất cho các ứng dụng nhúng, tác vụ nhân và chia là phần đầu tiên của các phần mở rộng tiêu chuẩn của RISC-V.

Sở dĩ, các lệnh cho phép nhân và phép chia số nguyên được tách khỏi tập lệnh cơ sở là để đơn giản hóa việc triển khai phần cứng. Thêm nữa, việc này cũng phù hợp đối với các ứng dụng mà các phép toán nhân và chia số nguyên không thường xuyên xảy ra hoặc được xử lý tốt hơn trong các bộ tăng tốc kèm theo.

Các lệnh được thêm vào của RV32M ISA được thể hiện như bên dưới [5].

RV32M

multiply

multiply **h**igh { _ / **u**nsigned / **s**igned **u**nsigned }

{ **div**ide / **rem**ainder } { _ / **u**nsigned }

B. RV32F and RV32D: Single- and Double-Precision Floating Point

RV32F và RV32D bổ sung thêm các lệnh tính toán dấu chấm động có độ chính xác đơn và độ chính xác kép tuân thủ tiêu chuẩn số học IEEE 754-2008.

RV32F thêm 32 thanh ghi dấu chấm động bên cạnh 32 thanh ghi số nguyên, f0 − f31, mỗi thanh ghi rộng 32 bit. Ngoài ra, cón có thêm thanh ghi trạng thái và điều khiển dấu chấm động fcsr, chứa thông tin về trạng thái hoạt động cũng như trạng thái ngoại lệ của đơn vị tính toán dấu chấm động.

RV32D mở rộng 32 thanh ghi dấu chấm động, f0 − f31, thành 64 bit. Các thanh ghi f bây giờ có thể chứa các giá trị dấu chấm động 32 bit hoặc 64 bit.

Nguyên nhân chính cho việc dùng hai bộ thanh ghi là để bộ xử lý có thể cải thiện hiệu suất bằng cách tăng gấp đôi dung lượng thanh ghi và băng thông mà không tăng không gian cho bộ xác định thanh

ghi trong tập lệnh RISC-V.

C. RV32C: Compressed Instructions

RV32C có tác dụng giảm kích thước chương trình bằng sử dụng các lệnh 16 bit được mã hóa cho các tác vụ phổ biến. Thông thường, 50% - 60% lệnh RISC-V trong một chương trình có thể được thay thế bằng lệnh nén 16 bit, giúp giảm 25% - 30% kích thước chương trình.

RV32C có một cách tiếp cận mới: mọi lệnh ngắn phải ánh xạ tới một lệnh RISC-V 32 bit tiêu chuẩn. Hơn nữa, chỉ có Assembler và Linker biết các lệnh 16 bit, và việc thay thế lệnh ngắn bằng lệnh dài của nó là tùy thuộc vào chúng. Người viết trình biên dịch và người lập trình hợp ngữ chỉ việc sử dụng các lệnh RV32C mà không cần biết tới cách thức chuyển đổi bên trong. Do đó, kích thước của các chương trình hợp ngữ sẽ nhỏ hơn đáng kể.

2.3 Nguyên lý lượng tử hóa – Quantization Specification

Với sự phát triển mạnh mẽ của trí tuệ nhân tạo đặc biệt là học sâu, ngày càng xuất hiện nhiều mô hình Deep Learning với khả năng nhận dạng, phân tích, đánh giá,... với độ chính xác cao. Tuy nhiên, đi kèm với khả năng chính xác này chính là sự gia tăng chóng mặt số lượng thông số mô hình mạng. Có nhiều phương pháp để rút gọn mô hình, một trong những phương pháp có thể kể đến là lượng tử hóa – Quantization. Lượng tử hóa là việc sử dụng các kiểu biểu diễn dữ liệu có kích thước nhỏ hơn để lưu trữ thông số của mô hình.

Ưu điểm của việc lượng tử hóa cũng như các phương pháp rút gọn khác là việc giảm kích thước của mô hình. Từ đó làm giảm kích thước lưu trữ cũng như kích thước bộ nhớ sử dụng trên thiết bị. Nhược điểm của phương pháp này là việc đánh đổi giữa độ chính xác mô hình với kích thước mô hình. Điều này nghĩa là, kích thước mô hình được lượng tử hóa càng nhỏ thì độ chính xác cũng càng giảm và ngược lại. Dĩ nhiên, mô hình lượng tử hóa ít khi đạt được độ chính xác bằng với mô hình chưa được lượng tử hóa. Rõ ràng, phương pháp này phù hợp trong việc triển khai mô hình Deep Learning ở trên thiết bị biên.

Phương pháp lượng tử hóa được lựa chọn sử dụng là 8-bit Integer Quantization. Nguyên nhân lựa chọn phương pháp này là vì:

- Thứ nhất, khi lượng tử hóa được biểu diễn thành dạng số nguyên thì hệ thống không cần phải hiện thực tập lệnh liên quan đến tính toán số thực theo dấu chấm động khi chạy tác vụ suy luận trên mô hình Deep Learning. Thông thường, phần cứng tính toán số thực theo dấu chấm động phức tạp hơn hẳn so với tính toán cho số nguyên nhất là các bộ nhân, chia. Điều này làm giảm lượng tài nguyên cần dùng cho hệ thống.
- Thứ hai, việc sử dụng 8-bit Integer Quantization còn có thêm ưu thế ở việc hỗ trợ từ cộng đồng. Thông thường, việc huấn luyện mô hình lượng tử hóa khá phúc tạp so với việc suy luận trên mô hình lượng tử hóa. Mà các mô hình lượng tử hóa theo cách này cũng được hiện thực và hỗ trợ bởi TensorFlow Lite và cộng đồng người dùng sử dụng. Do đó, có thể sử dụng các mô hình được huấn luyện và lượng tử hóa bằng nền tảng này mà không cần phải trực tiếp huấn luyện mô hình lượng tử hóa 8-bit Integer.

2.3.1 Phương pháp lượng tử hóa số nguyên

Phương pháp biểu diễn ánh xạ giữa giá trị nguyên i (giá trị sau khi lượng tử hóa) và giá trị thực r (giá trị trước khi lượng tử hóa) được thể hiện thông qua phương trình dưới đây [6]:

$$r = S(i - Z)$$

với S và Z là các giá trị hằng số nào đó. Hai giá trị S và Z này còn được gọi là tham số trong lượng tử hóa.

Hằng số S – scale – là một giá trị thực dương bất kỳ. Khi dùng kiểu dữ liệu nhỏ hơn để lưu trữ các giá trị lớn hơn thì ta cần phải dùng một giá trị của kiểu dữ liệu nhỏ hơn để biểu diễn cho nhiều giá trị của kiểu dữ liệu lớn hơn. Do đó, hằng số này thể hiện mức độ chính xác hay cụ thể hơn là độ chia nhỏ

nhất của giá trị nguyên i khi biểu diễn giá trị thực r. Thông số này còn ảnh hưởng tới giá trị thực lớn nhất và giá trị thực nhỏ nhất có thể biểu diễn được. Ví dụ với $S = 0.01$, hai giá trị liên tiếp i và $i - 1$ biểu diễn hai giá trị thực cách nhau một khoảng đúng bằng 0.01.

Hằng số Z – zero-point – là một giá trị nguyên bất kỳ có cùng kiểu dữ liệu với i. Hằng số này chính là giá trị i để biểu diễn cho giá trị 0 trong biểu diễn số thực. Hệ số này được dùng để tinh chỉnh tính đối xứng trong việc biểu diễn số thực cho phù hợp với sự phân bố giá trị số thực. Ví dụ, khi sử dụng hàm kích hoạt ReLU thì tất cả các giá trị thu được đều nằm trong khoảng $[0, +\infty)$, làm cho việc biểu diễn các giá trị thực nhỏ hơn 0 bằng giá trị nguyên trở nên không cần thiết. Khi đó, ta có thể lựa chọn giá tri Z phù hợp để tất cả các giá trị nguyên i đều biểu diễn cho giá trị thực r lớn hơn hoặc bằng 0.

2.3.2 Tính toán trên các giá trị lượng tử

Sau khi đã hiểu được cách biểu diễn, bước quan trọng tiếp theo là cần phải xây dựng được phương pháp tính toán cho hiệu quả. Như đã biết, đối với mô hình Deep Learning, tác vụ MAC là cực kỳ quan trọng khi nó chiếm đa số trong thời gian suy luận trên mô hình. Nếu biểu diễn một cách khéo léo, ta có thể chuyển hóa các tác vụ MAC dưới dạng phép nhân giữa hai ma trận.

Xem xét phép nhân ma trận của hai ma trận số thực kích thước $N \times N$ là $\mathbf{M}_{r_1}$ và $\mathbf{M}_{r_2}$. Giả sử ma trận $\mathbf{M}_{r_3}$ cũng có kích thước là $N \times N$ là kết quả của phép nhân ma trận $\mathbf{M}_{r_1}\mathbf{M}_{r_2}$.

Gọi ma trận $\mathbf{M}_{i_1}$, $\mathbf{M}_{i_2}$ và $\mathbf{M}_{i_3}$ là kết quả sau khi lượng tử hóa mỗi giá trị ở từng ô của ma trận với các thông số là (S_1, Z_1), (S_2, Z_2) và (S_3, Z_3) tương ứng với ma trận $\mathbf{M}_{r_1}$, $\mathbf{M}_{r_2}$ và $\mathbf{M}_{r_3}$. Khi đó, ta có:

$$\mathbf{M}_{r_\alpha}[x, y] = S_\alpha(\mathbf{M}_{i_\alpha}[x, y] - Z_\alpha)$$

với $\alpha = 1 \to 3$, $1 \leq x \leq N$ và $1 \leq y \leq N$

Thay kết quả trên theo giả thiết ban đầu về phép nhân ma trận, ta có:

$$S_3(\mathbf{M}_{i_3}[x, y] - Z_3) = \sum_{k=1}^{N} S_1(\mathbf{M}_{i_1}[x, k] - Z_1)S_2(\mathbf{M}_{i_2}[k, y] - Z_2)$$

$$\Leftrightarrow \mathbf{M}_{i_3}[x, y] = Z_3 + \frac{S_1S_2}{S_3}\sum_{k=1}^{N}(\mathbf{M}_{i_1}[x, k] - Z_1)(\mathbf{M}_{i_2}[k, y] - Z_2)$$

$$\Leftrightarrow \mathbf{M}_{i_3}[x, y] = Z_3 + \frac{S_1S_2}{S_3}(\sum_{k=1}^{N}\mathbf{M}_{i_1}[x, k]\mathbf{M}_{i_2}[k, y] - Z_1\sum_{k=1}^{N}\mathbf{M}_{i_2}[k, y] - Z_2\sum_{k=1}^{N}\mathbf{M}_{i_1}[x, k] - NZ_1Z_2)$$

Đặt M là số thực có giá trị bằng $\frac{S_1S_2}{S_3}$. Trừ phép nhân với hệ số M là phải thực hiện bằng phép nhân số thực dấu chấm động, tất cả các phép tính toán còn lại đều có thể thực hiện bằng các tính toán trên số nguyên. Do đó, để có thể thực hiện phép nhân số thực dấu chấm động chỉ bằng các tính toán trên số nguyên, ta tiến hành ước lượng giá trị M thành một dạng biểu diễn bằng số nguyên và tiến hành tính toán trên giá trị đó.

Một phương pháp được đề xuất là biểu diễn M dưới dạng sau [6]:

$$M = 2^{-n}M_0$$

với M_0 nằm trong khoảng $[0.5, 1)$ và n là số nguyên không âm. Hệ số M_0 bây giờ được hiểu dưới dạng số thực dấu chấm tĩnh (các phép toán áp dụng trên số nguyên có thể được áp dụng một cách khéo léo trên số thực dấu chấm tĩnh để cho ra kết quả đúng). Khi đó phép nhân với số thực dấu chấm động M sẽ trở thành phép nhân với số thực dấu chấm tĩnh M_0 và phép dịch phải n bit.

Chương 3

CÔNG TRÌNH NGHIÊN CỨU LIÊN QUAN

3.1 RISC-V

3.1.1 Lõi tính toán RISC-V

Các công trình liên quan tới các lõi RISC-V miễn phí, mã nguồn mở được khảo sát như sau.

3.1.1.1 PicoRV32

PicoRV32 là một bộ xử lý RISC-V đơn giản, khả tổng hợp được thiết kế để phục vụ như một bộ xử lý phụ trong các thiết kế FPGA và ASIC. PicoRV32 được phát triển và duy trì bởi Clifford Wolf. PicoRV32 có tần số cao và có thể nhúng tích hợp vào trong thiết kế mà không cần lo lắng về vấn đề về Timing.

PicoRV32 hỗ trợ các tập lệnh sau:

- Các tập lệnh cơ sở bao gồm RV32I Base Integer Instruction Set và RV32E Base Integer Instruction Set
- Tập lệnh mở rộng RV32C Standard Extension for Compressed Instructions

Tính năng [7]:

- Kích thước nhỏ (750-2000 LUTs đối với kiến trúc 7-Series của Xilinx)
- Tần số làm việc tối đa cao (250-450 MHz đối với FPGA 7-Series của Xilinx)
- Cho phép lựa chọn sử dụng giao thức bộ nhớ riêng hoặc sử dụng AXI4-Lite.
- Hỗ trợ IRQ (sử dụng ISA tùy chỉnh đơn giản)
- Hỗ trợ Co-Processor Interface. Pico Co-Processor (PCPI) cho phép ghép nối với các bộ xử lý hoặc bộ tăng tốc FPGA hoặc ASIC với PicoRV32.

Đánh giá: PicoRV32 cực kỳ phù hợp cho ứng dụng ở các thiết bị biên như các node của hệ thống IoT. Nguyên nhân là vì độ đơn giản trong phần code HDL, diện tích cũng như năng lượng tiêu thụ cực kỳ nhỏ và tần số làm việc cao, PicoRV32 là ứng cử viên phù hợp cho việc sử dụng trong nghiên cứu

mang tính sư phạm.

3.1.1.2 Ibex (Zero-riscy)

Ibex là bộ xử lý 32-bit in-order RISC-V có 2 stage. Ibex được thiết kế nhỏ gọn và hiệu quả. Thông qua 2 giá trị tham số, Ibex có thể được cấu hình để hỗ trợ bốn cấu hình ISA [8].

Ibex hỗ trợ các tập lệnh như sau:

- Các tập lệnh cơ sở bao gồm RV32I Base Integer Instruction Set và RV32E Base Integer Instruction Set
- Các tập lệnh mở rộng bao gồm RV32C Standard Extension for Compressed Instructions, RV32M Standard Extension for Integer Multiplication and Division và RV32B Standard Extension for Bit-Manipulation Instructions

Đánh giá: Ibex phù hợp cho các ứng dụng ở thiết bị biên vì tính nhỏ gọn và mức năng lượng tiêu thụ thấp đối với cấu hình nhỏ và các ứng dụng từ vừa cho tới lớn đối với cấu hình lớn hơn.

3.1.1.3 CV32E40P (RI5CY)

CV32E40P là bộ xử lý 32-bit in-order RISC-V có 4 stage. ISA của CV32E40P đã được mở rộng để hỗ trợ nhiều lệnh bổ sung mà không phải là một phần của ISA RISC-V tiêu chuẩn [9].

CV32E40P hỗ trợ các tập lệnh như sau:

- Tập lệnh cơ sở RV32I Base Integer Instruction Set
- Các tập lệnh mở rộng RV32C Standard Extension for Compressed Instructions và RV32M Integer Multiplication and Division Instruction Set Extension
- Hỗ trợ một phần cho tập lệnh mở rộng RV32F Single Precision Floating Point Extensions
- Các lệnh đặc thù PULP platform:
  - Post-Incrementing load and stores
  - Multiply-Accumulate extensions
  - ALU extensions
  - Hardware Loops

Đánh giá: CV32E40P là lõi tầm trung phù hợp hơn cho các ứng dụng IoT ở mức vừa cho tới lớn vì khả năng tính toán cũng như mức năng lượng tiêu thụ.

3.1.1.4 CVA6 (Ariane)

CVA6 là bộ xử lý 64-bit in-order RISC-V có 6 stage. CVA6 hiện thực ba cấp độ đặc quyền M, S, U để hỗ trợ đầy đủ hệ điều hành giống Unix [10].

CVA6 hỗ trợ đầy đủ các tập lệnh cơ sở RV64I, tập lệnh mở rộng RV64M và RV64C cũng như một vài tập lệnh đặc thù khác.

CVA6 có kích thước có thể định cấu hình, TLB riêng biệt, phần cứng PTW và hỗ trợ dự đoán nhánh

(bộ đệm nhánh, bảng lịch sử nhánh và ngăn xếp địa chỉ trả về) [10].

Đánh giá: CVA6 là core cỡ lớn phù hợp cho các ứng dụng tính toán hiệu năng cao vì khả năng của mình tính toán cũng như hỗ trợ các phần cứng đặc thù của mình.

3.1.2 Hệ thống trên chip – SoC RISC-V

3.1.2.1 PicoSoC

PicoSoC là SoC đơn giản có cấu trúc đơn lõi được xây dựng trên lõi RISC-V PicoRV32. PicoSoC cho phép chạy chương trình trực tiếp từ chip flash SPI. Vì cấu trúc đơn giản nên PicoSoC khá phù hợp cho việc tùy chỉnh và thêm các thành phần mới vào SoC.

Các tính năng của PicoSoC [11]:

- CPU: Sử dụng PicoRV32
- Ngoại vi: Bao gồm UART và XIP SPI Flash Controller

Đánh giá: PicoSoC là SoC đơn giản dựa trên lõi PicoRV32 với khả năng mở rộng và tích hợp tốt vào thiết kế. SoC được chọn để làm baseline cho đồ án này.

3.1.2.2 PULPino – Single-core micro-controllers

PULPino là SoC đơn lõi được xây dựng trên lõi RISC-V RI5CY (CV32E40P) và Zero-riscy (Ibex). PULPino sử dụng một RAM đơn cổng như là bộ nhớ lệnh và dữ liệu. PULPino bao gồm một ROM chứa boot loader có thể tải chương trình qua SPI từ thiết bị flash bên ngoài [12].

Đánh giá: PULPino là SoC phù hợp cho các thiết bị biên trong các ứng dụng IoT với số lượng ngoại vi đầy đủ.

3.1.2.3 PULPissimo – Single-core micro-controllers

PULPissimo là SoC đơn lõi 32 bit. PULPissimo là phiên bản thứ hai của hệ thống PULPino và nó có thể được mở rộng với cụm đa lõi của dự án PULP [13]

Khác với hệ thống tiền nhiệm PULPino, PULPissimo sử dụng hệ thống bộ nhớ phức tạp hơn, hệ thống I/O sử dụng uDMA, ngoại vi mới và bộ SDK mới.

Đánh giá: PULPissimo là SoC phù hợp cho các thiết bị biên trong các ứng dụng IoT với số lượng ngoại vi đầy đủ.

3.1.2.4 OpenPULP – Multi-core IoT Processors

OpenPULP là một nền tảng mã nguồn mở triển khai một phiên bản mở rộng của tập lệnh RISC-V. OpenPULP cho phép phát triển, triển khai và vận hành tự động hiệu quả về chi phí đối với các thiết bị thu thập, phân tích, phân loại và hành động dựa trên sự kết hợp của các nguồn dữ liệu phong phú như hình ảnh, âm thanh hoặc rung động [14]. Đặc biệt, OpenPULP được tối ưu hóa để thực hiện nhiều thuật toán hình ảnh và âm thanh.

Đánh giá: OpenPULP là SoC có độ lớn tầm trung dựa trên Cluster đa lõi phù hợp cho các ứng dụng nằm giữa IoT và HPC.

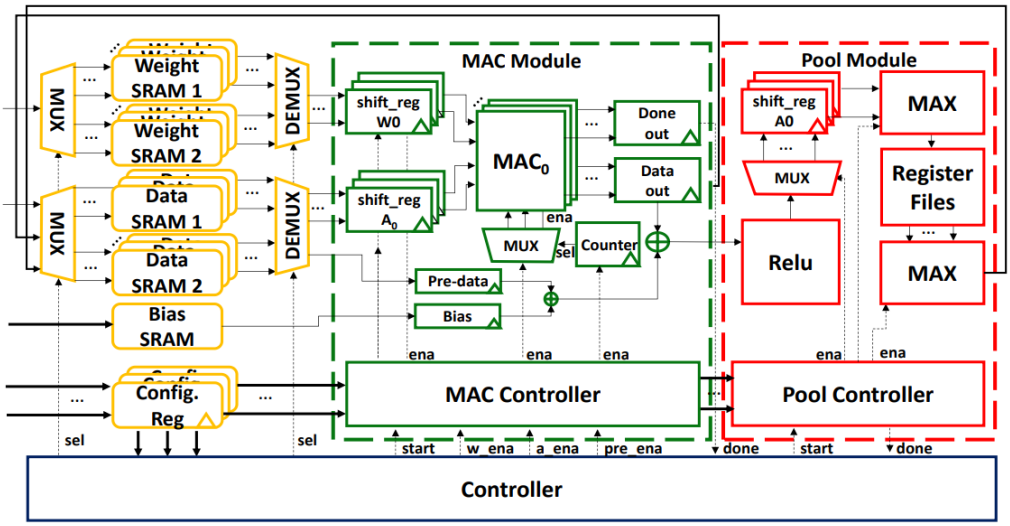
3.2 Hiện thực Deep Learning trên phần cứng

3.2.1 Về kiến trúc

Các khảo sát ở phần này tập trung vào kiến trúc của công trình cũng như cách thức đánh giá kết quả tương ứng.

3.2.1.1 Tiny Neuron Network System based on RISC-V Processor: A Decentralized Approach for IoT Applications

Kiến trúc khối AI được thể hiện ở hình bên dưới.



Hình 3.1: Kiến trúc của khối AI [15]

A. Khối MAC

Hiện thực hai mô hình MAC khác nhau: Variable-Bit-Precision (VBP) MAC và Stochastic (SC) MAC. Việc lựa chọn mô hình MAC nào được quyết định lúc thiết kế hệ thống.

B. Quản lý dữ liệu

Khối MAC hỗ trợ việc chuyển đổi giữa hai chế độ làm việc song song (parallel) và tuần tự (serial). Chế độ làm việc được lựa chọn phụ thuộc vào đặc tính của từng quá trình tính toán trong DNN [15].

Dữ liệu được gửi từ bộ nhớ hệ thống thông qua khối Direct Memory Access (DMA).

3.2.1.2 UNPU: An Energy-Efficient Deep Neural Network Accelerator With Fully Variable Weight Bit Precision

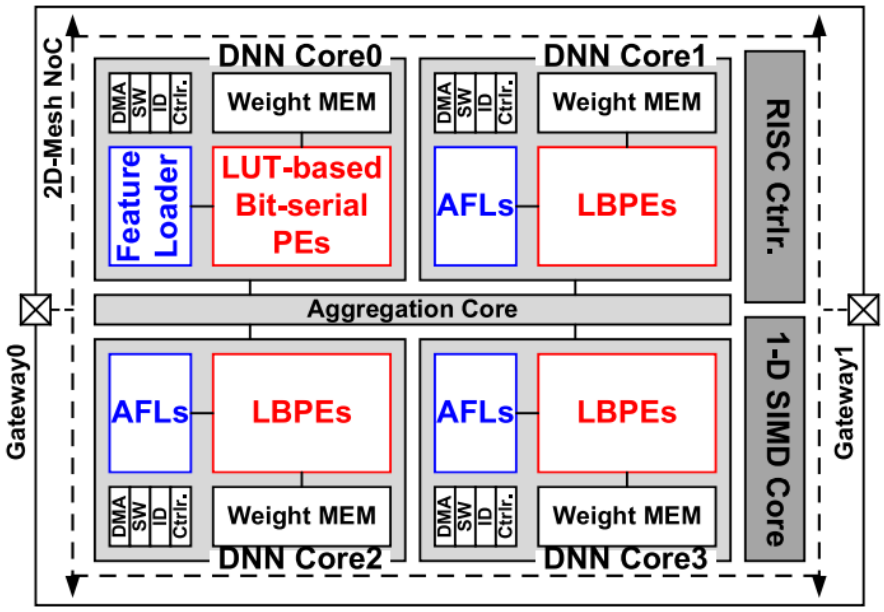
Ý tưởng:

- Giới hạn về độ chính xác:
  - Các DNN khác nhau có các giá trị tối ưu về độ chính xác của weight khác nhau.
  - Các layer khác nhau ở trong một DNN có các giá trị tối ưu về độ chính xác của weight khác nhau.

  - Độ chính xác của weight cần cho tác vụ suy luận của DNN phụ thuộc vào độ chính xác mong muốn của DNN

- Sự cần thiết của một phần cứng cho các tính toán ở các layer khác nhau (Convolution Layer, Recurrent Layer và Fully-Connected Layer)

Kiến trúc tổng quát của hệ thống như hình bên dưới.



Hình 3.2: Kiến trúc tổng quát của hệ thống [16]

Hệ thống bao gồm: 2-D mesh type network-on-chip (NoC), 4 unified DNN cores, 1-D SIMD core, RISC controller, aggregation core, và two external gateways.

- 4 unified DNN core thực hiện tính toán độc lập tạo ra các tổng một phần (partial sum) và gửi kết quả tới aggregation core để thực hiện cộng tích lũy tạo ra kết quả cuối cùng
- 1-D SIMD core thực hiện các tính toán vectore như non-linear activation function, element-wise multiplication,...
- RISC controller thực thi lệnh cần cho việc trao đổi dữ liệu giữa các core với nhau thông qua NoC
- External gateway kết nối UNPU với bộ nhớ bên ngoài để tải các giá trị weight và input feature map từ bộ nhớ ngoài vào UNPU hoặc ghi kết quả từ UNPU ra bộ nhớ ngoài

3.2.1.3 ShiDianNao: Shifting Vision Processing Closer to the Sensor

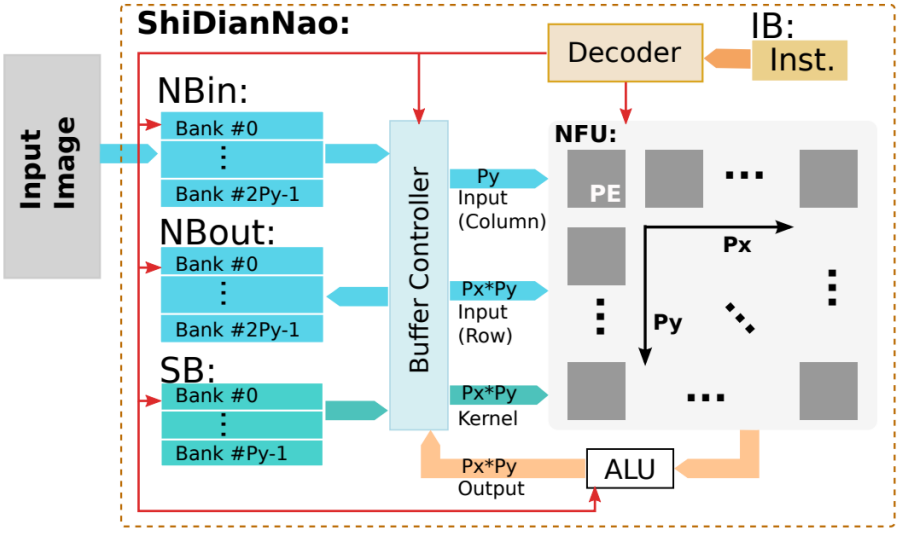
Hệ thống bao gồm:

- NBin và NBout lần lượt là hai buffer cho input và output activation
- SB là buffer cho weight
- Neural Functional Unit (NFU) cùng với Arithmetic Logic Unit (ALU) để thực hiện tính toán ở

độ chính xác 16 bit fixed-point cho output activation

- Buffer và decoder (IB) phục vụ cho các lệnh thực thi

Kiến trúc hệ thống như bên dưới.



Hình 3.3: Kiến trúc tổng quát của ShiDianNao [17]

3.2.2 Về MAC

Các khảo sát ở phần này tập trung vào khối MAC của các công trình.

3.2.2.1 UNPU: An Energy-Efficient Deep Neural Network Accelerator With Fully Variable Weight Bit Precision

Ý tưởng: Dựa trên phương pháp bit-serial, tác vụ MAC với giá trị weight có độ chính xác N-bit được tính toán tuần tự từ bit trọng số thấp cho tới bit trọng số cao trong N chu kỳ. Để ý rằng, vì số lượng input activation là cố định trong lúc thực hiện tác vụ MAC, số lượng và giá trị tổng riêng phần được xác định ngay. Do đó, bộ phận tính toán của UNPU là LBPE lập bảng bao gồm các tổng riêng phần này để phục vụ cho tác vụ MAC.

3.2.2.2 An Efficient Hardware Implementation of Artificial Neural Network based on Stochastic Computing

Ý tưởng: Stochastic Computing là phương thức tính toán dựa trên xác suất của các bit trong một chuỗi bit [18]. Điểm mạnh của phương pháp này là các thành phần cần cho tính toán được giảm mạnh, do đó diện tích và năng lượng giảm rất nhiều. Tuy nhiên, điểm yếu của phương pháp này là cần nhiều thời gian cho tính toán và độ chính xác tính toán cũng bị giảm xuống tương đối nhiều.

3.2.2.3 BRein Memory: A Single-Chip Binary/Ternary Reconfigurable in-Memory Deep Neural Network Accelerator Achieving 1.4 TOPS at 0.6 W

Binary Neural Network (BNN) giới hạn nhị phân hóa tất cả các giá trị activation và weight hoặc là +1 hoặc là -1. Khi giá trị -1 được thể hiện bằng bit 0, quá trình tính toán trở thành [19]:

1. Thực hiện XNOR giữa các bit của weight và input activation (đại diện cho phép nhân)
2. Đếm số lượng bit 1 và 0 trong kết quả tức thời sau khi thực hiện phép XNOR (đại diện cho phép cộng)
3. Xác định output activation là 1 (hoặc 0) khi mà bit 1 (hoặc 0) chiếm đại đa số (đại diện cho activation function)

3.2.2.4 Convolutional Neural Networks using Logarithmic Data Representation

Ý tưởng: Chuyển việc biểu diễn số nguyên thông thường thành dạng logarith. Khi đó việc tính toán phép nhân sẽ thay bằng việc cộng hai số mũ và số bit cần để biểu diễn một số giảm xuống. Tuy nhiên, độ chính xác số học cũng bị giảm xuống [20].

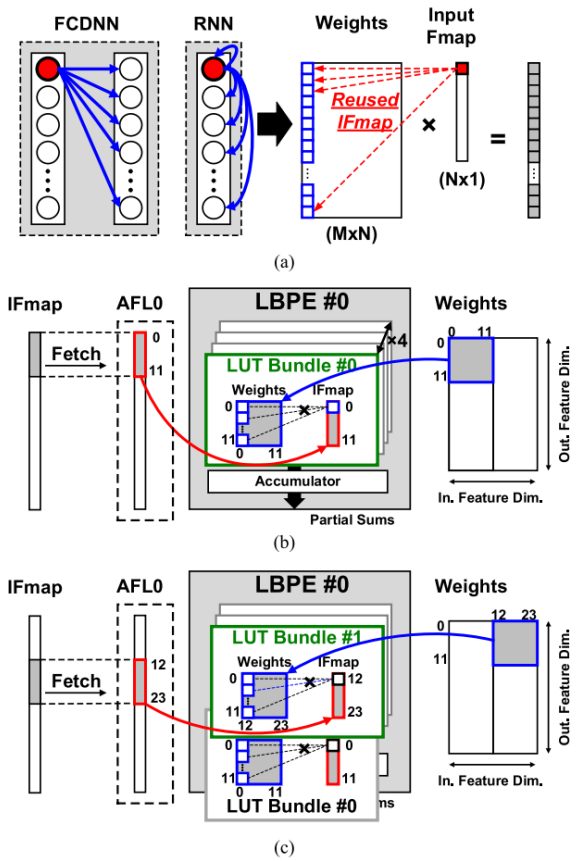
3.2.3 Về Dataflow

Các khảo sát ở phần này tập trung vào luồn tính toán (dataflow) của các công trình.

3.2.3.1 UNPU: An Energy-Efficient Deep Neural Network Accelerator With Fully Variable Weight Bit Precision

Fully-Connected Layers hoặc Recurrent Layers

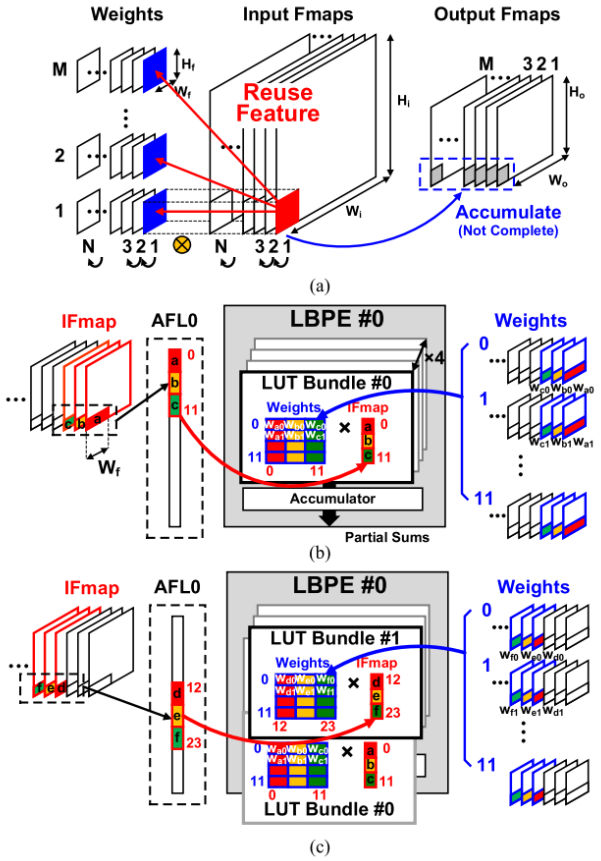
FCL và RL được tạo thành từ tích ma trận-vector và các phép toán đơn nguyên như activation function.



Hình 3.4: Thực hiện tính toán đối với FCL và RL của LBPE [16]

Convolution Layers

Tính toán cơ bản cho CL cũng giống như RL (hoặc FCL), là việc tái sử dụng input feature map. Điểm khác biệt chính của CL so với RL (hoặc FCL) là input feature map và weight có dạng 2-D. Để cung cấp tính trừu tượng cho các bộ phận tính, Input Feature Map 2-D của CL phải được chuyển đổi thành hình dạng 1-D để tạo hình dạng giống như RL (hoặc FCL) và bộ nhớ weight cục bộ sẽ lưu trữ giá trị weight cần thiết với thứ tự phù hợp cho tác vụ ở CL theo phương pháp bit-serial.



Hình 3.5: Thực hiện tính toán đối với CL của LBPE [16]

3.2.3.2 Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks

Ý tưởng: Eyeriss dựa vào hai cách tiếp cận chính để cải thiện về hiệu suất dùng năng lượng:

- Giảm lượng dịch chuyển dữ liệu
- Khai thác tính phân phối, thống kê của dữ liệu

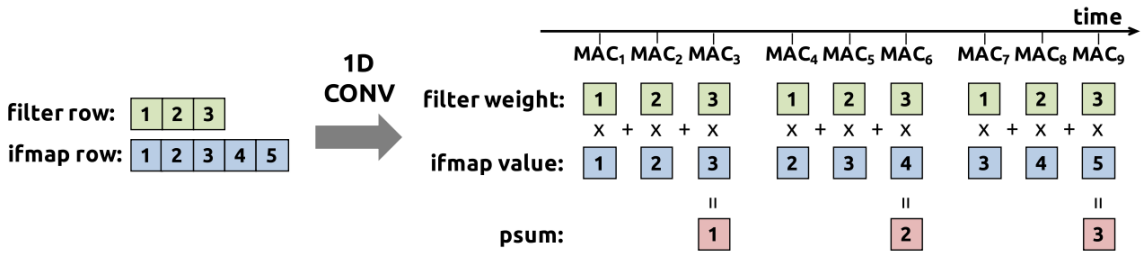
Row Stationary Dataflow

RS Dataflow sử dụng một cách tiếp cận hệ thống để tối ưu hóa quá trình này:

- 1-D Convolution Primitive: RS Dataflow đầu tiên chia quá trình tính toán thành các 1-D Convolution Primitive để các thành phần này có thể chạy song song với nhau. Mỗi tính toán primitive ở trên 1 hàng của kernel weight và 1 hàng của input feature map và tạo ra 1 hàng các tổng riêng

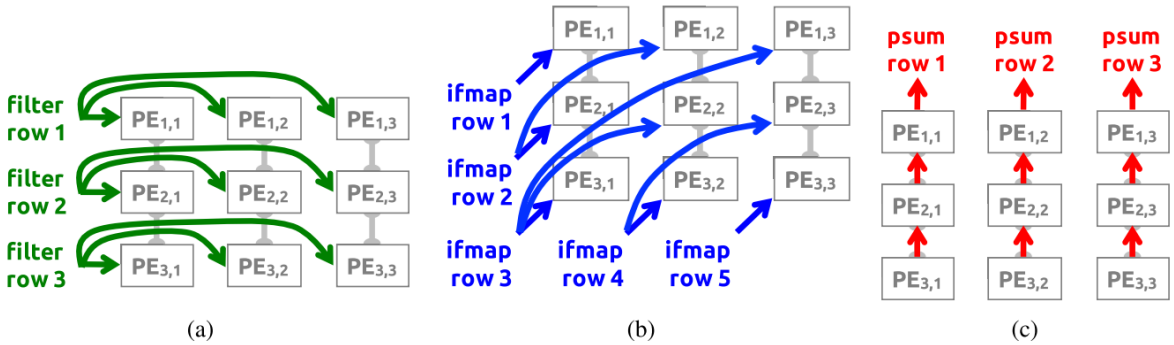
phần. Các tổng này sau đó được cộng tích lũy giữa các PE để tạo thành output feature map cuối cùng.

Bằng cách ánh xạ mỗi primitive cho 1 PE, quá trình tính toán của từng cặp hàng ổn định ở trong PE. (row pair stays stationary)



Hình 3.6: Ánh xạ 1-D convolution primitive [21]

- 2-D Convolution PE Set: Một 2-D Convolution Set là tập hợp của nhiều 1-D Convolution primitive và việc tính toán của nó:
  - Chia sẻ cùng hàng của kernel hoặc input feature map giữa các 1-D Convoluton primitive.
  - Cộng tích lũy giá trị tổng riêng phần ở nhiều primitive với nhau



Hình 3.7: Ánh xạ 2-D Convolution PE Set [21]

Trong một Set, mỗi hàng của kernel được tái sử dụng theo chiều ngang, mỗi hàng của input feature map được tái sử dụng theo đường chéo và mỗi hàng của tổng riêng phần được cộng tích lũy theo chiều dọc

- PE Set Mapping: Kích thước của PE set là hàm số của hình dạng của một layer và độc lập với kích thước vật lý của PE array. Một PE set có thể ánh xạ vào bất kỳ nhóm nào của PE array mà có cùng kích thước. Tuy nhiên, không thể áp dụng cho:
  - PE set có nhiều hơn 168 thành phần
  - PE set có ít hơn 168 thành phần, nhưng có kích thước chiều dài lớn hơn 14 hoặc chiều rộng lớn hơn 12.
- Dimensions Beyond 2-D in PE Array: Xử lý nhiều phép 2-D convolution là bắt buộc để hoàn

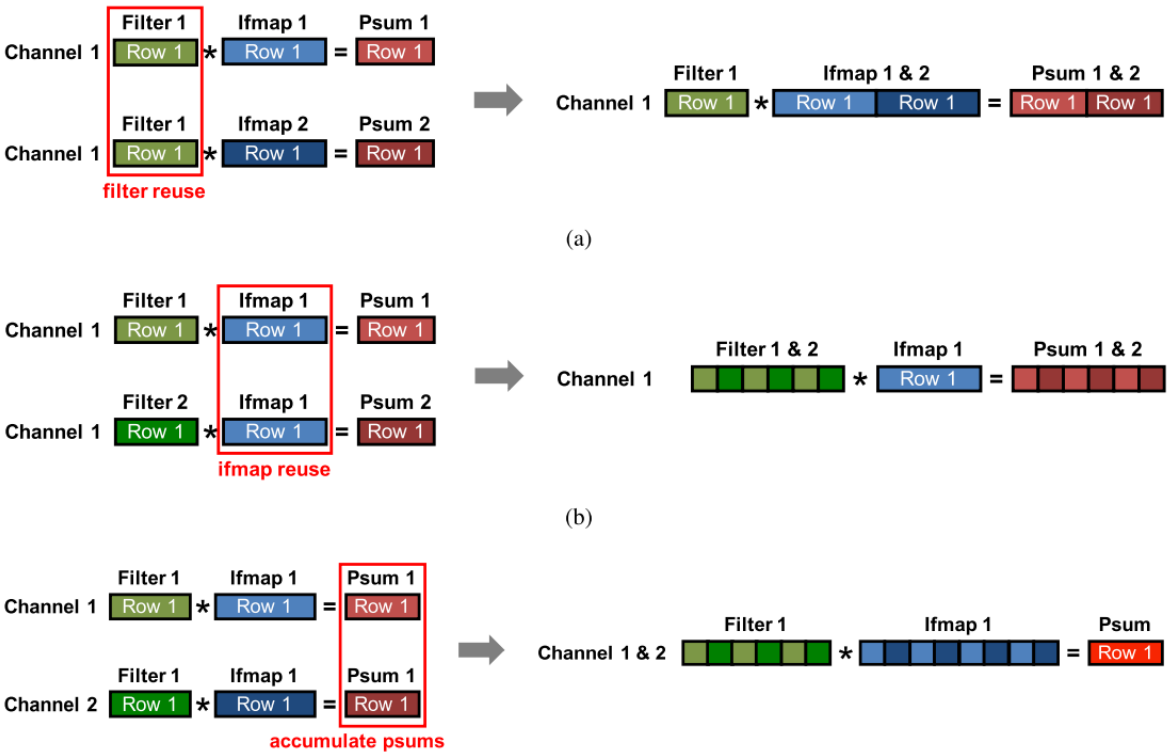
thành bởi vì sự có mặt của 3 yếu tố là batch size, số lượng channel và số lượng kernel weight. 2-D convolution sử dụng:

- Các input feature map khác nhau sử dụng cùng một kernel weight (kernel reuse)
- Các kernel weight khác nhau sử dụng cùng input feature map (input feature map reuse)
- Các kernel và input feature map từ các channel khác nhau cộng tích lũy kết quả tổng riêng phần với nhau (partial sum accumulate).

Việc tái sử dụng kernel có thể đạt được bằng cách cho đi ngang qua nhiều input feature map ở cùng một PE set một cách tuần tự.

Việc tái sử dụng input feature map và tích lũy tổng riêng phần có thể được khai thác bằng cách tận dụng local spad hoặc tính chất song song hóa độc lập của PE array.

- Multiple 2-D Convolutions in a PE Set: Mỗi PE set được chạy nhiều tác vụ 2-D convolution ở kernel và channel khác nhau.
- Multiple PE Sets in the PE Array: PE array có thể chạy nhiều hơn một PE set nếu PE set đủ nhỏ.



Hình 3.8: Xử lý kích thước ngoài 2-D trong mỗi PE [21]

3.2.3.3 Origami: A 803 GOp/s/W Convolutional Network Accelerator

Ý tưởng: Quá trình tính toán như sau [22]:

- Dữ liệu đầu vào được đưa Image Window SRAM với kích thước được cài đặt sẵn và cách một khoảng bằng stripe, và đảm bảo vị trí của spatial window của dữ liệu hình ảnh đầu vào. (Đơn

giản là load đoạn input feature map tiếp theo cho kernel)

- Sau đó, dữ liệu được tải vào Image Bank, mà ở đó một phần input feature map có kích thước nhỏ hơn kernel được giữ lại ở register và di chuyển xuống trên giá trị stripe trước khi sang cột tiếp theo. Image Bank cung cấp đầu vào cho các đơn vị SoP để thực hiện các tính toán bên trong cho từng kernel.
- Mỗi đơn vị SoP thực hiện tính toán với cùng channel của input feature map ở các kernel khác nhau, do đó cho ra kết quả là tổng riêng phần của các output feature map ở các channel khác nhau.
- Hệ thống lặp lại tính toán trên ở tất cả channel của input feature map trong khi các tổng riêng phần được tính lũy ở đơn vị ChSum để tạo ra kết quả cuối cùng. Sau đó, kết quả này được đưa ra khỏi Origami.

3.2.3.4 ShiDianNao: Shifting Vision Processing Closer to the Sensor

Convolution Layer

Ý tưởng: Ở layer này, hệ thống liên tục thực hiện tính toán cho một output feature map và không chuyển sang output feature map tiếp theo cho tới khi tính xong feature map hiện tại.

Pooling Layer

Ý tưởng: Khi ở layer này, việc tính toán tương tự như Convolution layer, hệ thống liên tục thực hiện tính toán cho một output feature map và không chuyển sang output feature map tiếp theo cho tới khi tính xong feature map hiện tại. Với mỗi tính toán cho từng output feature map, các PE được gắn cho một đơn vị output feature map và chỉ thực hiện phép toán trên đó tới khi hoàn thành.

Fully-Connected Layer

Ý tưởng: Khi ở layer này, các PE được gắn cho một đơn vị output feature map và chỉ thực hiện phép toán trên đó tới khi hoàn thành mà không chuyển sang đơn vị khác.

3.2.4 Công trình khác

3.2.4.1 BRein Memory: A Single-Chip Binary/Ternary Reconfigurable in-Memory Deep Neural Network Accelerator Achieving 1.4 TOPS at 0.6 W

Ý tưởng:

- Cách tiếp cận là single-chip in-memory, nghĩa là tất cả thông số của DNN được lưu trữ ở on-chip SRAMs và tất cả tính toán được thực thi bằng logic unit ở gần SRAMs.
- Đặc điểm quan trọng:
  - Tích hợp các mạch xử lý hài hòa với bản chất của SRAM
  - Phương pháp nén tham số của DNN để đủ lưu trữ ở on-chip SRAM
  - Khai thác tối đa song song hóa intra-word và che giấu hành vi tuần tự của truy xuất inter-word

Chương 4

THIẾT KẾ BỘ TĂNG TỐC

4.1 Giới thiệu tổng quát

Bộ tăng tốc – được gọi là Alpha Accelerator – được thiết kế để tăng tốc quá trình tính toán của toàn bộ hệ thống nói chung và tác vụ đặc thù của mô hình Deep Learning nói riêng.

Alpha Accelerator có các tính năng như sau:

- Hỗ trợ tính toán các tác vụ Deep Learning theo cấp Layer. Nghĩa là một chu trình tính toán của bộ tăng tốc nhận tất cả giá trị đầu vào của một Layer nhất định và cho ra kết quả là giá trị đầu ra của chính Layer đó. Hiện tại, Alpha Accelerator hỗ trợ tính toán trên:
  - Ba loại Deep Learning Layer:
    - Fully-Connected Layer
    - Convolution Layer
    - Max Pooling Layer
  - Hai loại Activation Function:
    - ReLU
    - ReLU6
- Hỗ trợ hoàn toàn cho các mô hình được rút gọn dựa trên phương pháp 8-bit Integer Quantization. Việc hỗ trợ này bao gồm việc tính toán và đọc dữ liệu một cách hiệu quả.
- Alpha Accelerator cung cấp thêm khả năng đọc dữ liệu, tính toán và trả lại kết quả theo cơ chế Pipeline bao gồm 3 Stage là RDATA, COMPS và WBACK.

4.2 Thiết kế chức năng

Đầu tiên, ta đặt một vài điều kiện tiên quyết như sau. Trong một chu kỳ tính toán, bộ tăng tốc cho phép tính toán với khả năng:

- Số lượng đầu vào (Input Activation đối với Fully-Connected Layer hoặc Input Feature Map đối với Convolution Layer) tối đa cho phép là:

  $$N_0 \times N_2 \times N_3$$

- Số lượng trọng số (Weight đối với Fully-Connected Layerr hoặc Kernel đối với Convolution Layer) tối đa cho phép là:

  $$N_0 \times N_1 \times N_2 \times N_3$$

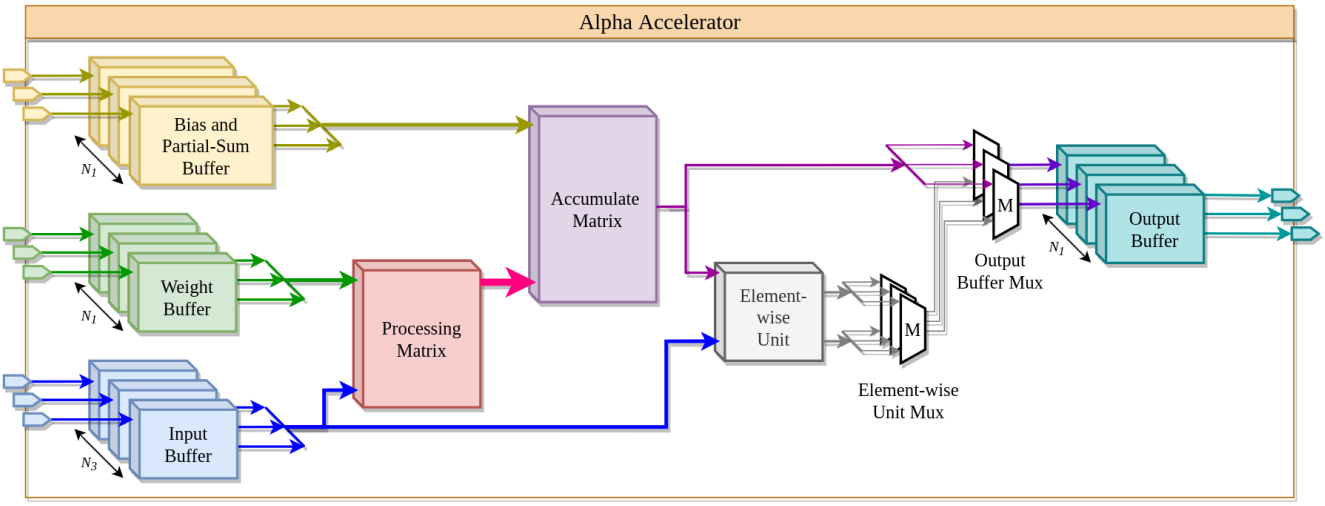
- Số lượng giá trị (Bias) hoặc (Partial-Sum) tối đa là:

  $$N_1$$

với hằng số N_0, N_1, N_2 và N_3 là các số nguyên dương. Thông tin cụ thể về các giả thiết này sẽ được giải thích cụ thể ở các phần sau.

4.2.1 Kiến trúc tổng quát

Kiến trúc tổng quát của Alpha Accelerator được thể hiện ở hình bên dưới.



Hình 4.1: Sơ đồ khối về kiến trúc của bộ tăng tốc

Dựa trên tác vụ, các khối chức năng của bộ tăng tốc có thể được phân chia thành 3 nhóm chính:

- Nhóm phục vụ tác vụ tính toán. Nhóm này bao gồm các khối:
  - *Processing Matrix*: Đây là khối quan trọng nhất của bộ tăng tốc. Nhiệm vụ của khối là thực hiện các tính toán MAC – nhân và cộng tích lũy – để tạo ra tổng riêng phần hoặc tổng hoàn chỉnh đối với 2 loại Layer là Fully-Connected Layer và Convolution Layer.

    Đối với Convolution Layer, điểm đáng chú ý nhất của Processing Matrix là cho phép tính

toán theo cấp độ 3D vì cấu trúc 3D ở Input Feature Map và Kernel. Thêm nữa, tác vụ chiếm vai trò trọng yếu của Layer này là việc tính toán phép tích chập nên việc khai thác tính chất 3D này là cần thiết và khối này gần như đóng vai trò gần như quan trọng nhất để cải thiện hiệu suất hoạt động ở Convolution Layer. Ngoài ra, việc tận dụng tính chất 3D còn góp phần làm giảm số lượng tổng riêng phần cần tính để cho ra kết quả cuối cùng từ đó cải thiện việc lưu trữ và tải lại các tổng riêng phần này.

Đối với Fully-Connected Layer, bởi vì tính toán chỉ thực hiện trên 2D và tác vụ chiếm vai trò trọng yếu của Layer này là việc đọc bộ nhớ nên khối này không đóng vai trò quát quan trọng trong việc cải thiện hiệu suất hoạt động ở Fully-Connected Layer.

Alpha Accelerator chỉ có duy nhất 1 khối *Processing Matrix*.

– *Accumulate Matrix*: Đối với *Processing Matrix*, sau khi thực hiện xong tác vụ tính toán MAC thì trả về các kết quả rời rạc là các tổng riêng phần của kết quả đầu ra. Các tổng riêng phần này có thể có mối liên hệ với nhau và là một phần của một số kết quả hoàn chỉnh. Do đó, nhiệm vụ của khối này là để cộng lại các kết quả từ *Processing Matrix* để tạo thành các tổng riêng phần lớn hơn mà mỗi một tổng riêng phần chỉ thuộc về duy nhất một kết quả hoàn chỉnh của Layer.

Alpha Accelerator chỉ có duy nhất 1 khối *Accumulate Matrix*.

– *Element-Wise Unit*: Khối này đóng vai trò thực hiên các phép toán đơn nguyên – phép toán thực hiện trên một đơn vị tính toán – của Layer. Các tác vụ này bao gồm:

∗ Làm tròn kết quả tính toán. Kết quả sau khi hoàn thành tính toán có kích thước 32-bit cần được nhân tham số làm tròn và đưa về dạng 8-bit phục vụ cho việc cho ra kết quả hoặc là đầu vào cho các Layer tiếp theo.

∗ Các hàm kích hoạt ở Fully-Connected Layer và Convolution Layer.

∗ Các phép tính toán đặc thù ở Pooling Layer như là phép so sánh hoặc phép nhân.

Alpha Accelerator chỉ có duy nhất 1 khối *Element-Wise Unit*.

• Nhóm phục vụ tác vụ đọc dữ liệu. Nhóm này bao gồm các khối:

– *Bias and Partial-Sum Buffer*: Đây là bộ nhớ đệm với kích thước nhỏ để lưu trữ các giá trị độ lệch – Bias và các giá trị tổng riêng phần – Partial-Sum phục vụ cho việc tính toán ở chu kỳ làm việc hiện tại.

Nói về tổng riêng phần, đối với các Layer có kích thước quá lớn, bộ tăng tốc phải chia nhỏ việc tính toán ra chứ không tính toán toàn bộ giá trị trong một chu kỳ làm việc duy nhất. Các kết quả tạm thời trong chu kỳ làm việc được gọi là tổng riêng phần cần được lưu trữ ở bên ngoài và cần được tải vào các tính toán liên quan để tạo ra tổng hoàn chỉnh và cho ra kết quả cuối cùng.

Alpha Accelerator có N_1 khối *Bias and Partial-Sum Buffer*.

  - *Weight Buffer*: Đây là bộ nhớ đệm với kích thước nhỏ để lữu các giá trị trọng số của mô hình như là Weight (đối với Fully-Connected Layer) hoặc Kernel (đối với Convolution Layer).

    Alpha Accelerator có N_1 khối *Weight Buffer*.

  - *Input Buffer*: Đây là bộ nhớ đệm với kích thước nhỏ để lưu trữ các giá trị đầu vào của Layer như Input Activation (đối với Fully-Connected Layer) hoặc Input Feature Map (đối với Convolution Layer).

    Alpha Accelerator có N_3 khối *Input Buffer*.

- Nhóm phục vụ tác vụ ghi dữ liệu. Nhóm này chỉ gồm khối chức năng là *Output Buffer*. Đây là bộ nhớ đệm với kích thước nhỏ để lưu trữ các giá trị tổng riêng phần hoặc kết quả hoàn chỉnh (sau khi đã thực hiện các hàm kích hoạt) của Layer. Các giá trị này sau đó sẽ được ghi và lưu trữ ở bộ nhớ bên ngoài.

  Alpha Accelerator có N_1 khối *Output Buffer*.

Ngoài các thành phần trên, còn có các thành phần khác phục vụ cho việc điều chỉnh luồng tính toán. Cụ thể như sau:

- *Output Buffer Mux*: Nhóm Multiplexer phục vụ việc lựa chọn giá trị đầu vào là từ *Accumulate Matrix* – các giá trị tổng riêng phần hay là từ *Element-Wise Unit* – các giá trị kết quả hoàn chỉnh của Layer.

  Alpha Accelerator có N_1 bộ *Output Buffer Mux*.

- *Element-Wise Unit Mux*: Nhóm Multiplexer phục vụ việc lựa chọn giá trị đầu ra của *Element-Wise Unit*. Các đầu ra này hoặc là kết quả hoàn chỉnh sau khi đã tính toán, làm tròn và qua các hàm kích hoạt ở Fully-Connected Layer hoặc Convolution Layer hoặc là kết quả hoàn chỉnh sau khi đã tính toán ở Pooling Layer.

  Alpha Accelerator có N_1 bộ *Element-Wise Unit Mux*.

4.2.2 Thiết kế các khối chức năng

4.2.2.1 Nhóm phục vụ tác vụ tính toán

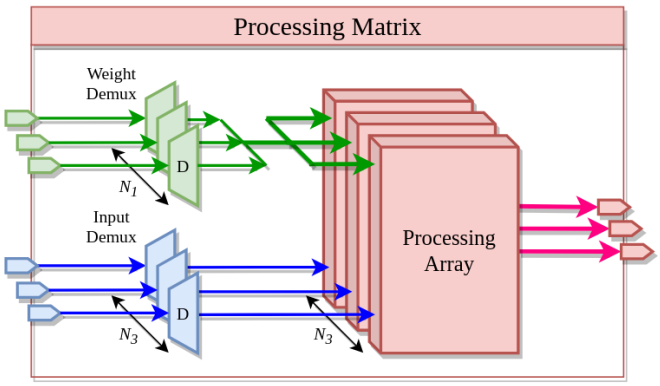
A. Processing Matrix

Processing Matrix làm khối thực hiện các tác vụ tính toán chính của bộ tăng tốc. Việc tính toán sao cho hiệu quả là yêu cầu tối quan trọng cho khối chức năng này.

Nhắc lại, trong Convolution Layer, Processing Matrix khai thác đặc tính 3D trong phép tích chập giữa Input Feature Map và Kernel. Điều này đồng nghĩa với việc cho phép tính toán cùng lúc các giá trị theo 3 chiều dữ liệu, từ đó làm giảm số lượng tổng riêng phần cần tính để cho ra kết quả cuối cùng từ đó giảm số lần lưu trữ và tải lại các tổng này. Còn đối với Fully-Connected Layer, các tính toán chỉ dừng lại ở chiều dữ liệu thứ 2 nên việc khai thác đặc tính 3D này không có nhiều ý nghĩa mà thay vào đó là cho phép việc tính toán thêm dữ liệu ở chiều thứ 2. Tuy nhiên, Layer này không cần phải tính

toán quá nhiều mà thay vào đó là tác vụ đọc dữ liệu mới chiếm vai trò tối quan trong.

Kiến trúc tổng quát của Processing Matrix được thể hiện ở hình bên dưới.



Hình 4.2: Sơ đồi khối về kiến trúc của khối Processing Matrix

Khối Processing Matrix bao gồm:

- *Processing Array*: Khối này đóng vai trò thực hiện các tính toán cấp 2D trong các tính toán cấp 3D của Processing Matrix.

  Processing Matrix có N_3 khối *Processing Array*.

- *Weight Demux*: Nhóm Demultiplexer phục vụ việc đưa các giá trị trọng số của mô hình vào cho *Processing Array*. Nguyên nhân của sự xuất hiện của nhóm này là vì số lượng bộ nhớ đệm bên ngoài ít hơn rất nhiều so với số lượng đơn vị tính toán bên trong.

  Điểm đáng chú ý nhất ở nhóm này là việc để có đầy đủ dữ liệu cho tính toán của một *Processing Array* cần sự kết hợp của tất cả đầu ra của các Demultiplexer. Điều này có nghĩa là dữ liệu từ tất cả *Weight Buffer* mới đủ cung cấp cho một *Processing Array*.

  Processing Matrix có N_1 bộ *Weight Demux*.

- *Input Demux*: Nhóm Demultiplexer phục vụ việc đưa các giá trị đầu vào của Layer vào cho *Processing Array*. Nguyên nhân của sự xuất hiện của nhóm này là vì số lượng bộ nhớ đệm bên ngoài ít hơn rất nhiều so với số lượng đơn vị tính toán bên trong.

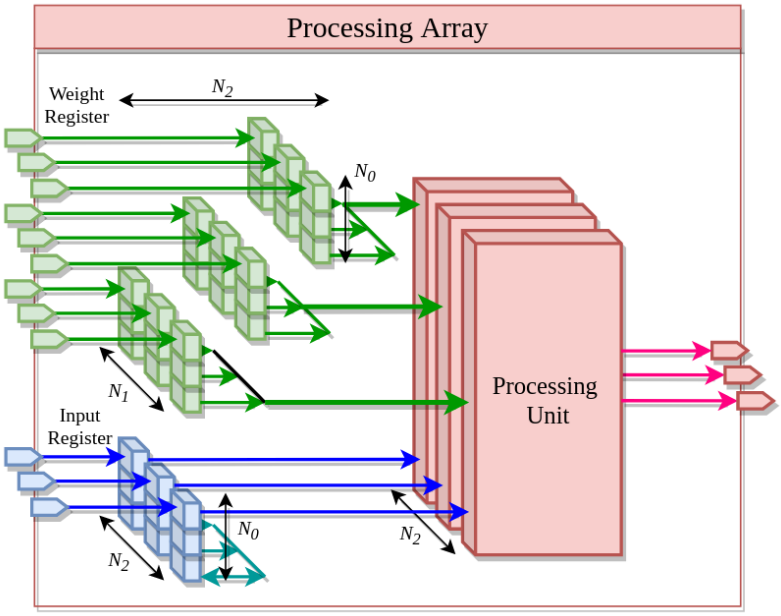
  Điểm đáng chú ý nhất ở nhóm này là mỗi một Demultiplexer đi theo thành cặp với một khối *Processing Array*. Điều này có nghĩa là dữ liệu cần để tính toán cho một *Processing Array* chỉ cần từ một *Input Buffer* là đủ.

  Processing Matrix có N_3 bộ *Input Demux*.

a. Processing Array

Processing Array là khối thực hiện tác vụ MAC ở cấp 2D. Tuy nhiên, khối này vẫn chưa phải là đơn vị tính toán cơ bản nhất.

Kiến trúc tổng quát của Processing Array được thể hiện ở hình bên dưới.



Hình 4.3: Sơ đồ khối về kiến trúc của khối Processing Array

Khối Processing Array bao gồm:

- *Processing Unit*: Khối này đóng vai trò thực hiện các tính toán cấp 1D trong các tính toán cấp 2D của Processing Array.

  Processing Array có N_2 khối *Processing Unit*.

- *Weight Register*: Các thanh ghi này đóng vai trò lưu trữ các giá trị trọng số của mô hình cần cho quá trình thực hiện tác vụ MAC.

  Processing Array có $N_2 \times N_1$ *Weight Register*. Mỗi thanh ghi có kích thước lưu trữ N_0 giá trị 8-bit.

- *Input Register*: Các thanh ghi này đóng vai trò lưu trữ các giá trị đầu vào của Layer cần cho quá trình thực hiện tác vụ MAC. Để tận dụng tích cục bộ của Input Feature Map (ở Convolution Layer), các thanh ghi nên có thể chia sẻ thông tin với nhau để làm giảm số lượng giá trị mới cần được đọc vào.

  Processing Array có N_2 *Input Register*. Mỗi thanh ghi có kích thước lưu trữ N_0 giá trị 8-bit.

Thêm nữa, mỗi Processing Unit tạo thành cặp với 1 Input Register và N_1 Weight Register. Điều này nghĩa là trong mỗi chu kỳ tính toán của Processing Unit, cho phép tính cùng lúc 1 giá trị đầu vào và N_1 giá trị trọng số. Và cũng có nghĩa là tái sử dụng giá trị đầu vào cho các tính toán với các trọng số khác nhau. Ta xem xét từng loại Layer:

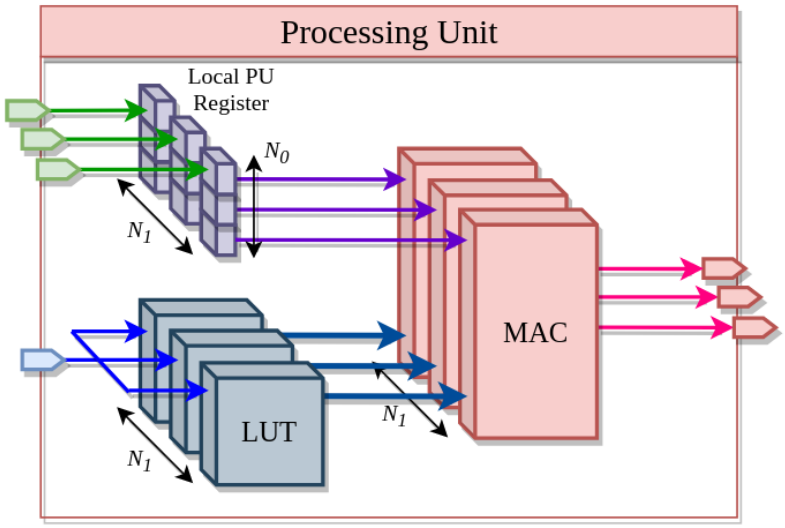
- Đối với Convolutional Layer, phép tích chập được thực hiện tính ở cùng một Input Feature Map với nhiều bộ Kernel để tạo ra Ouptut Feature Map. Do đó, ta hoàn toàn có thể tính toán cùng lúc một bộ giá trị đầu vào Input Feature Map với nhiều bộ giá trị trọng số Kernel.
- Đối với Fully-Connected Layer, để tạo ra một kết quả hoàn chỉnh Output Activation thì cần sự

tham giá của tất cả giá trị đầu vào Input Activation khi thực hiện phép nhân và cộng tích lũy với tất cả giá trị trọng số Weight. Do đó, mỗi một bộ giá trị đầu vào cần phải thực hiện tính toán với nhiều bộ giá trị trọng số để cho ra kết quả cuối cùng. Do đó, ta cũng hoàn toàn có thể tính toán cùng lúc một bộ giá trị đầu vào Input Activation với nhiều bộ giá trị trọng số Weight.

b. Processing Unit

Processing Unit là khối thực hiện tác vụ MAC ở cấp 1D. Tuy nhiên, khối này vẫn chưa phải là đơn vị tính toán cơ bản nhất.

Kiến trúc tổng quát của Processing Unit được thể hiện ở hình bên dưới.



Hình 4.4: Sơ đồ khối về kiến trúc của khối Processing Unit

Khối Processing Unit bao gồm:

- *Local PU Register*: Các thanh ghi này đóng vai trò lưu trữ các giá trọng số của mô hình cần cho quá trình thực hiện tác vụ tính toán. Nguyên nhân tồn tại đồng thời bộ thanh ghi này và bộ thanh ghi *Weight Register* ở bên ngoài là để phục vụ cho việc Pipeline.

  Processing Unit có N_1 *Weight Register*. Mỗi thanh ghi có kích thước lưu trữ N_0 giá trị 8-bit.

- *Look-Up Table – LUT*: Đây là thành phần cần thiết để hỗ trợ cho việc tính toán theo mô hình Bit-Serial.

  Processing Unit có N_1 khối *LUT*.

- *Multiply–ACcumulate – MAC*: Khối này đóng vai trò là đơn vị tính toán cơ bản nhất với khả năng thực hiện tác vụ MAC ở một thời điểm là N_0 giá trị đầu vào và N_0 giá trị trọng số. Mô hình tính toán được sử dụng là mô hình Bit-Serial.

  Processing Unit có N_1 khối *MAC*.

c. Mô hình tính toán Bit-Serial

Đầu tiên, bản chất của phép nhân giữa hai số nguyên với nhau là thực hiện nhiều phép cộng và phép

dịch bit với nhau. Hình bên dưới là ví dụ về phép nhân giữa hai số nguyên dưới dạng nhị phân:

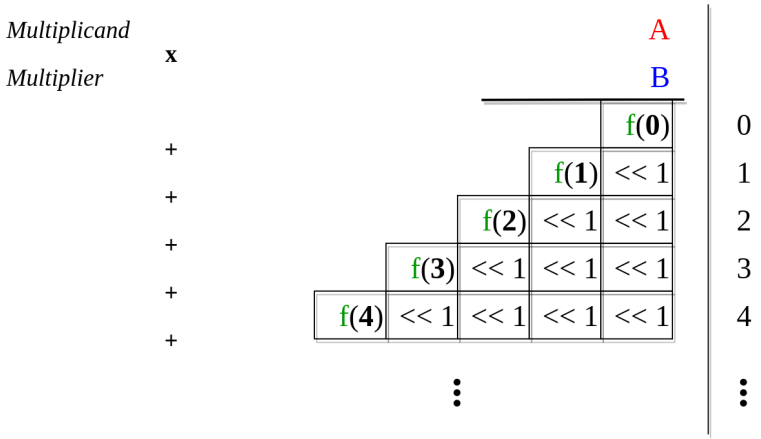
Multiplicand		101101_2	
Multiplier	x	011110_2	
		000000	0
		1011010	1
		10110100	1
		101101000	1
		1011010000	1
		000000000000	0
Product		10101000110_2	

Hình 4.5: Ví dụ về phép nhân giữa hai số nguyên nhị phân 6-bit với nhau

Để thực hiện phép nhân trên, ta lần lượt đi qua từng bit của số nhân 011110_2 theo chiều từ bit có trọng số thấp nhất tới bit có trọng số cao nhất. Bit 0 có giá trị là 0 nên tạo ra số hạng đầu tiên có giá trị là 000000_2. Bit 1 có giá trị là 1 nên tạo ra số hạng thứ hai có giá trị bằng số bị nhân là 101101_2 dịch sang trái 1 bit là 1011010_2. Bit 2 có giá trị là 1 nên tạo ra số hạng thứ ba có giá trị bằng số bị nhân là 101101_2 dịch sang trái 2 bit là 10110100_2. Quá trình này tiếp tục, nếu bit thứ n có giá trị là 0 thì số hạng thu được là 0, ngược lại nếu bit thứ n có giá trị là 1 thì số hạng là số bị nhân 101101_2 được dịch sang trái n bit. Sau đó, tất cả số hạng này được cộng lại với nhau tạo ra kết quả cuối cùng của phép nhân.

Bởi vì biểu diễn số nhị phân chỉ cần hai bit 0 và 1, số hạng trong kết quả cuối cùng của phép nhân có thể là 0 hoặc là số bị nhân được dịch trái một số bit nhất định. Ta có thể tổng quát hóa quá trình tính toán phép nhân thành một ví dụ như bên dưới.

f(i) = (B[i] == 1) ? A : 0



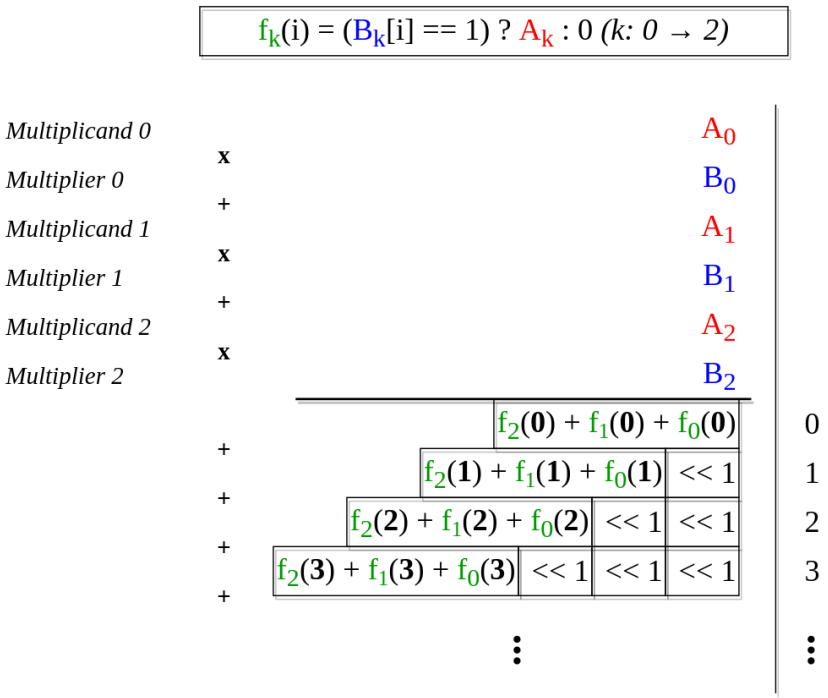
Hình 4.6: Tổng quát về phép nhân giữa hai số nguyên bất kỳ với nhau

với hàm số $f(i)$ trả về kết quả là A nếu giá trị bit thứ i của B là 1 và là 0 nếu giá trị bit thứ i của B là 0.

Để tính toán, ta cần liên tục đi qua các bit từ trọng số thấp nhất cho tới cao nhất của số nhân và cộng dồn các giá trị số hạng thu được phụ thuộc vào các bit đó để tạo ra kết quả cuối cùng. Do đó, phương pháp này mới có tên là Bit-Serial. Nếu kích thước của số nhân là n bit thì phương pháp này cần đúng n chu kỳ để cho ra kết quả cuối cùng.

Không dừng lại ở đó, nếu ta có thể mở rộng nguyên lý tính toán trên để dùng cho phép MAC như sau. Giả sử, ta có ba đối số bị nhân là A_0, A_1 và A_2 với ba số nhân là B_0, B_1 và B_2 đi kèm với các hàm số định trị của số hàng là $f_0(i)$, $f_1(i)$ và $f_2(i)$. Khi đó, ta có thể dùng phương pháp Bit-Serial để tính toán giá trị cho phương trình sau:

$$sum = A_0 \times B_0 + A_1 \times B_1 + A_2 \times B_2$$



Hình 4.7: Tổng quát về phép nhân và cộng tích lũy giữa ba cặp số nguyên bất kỳ với nhau

Nếu ta thay hàm $g(\{B_2[i], B_1[i], B_0[i]\}) = f_2(i) + f_1(i) + f_0(i)$ và tính toán sẵn các kết quả này theo dạng bảng như sau.

$\{B_2[i], B_1[i], B_0[i]\}$	**Giá trị**
111	$A_2 + A_1 + A_0$
110	$A_2 + A_1$
101	$A_2 + A_0$
100	A_2
011	$A_1 + A_0$
010	A_1
001	A_0
000	0

thì toàn hoàn toàn có thể tính toán được kết quả sum trong vòng đúng n chu kỳ. Việc này mang tới

hiệu quả tính toán rất lớn nếu ta lựa chọn phù hợp trong việc đánh tài nguyên giữa Look-Up Table và số lượng cặp số tính toán và số chu kỳ tính toán.

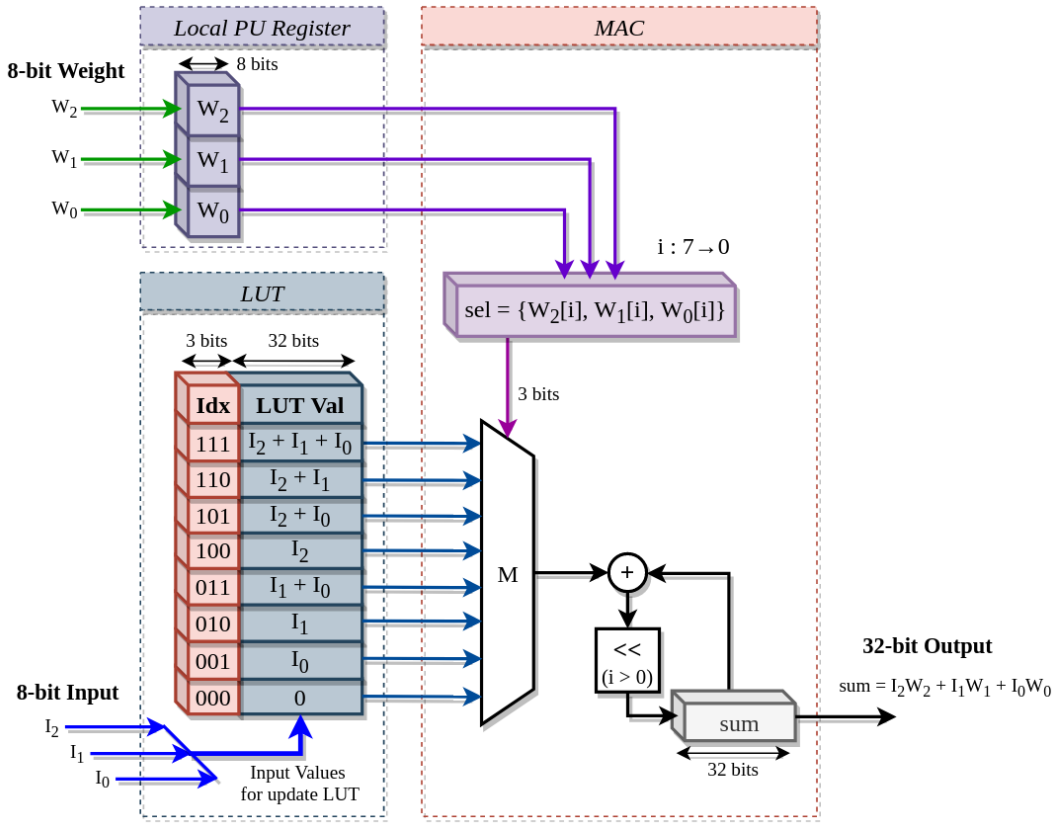
d. Hiện thực mô hình Bit-Serial

Dựa theo đúng mô hình tính toán Bit-Serial, ta xây dựng các phần cứng cần thiết phục vụ cho việc tính toán MAC. Mỗi một bộ đơn vị tính toán cơ bản bao gồm 3 thành phần lần lượt là:

1. *Local PU Register*: Thanh ghi lưu trữ các giá trị trọng số phục vụ cho việc Pipeline. Mỗi thanh ghi có kích thước lưu trữ là N_0 giá trị 8-bit.
2. *LUT*: Bảng giá trị tính toán sẵn các giá trị có thể có của hàm g theo lý thuyết mô hình tính toán Bit-Serial.
3. *MAC*: Đơn vị tính toán và điều khiển khi nó thực hiện quá trình dịch bit ở số nhân, cộng kết quả từ bảng giá trị và kết quả tích lũy ở thanh ghi sum, dịch trái 1 bit kết quả thu được (trừ chu kỳ cuối cùng là không dịch bit) để cho ra kết quả cuối cùng sau 8 chu kỳ.

   Điểm đặc biệt ở đây là việc thứ tự các bit theo quá trình tính toán là chiều ngược lại (từ 7 về 0) và phép toán dịch trái 1 bit được thực hiện sau khi đã cộng tích lũy chỉ trừ chu kỳ cuối. Nếu xem xét kỹ thì quá trình này hoàn toàn giống với quá trình tính toán đã đề xuất ở phần trước. Xét bit 7, được thực hiện tính toán ở chu kỳ đầu tiên, sau mỗi chu kỳ dịch một bit chỉ trừ chu kỳ cuối thì sau 8 chu kỳ, kết quả từ bảng vẫn được dịch trái đúng 7 bit.

Bên dưới là kiến trúc của đơn vị tính toán được hiện thực với giá trị $N_0 = 3$.

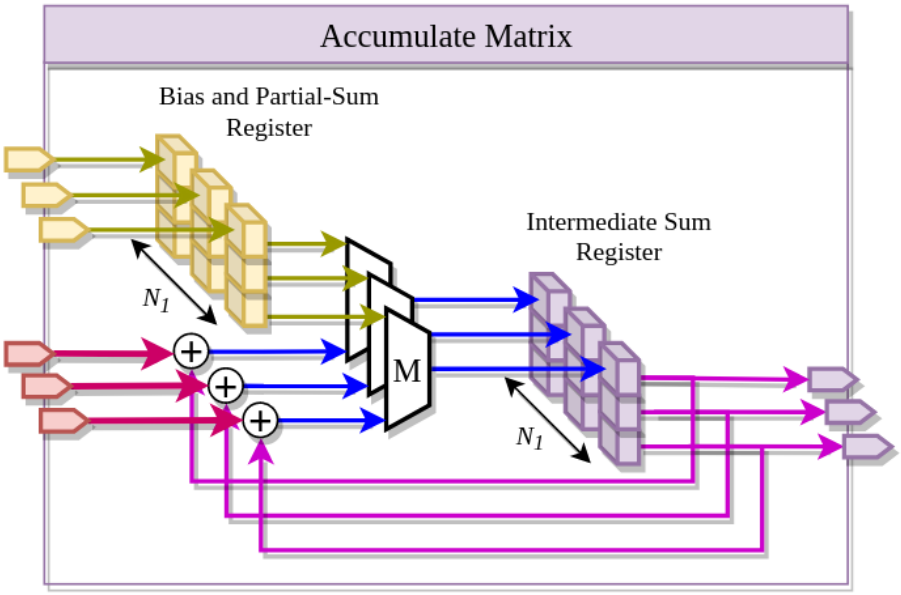


Hình 4.8: Sơ đồ nguyên lý đơn vị tính toán MAC với giá trị $N_0 = 3$

B. Accumulate Matrix

Accumulate Matrix đóng vai trò thực hiện việc cộng các tổng riêng phần thu được ở mỗi đơn vị tính toán lại với nhau. Việc cộng này chỉ được áp dụng trên các tổng riêng phần thuộc về cùng một kết quả đầu ra nhất định của Layer. Thêm nữa, khối này còn đóng vai trò đọc các gía trị đối số nếu đây là lần tính toán đầu tiên các tổng riêng phần thuộc về kết quả đầu ra và đọc các tổng riêng phần trước đó thuộc về kết quả đầu ra ở những lần tính toán tiếp theo.

Kiến trúc tổng quát của Accumulate Matrix được thể hiện ở hình bên dưới.



Hình 4.9: Sơ đồ khối về kiến trúc của khối Accumulate Matrix

Accumulate Matrix bao gồm:

- *Bias and Partial-Sum Register*: Bộ thanh ghi phục vụ việc lưu trữ tạm thời các giá trị đối số hoặc tổng riêng phần phục vụ cho chu kỳ tính toán tiếp theo.

  Accumulate Matrix có N_1 bộ *Bias and Partial-Sum Register*.

- *Intermediate Sum Register*: Bộ thanh ghi phục vụ việc lưu trữ tạm thời kết quả tính toán các tổng riêng phần và hỗ trợ cho việc Pipeline.

  Accumulate Matrix có N_1 bộ *Intermediate Sum Register*.

- Bộ cộng và bộ Multiplexer: Nhiệm vụ của bộ Multiplexer là để lựa chọn quá trình hiện tại là đọc giá trị khởi đầu cho tổng riêng phần (giá trị đối số hoặc các tổng riêng phần trước đó) hay là cộng tích lũy giữa giá trị hiện thời của thanh ghi với kết quả thu được từ *Processing Matrix*.

  Accumulate Matrix có N_1 bộ cộng và bộ Multiplexer.

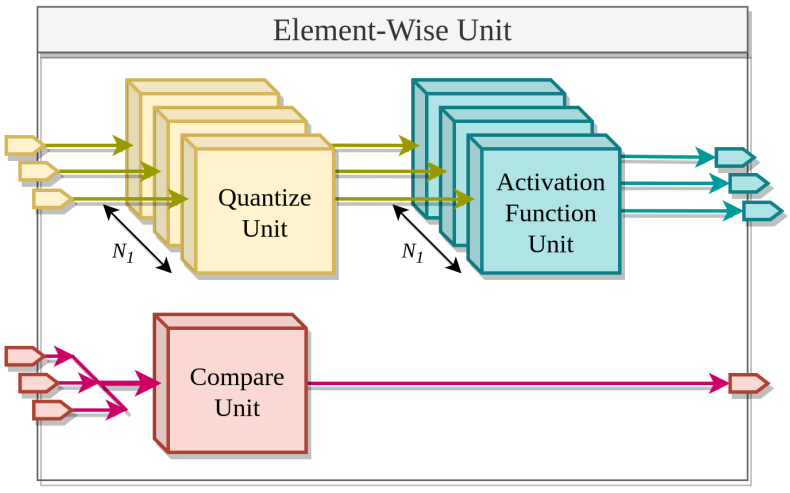
Mỗi thành phần đơn vị của Accumulate Matrix gồm 1 bộ cộng, 1 bộ Multiplexer, 1 thanh ghi *Bias and Partial-Sum* và 1 thanh ghi *Intermediate Sum* làm việc với nhau và không có bất kỳ liên hệ gì với các đơn vị khác.

C. Element-Wise Unit

Element-wise Unit đóng vai trò hực hiên các phép toán đơn nguyên – phép toán thực hiện trên một đơn vị tính toán (có thể là đầu vào hoặc kết quả) – của Layer. Cụ thể như sau:

- Activation Function: Các hàm kích hoạt phi tuyến cần thiết đối với kết quả sau khi hoàn thành tất cả các tính toán MACs. Hiện tại, các hàm kích hoạt được hiện thực là:
  - ReLU
  - ReLU6
- Quantize: Việc sử dụng mô hình được làm tròn ở 8-bit đối với cả input và weight, việc Quantize là bắt buộc cần thiết để đưa kết quả từ 32-bit về 8-bit phục vụ cho các tính toán tiếp theo. Quá trình làm tròn được thực hiện theo đúng đặc tả, đầu tiên là nhân với hệ số M_0, sau đó dịch phải n bit để tạo thành kết quả ước lượng khi nhân với số thực dấu chấm động M.
- Phép so sánh ở Max Pooling Layer: Về lý thuyết, đối với phép toán này, ta dùng một thanh ghi tạm thời lưu trữ giá trị lớn nhất. Giá trị mới được đưa vào sẽ so sánh với các giá trị tạm này, nếu kết quả lớn hơn thì giá trị đó sẽ được ghi lại vào trong thanh ghi, ngược lại thì không có gì xảy ra. Ở cấp hiện thực, ta có thể đồng thời so sánh và lưu trữ nhiều giá trị đầu vào cùng lúc để tăng hiệu suất tính toán.
- Phép chia ở Average Pooling Layer: Với phép toán này, ta cần phải cộng tất cả giá trị đầu vào trong phạm vi tính toán. Sau đó, tiến hành chia để lấy giá trị trung bình cộng. Hiện tại, Element-wise Unit không hỗ trợ cho tác vụ này.

Kiến trúc tổng quát của Element-Wise Unit được thể hiện ở hình bên dưới.



Hình 4.10: Sơ đồ khối về kiến trúc của khối Element-Wise Unit

Element-Wise Unit bao gồm:

- *Quantize Unit*: Khối này đóng vai trò thực hiện quá trình làm tròn giá trị tổng cuối cùng trước khi thực hiện các hàm phi tuyến. Quá trình làm tròn của khối này tuân theo lý thuyết làm tròn đã được đề cập ở chương trước.

Element-Wise Unit có N_1 khối *Quantize Unit*.

- *Activation Function Unit*: Khối này thực hiện quá trình chạy các hàm phi tuyến để cho ra kết quả đầu ra của Layer.

  Element-Wise Unit có N_1 khối *Activation Function Unit*.

- *Compare Unit*: Khối chức năng chỉ phục vụ cho Max Pooling Layer với khả năng so sánh cùng lúc N_3 giá trị đầu vào.

  Element-Wise Unit chỉ có 1 khối *Compare Unit*.

Để ý rằng, mỗi khối *Quantize Unit* đều đi thành cặp và làm việc với khối *Activation Function Unit*. Nguyên nhân là vì 2 tác vụ này liên quan mật thiết và không thể tách rời nhau.

Bộ *Element-Wise Unit Mux* ở phần đầu tiên được dùng để lựa chọn xem đầu ra hoặc là kết quả sau khi làm tròn và thực hiện hàm phi tuyến từ khối *Activation Function Unit* hoặc là kết quả so sánh từ khối *Compare Unit*.

4.2.2.2 Nhóm phục vụ tác vụ đọc dữ liệu

A. Bias and Partial-Sum Buffer

Bias and Partial-Sum Buffer đóng vai trò là bộ nhớ đệm để lưu trữ các giá trị độ lệch hoặc tổng riêng phần được lưu trữ ở bộ nhớ ngoài để phục vụ cho việc tính toán ở chu kỳ làm việc hiện tại.

Các giá trị độ lệch hoặc tổng riêng phần đều có kích thước là 32-bit và ở mỗi chu kỳ tính toán, mỗi giá trị này chỉ xuất hiện với tần suất tương đối thấp. Do đó, kích thước của bộ nhớ đệm này chỉ cần 32-bit là đủ để phục vụ tính toán.

Về quá trình đọc bộ nhớ, Bias and Partial-Sum Buffer cần đúng một lần đọc duy nhất để hoàn thành quá trình đọc của mình khi chỉ cần lưu trữ đúng một đơn vị từ một lần đọc 32-bit dữ liệu ở bộ nhớ ngoài. Có tổng cộng N_1 Bias and Partial-Sum Buffer, nên cần tiêu tốn N_1 lần đọc để chuẩn bị đủ các giá trị trọng số cần thiết phục vụ cho tính toán.

B. Weight Buffer

Weight Buffer đóng vai trò là bộ nhớ đệm để lưu trữ các giá trị trọng số của mô hình ở bộ nhớ ngoài để phục vụ cho việc tính toán ở chu kỳ làm việc hiện tại. Weight Buffer chỉ được dùng với hai loại Layer là Fully-Connected và Convolution.

Về kích thước, bộ nhớ đệm này phải đảm bảo được lưu trữ $N_0 \times N_2$ đơn vị 8-bit để phục vụ cho quá trình tính toán ở chu kỳ hiện tại. Vì không có tính cục bộ như giá trị đầu vào Input Feature Map trong Convolution Layer nên để đơn giản, ta chọn kích thước đúng bằng $N_0 \times N_2 \times 8$ bits.

Để đọc $N_0 \times N_2$ giá trị thì thông thường ta cần tốn $N_0 \times N_2$ lần đọc dữ liệu từ bộ nhớ ngoài. Tuy nhiên, trọng số có kích thước lưu trữ là 8-bit mà mỗi lần đọc dữ liệu cho phép đọc lên tới 32-bit – gấp 4 lần so với kích thước lưu trữ của trọng số. Thêm nữa, các giá trị trọng số này lại nằm gần nhau trên bộ nhớ. Khai thác các đặc tính này cho phép giảm số lần cần thiết để lấy đủ $N_0 \times N_2$ dữ liệu 8-bit là $\lceil (N_0 \times N_2)/4 \rceil$.

Có tổng cộng N_1 Weight Buffer, nên ở một thời điểm có tổng cộng $N_0 \times N_1 \times N_2$ giá trị trọng số được lưu trữ trong bộ nhớ đệm. Vì vậy, cần tiêu tốn tối thiểu $N_3 \times \lceil (N_0 \times N_2)/4 \rceil$ lần đọc để chuẩn bị đủ các giá trị trọng số cần thiết phục vụ cho tính toán.

B. Input Buffer

Input Buffer đóng vai trò là bộ nhớ đệm để lưu trữ các giá trị đầu vào của mô hình ở bộ nhớ ngoài để phục vụ cho việc tính toán ở chu kỳ làm việc hiện tại. Input Buffer được dùng với ba loại Layer là Fully-Connected, Convolution và Pooling.

Về kích thước, bộ nhớ này phải đảm bảo được lưu trữ $N_0 \times N_2$ đơn vị 8-bit để phục vụ cho quá trình tính toán ở chu kỳ hiện tại. Đối với trường hợp Layer làm việc là Convolution và Pooling, giá trị đầu vào Input Feature Map trong tính toán có tính cục bộ nên ta có thể tận dụng việc này để tái sử dụng trong tính toán. Điều này cho phép việc tải nhiều dữ liệu hơn bình thường và sau đó, các dữ liệu này có thể được dùng cho các vòng tính toán tiếp theo. Do đó, kích thước của bộ nhớ này là $N_0 \times (4 \times \lceil N_2/4 \rceil) \times 8$ bits.

Đối với trường hợp Layer là Fully-Connected, lượng bộ nhớ được dùng là $N_0 \times N_2$ đơn vị nhớ 8-bit và $N_0 \times (4 \times \lceil N_2/4 \rceil - N_2)$ đơn vị nhớ không dùng tới.

Tương tự trường hợp của Weight Buffer, đầu vào có kích thước là 8-bit mà mỗi lần đọc dữ liệu cho phép đọc lên tới 32-bit – gấp 4 lần so với kích thước của đầu vào. Tuy nhiên, tính cục bộ về mặt lưu trữ của đầu vào lại khác với trọng số ở trường hợp Convolution và Pooling Layer.

- Trường hợp Fully-Connected Layer, giá trị đầu vào Input Activation nằm liên tục nhau trên bộ nhớ. Do đó, số lần cần thiết để tối thiểu lấy đủ $N_0 \times N_2$ dữ liệu 8-bit là $\lceil (N_0 \times N_2)/4 \rceil$.
- Trường hợp Convolution và Pooling Layer, giá trị đầu vào Input Feature Map với cùng một hàng mới nằm cạnh nhau, còn các hàng khác nhau thì cách nhau một khoảng cách nhất định. Trong một chu kỳ tính toán lại cần dữ liệu ở nhiều hàng khác nhau nên không thể áp dụng cách như ở Fully-Connected Layer. Vì vậy, ta chỉ có thể tải nhiều hơn dữ liệu trên cùng một hàng và dùng nó cho các chu kỳ tính toán tiếp theo. Do đó, số lần cần thiết tối thiểu để lấy đủ $N_0 \times N_2$ dữ liệu là $N_0 \times \lceil N_2/4 \rceil$.

Có tổng cộng N_3 Input Buffer, nên ở một thời điểm có ít nhất $N_0 \times N_2 \times N_3$ giá trị đầu vào được lưu trữ trong bộ nhớ đệm. Lượng dữ liệu này là đủ để phục vụ cho việc tính toán ở chu kỳ hiện tại. Các dữ liệu được lấy thêm cũng được phục vụ cho các chu kỳ tính toán tiếp theo. Điều này góp phần hạn chế số lần cần phải đọc thêm dữ liệu từ bộ nhớ ngoài. Vì vậy:

- Trường hợp Fully-Connected Layer, cần tiêu tốn tối thiểu $N_3 \times \lceil (N_0 \times N_2)/4 \rceil$ lần đọc để chuẩn bị đủ các giá trị trọng số cần thiết phục vụ cho tính toán.
- Trường hợp Convolution và Pooling Layer, cần tiêu tốn tối thiểu $N_3 \times N_0 \times \lceil N_2/4 \rceil$ lần đọc để chuẩn bị đủ các giá trị trọng số cần thiết phục vụ cho tính toán.

4.2.2.3 Nhóm phục vụ tác vụ ghi dữ liệu

Output Buffer

Output Buffer đóng vai trò là bộ nhớ đệm để lưu trữ các giá trị tổng riêng phần hoặc các kết quả hoàn chỉnh của Layer. Sau đó, các giá trị này được ghi vào bộ nhớ ngoài.

Các giá trị tổng riêng phần có kích thước là 32-bit, còn kết quả của Layer lại có kích thước là 8-bit. Vì vậy, kích thước của bộ nhớ đệm phải chứa được dữ liệu có kích thước lớn nhất nên kích thước của nó là 32-bit. Do đó, trong trường hợp đối tượng được lưu trữ là kết quả của Layer thì có 24-bit không dùng tới.

Về quá trình ghi bộ nhớ, Output Buffer cần duy nhất một lần ghi để lưu trữ giá trị ra bộ nhớ ngoài. Có tổng cộng N_1 Output Bufer, nên cần tiêu tốn N_1 lần ghi để hoàn thành quá trình ghi dữ liệu ở một chu kỳ tính toán.

4.2.3 Khả năng tính toán

Phân tích về khả năng tính toán của thành phần chức năng của bộ tăng tốc, ta đi từ phần thấp nhất là MAC cho đến phần cao nhất là Processing Matrix.

- Đầu tiên, đơn vị tính toán cơ bản nhất của của bộ tăng tốc là MAC với khả năng tính toán ở một tác vụ tính toán là N_0 giá trị đầu vào và N_0 giá trị trọng số theo mô hình Bit-Serial và tiêu tốn 8 chu kỳ tương ứng với 8-bit kích thước dữ liệu để hoàn thành tính toán.
- Đơn vị tính toán ở cấp tiếp theo là Processing Unit. Mỗi Processing Unit gồm N_1 bộ MAC, LUT và Local Register để phục vụ cho tính toán. Processing Unit có khả năng tính toán ở một tác vụ tính toán là N_0 giá trị đầu vào và $N_0 \times N_1$ giá trị trọng số và tiêu tốn 8 chu kỳ để hoàn thành tính toán.
- Đơn vị tính toán ở cấp tiếp theo là Processing Array. Mỗi Processing Array gồm N_2 khối Processing Unit để phục vụ cho tính toán. Processing Array có khả năng tính toán ở một tác vụ tính toán là $N_0 \times N_2$ giá trị đầu vào và $N_0 \times N_1 \times N_2$ giá trị trọng số và tiêu tốn 8 chu kỳ để hoàn thành tính toán.
- Đơn vị tính toán ở cấp cao nhất là Processing Matrix. Processing Matrix gồm N_3 khối Processing Array để phục vụ cho tính toán. Processing Matrix có khả năng tính toán ở một tác vụ tính toán là $N_0 \times N_2 \times N_3$ giá trị đầu vào và $N_0 \times N_1 \times N_2 \times N_3$ giá trị trọng số và tiêu tốn 8 chu kỳ để hoàn thành tính toán.

Ánh xạ các hằng số N_0, N_1, N_2 và N_3 vào các Layer cần tính toán như sau:

- Convolution Layer:
  - Input Feature Map

$$\underbrace{N_0}_{\text{width}} \times \underbrace{N_2}_{\text{height}} \times \underbrace{N_3}_{\text{depth}}$$

  - Kernel

$$\underbrace{N_0}_{\text{width}} \times \underbrace{N_2}_{\text{height}} \times \underbrace{N_3}_{\text{depth}} \times \underbrace{N_1}_{\text{number of kernel}}$$

- Fully-Connected Layer:
  - Input Activation

$$\underbrace{N_0 \times N_2 \times N_3}_{\text{width}}$$

  - Weight

$$\underbrace{N_0 \times N_2 \times N_3}_{\text{width}} \times \underbrace{N_1}_{\text{height}}$$

- Pooling Layer: Tuy rằng Layer này không thực hiện tính trên Processing Matrix nhưng tài nguyên đọc và ghi của Layer này vẫn được kết thừa từ Convolution Layer.
  - Input Feature Map

$$\underbrace{N_0}_{\text{width}} \times \underbrace{N_2}_{\text{height}} \times \underbrace{N_3}_{\text{depth}}$$

Lưu ý rằng, các thông số được ánh xạ như trên thể hiện khả năng tính toán tối đa của bộ tăng tốc đối với từng loại Layer cụ thể nên hoàn toàn có thể dùng ít tài nguyên trong tính toán hơn.

Hiện tại, bộ tăng tốc được hiện thực với các thông số N_0, N_1, N_2 và N_3 đều có giá trị là 3.

4.3 Thiết kế điều khiển

Để Alpha Accelerator hoạt động đúng như mong muốn, cần một thành phần điều khiển đảm bảo về điều khiển luồng thực thi đúng đắn cho các khối chức năng và có thể tùy chỉnh hoạt động theo yêu cầu người dùng.

4.3.1 Cấu hình cho bộ tăng tốc

4.3.1.1 Đặc tả về thông tin của Layer

Mỗi Layer cần phải cung cấp một lượng thông tin đầy đủ về thông số của bản thân Layer để có thể tiến hành tính toán trên Layer đó. Các thông tin này là khác nhau và đặc thù cho mỗi loại Layer.

Những thông tin tối thiểu cần cung cấp của mỗi loại Layer như sau:

- Fully-Connected Layer:
  - Kích thước của dữ liệu đầu vào Input Activation in_W (1D)
  - Kích thước của dữ liệu trọng số Weight $w_W \times w_H$ (2D)
  - Kích thước của dữ liệu kết quả đầu ra Output Activation ou_W (1D)
  - Kích thước của dữ liệu đối số Bias b_W (1D)
  - Loại hàm kích hoạt phi tuyến
- Convolution Layer:
  - Kích thước của dữ liệu đầu vào Input Feature Map $ifm_W \times ifm_H \times ifm_D$ (3D)
  - Kích thước của dữ liệu trọng số Kernel $k_W \times k_H \times k_D \times k_N$ (4D)
  - Kích thước của dữ liệu đầu ra Output Feature Map $ofm_W \times ofm_H \times ofm_D$ (3D)
  - Kích thước của dữ liệu đối số Bias b_W (1D)
  - Giá trị bước Stride theo chiều thứ nhất s_W và theo chiều thứ hai s_H
  - Loại hàm kích hoạt phi tuyến
- Max Pooling Layer
  - Kích thước của dữ liệu đầu vào Input Feature Map $ifm_W \times ifm_H \times ifm_D$ (3D)
  - Kích thước của dữ liệu đầu ra Output Feature Map $ofm_W \times ofm_H \times ofm_D$ (3D)
  - Kích thước của khối Patch $P_W \times P_H$ (2D)
  - Giá trị bước Stride theo chiều thứ nhất s_W và theo chiều thứ hai s_H

4.3.1.2 Tập thanh ghi cấu hình

Ngoài các thông tin liên quan tới Layer, Alpha Accelerator còn cần phải kiểm soát thêm về quá trình đọc ghi bộ nhớ và quá trình đặc thù liên quan tới mô hình Deep Learning làm tròn theo chuẩn 8-bit. Ngoài ra, còn các thông tin cần thiết khác phục vụ cho hoạt động của bộ tăng tốc.

Tập thanh ghi cấu hình bao gồm 17 thanh ghi có ý nghĩa khác nhau với từng loại Layer. Trong đó,

- Thanh ghi từ 0 - 4 phục vụ việc quản lý bộ nhớ.
- Thanh ghi từ 5 - 11 phục vụ việc lưu trữ thông tin ban đầu và thông tin liên quan có thể được tính toán từ những thông tin ban đầu của Layer.
- Thanh ghi từ 12 - 16 phục vụ việc lưu trữ các giá trị liên quan tới quá trình làm tròn kết quả từ 32-bit về 8-bit.

Theo từng loại Layer, các thanh ghi này có nhiệm vụ như sau:

- Convolution Layer
  - Thanh ghi 0: 32-bit địa chỉ cho vùng lưu trữ dữ liệu đầu vào Input Feature Map
  - Thanh ghi 1: 32-bit địa chỉ cho vùng lưu trữ dữ liệu trọng số Kernel
  - Thanh ghi 2: 32-bit địa chỉ cho vùng lưu trữ dữ liệu đầu ra Output Feature Map
  - Thanh ghi 3: 32-bit địa chỉ cho vùng lưu trữ dữ liệu đối số Bias
  - Thanh ghi 4: 32-bit địa chỉ cho vùng lưu trữ dữ liệu tổng riêng phần Partial-Sum
  - Thanh ghi 5: 4-bit loại Layer, 4-bit loại hàm kích hoạt, 4-bit gía trị bước Stride theo chiều thứ nhất S_W và 4-bit gía trị bước Stride theo chiều thứ hai S_H. 16-bit còn lại không dùng.
  - Thanh ghi 6: 16-bit kích thước theo chiều thứ nhất của Kernel k_W và 16-bit kích thước theo chiều thứ hai của Kernel k_H.
  - Thanh ghi 7: 16-bit kích thước theo chiều thứ ba của Kernel k_D và Input Feature Map ifm_D ($k_D == ifm_D$) và 16-bit số lượng Kernel k_N cũng là kích thước theo chiều thứ ba của Output Feature Map ofm_D ($k_N == ofm_D$).
  - Thanh ghi 8: 16-bit kích thước theo chiều thứ nhất của Input Feature Map ifm_W và 16-bit kích thước theo chiều thứ hai của Input Feature Map ifm_H.
  - Thanh ghi 9: 16-bit kích thước theo chiều thứ nhất của Output Feature Map ofm_W và 16-bit kích thước theo chiều thứ hai của Output Feature Map ofm_H.
  - Thanh ghi 10: 16-bit kích thước 2D của Input Feature Map $ifm_W \times ifm_H$ và 16-bit kích thước 2D của Output Feature Map $ofm_W \times ofm_H$.
  - Thanh ghi 11: 32-bit kích thước 3D của Kernel $k_W \times k_H \times k_D$.
  - Thanh ghi 12, 13 và 14: Kết hợp cả 3 thanh ghi này để truy xuất vào vùng nhớ đặc thù lưu trữ các giá trị cho quá trình lượng tử hóa từ 32-bit về 8-bit. Ở Layer này, đòi hỏi mỗi một Kernel cần dùng một bộ hệ số khác nhau cho quá trình lượng tử hóa.
    - 8-bit của thanh ghi 12 là chỉ mục để truy cập vào vùng nhớ.
    - 32-bit của thanh ghi 13 là giá trị hệ số nhân M_0
    - 8-bit của thanh ghi 14 là giá trị hệ số dịch phải n.

  - Thanh ghi 15: 32-bit giá trị phần thêm vào Z_α cho Input Feature Map
  - Thanh ghi 16: 32-bit giá trị phần thêm vào Z_α cho Output Feature Map

- Fully-Connected Layer
  - Thanh ghi 0: 32-bit địa chỉ cho vùng lưu trữ dữ liệu đầu vào Input Activation
  - Thanh ghi 1: 32-bit địa chỉ cho vùng lưu trữ dữ liệu trọng số Weight
  - Thanh ghi 2: 32-bit địa chỉ cho vùng lưu trữ dữ liệu đầu ra Output Activation
  - Thanh ghi 3: 32-bit địa chỉ cho vùng lưu trữ dữ liệu đối số Bias
  - Thanh ghi 4: 32-bit địa chỉ cho vùng lưu trữ dữ liệu tổng riêng phần Partial-Sum
  - Thanh ghi 5: 4-bit loại Layer, 4-bit loại hàm kích hoạt. 24-bit còn lại không dùng.
  - Thanh ghi 6: 16-bit kích thước theo chiều thứ nhất của Weight w_W và 16-bit kích thước theo chiều thứ hai của Weight w_H.
  - Thanh ghi 7: Không dùng.
  - Thanh ghi 8: 16-bit kích thước của Input Activation in_W. 16-bit còn lại không dùng.
  - Thanh ghi 9: 16-bit kích thước của Output Activation ou_W. 16-bit còn lại không dùng.
  - Thanh ghi 10: 16-bit kích thước 2D của Input Activation in_W và 16-bit kích thước 2D của Output Activation ou_W.
  - Thanh ghi 11: 32-bit kích thước 2D của Weight $w_W \times w_H$.
  - Thanh ghi 12, 13 và 14: Tương tự, kết hợp 3 thanh ghi này để truy xuất vào vùng nhớ đặc thù cho quá trình lượng tử hóa. Tuy nhiên, ở Layer này cho dùng duy nhất một bộ hệ số cho quá trình lượng tử hóa nên cho nền chỉ cần cài đặt một bộ giá trị duy nhất.
    * 8-bit của thanh ghi 12 là chỉ mục để truy cập vào vùng nhớ.
    * 32-bit của thanh ghi 13 là giá trị hệ số nhân M_0
    * 8-bit của thanh ghi 14 là giá trị hệ số dịch phải n.
  - Thanh ghi 15: 32-bit giá trị phần thêm vào Z_α cho Input Activation.
  - Thanh ghi 16: 32-bit giá trị phần thêm vào Z_α cho Output Activation.

- Pooling Layer
  - Thanh ghi 0: 32-bit địa chỉ cho vùng lưu trữ dữ liệu đầu vào Input Feature Map
  - Thanh ghi 1: Không dùng
  - Thanh ghi 2: 32-bit địa chỉ cho vùng lưu trữ dữ liệu đầu ra Output Feature Map
  - Thanh ghi 3: Không dùng
  - Thanh ghi 4: Không dùng

– Thanh ghi 5: 4-bit loại Layer (Pooling), 4-bit oại Pooling Layer, 4-bit gía trị bước Stride theo chiều thứ nhất S_W và 4-bit gía trị bước Stride theo chiều thứ hai S_H. 16-bit còn lại không dùng.

– Thanh ghi 6: 16-bit kích thước theo chiều thứ nhất của Patch P_W và 16-bit kích thước theo chiều thứ hai của Patch P_H.

– Thanh ghi 7: 16-bit kích thước theo chiều thứ ba của Input Feature Map ifm_D và 16-bit kích thước theo chiều thứ ba của Output Feature Map ofm_D.

– Thanh ghi 8: 16-bit kích thước theo chiều thứ nhất của Input Feature Map ifm_W và 16-bit kích thước theo chiều thứ hai của Input Feature Map ifm_H.

– Thanh ghi 9: 16-bit kích thước theo chiều thứ nhất của Output Feature Map ofm_W và 16-bit kích thước theo chiều thứ hai của Output Feature Map ofm_H.

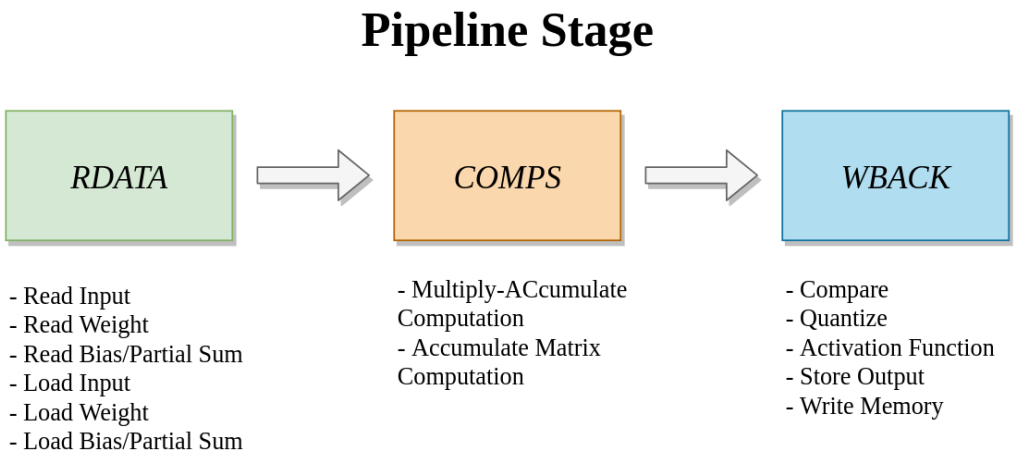
– Thanh ghi 10: 16-bit kích thước 2D của Input Feature Map $ifm_W \times ifm_H$ và 16-bit kích thước 2D của Output Feature Map $ofm_W \times ofm_H$.

– Thanh ghi 11: 32-bit giá trị cần phải chèn thêm khi quá trình dịch Patch không chia hết cho kích thước của Input Feature Map.

– Thanh ghi 12, 13, 14, 15 và 16: Không dùng

4.3.2 Pipeline ở Alpha Accelerator

Alpha Accelerator thực hiện tác vụ tính toán ở mỗi Layer theo phương pháp Pipeline và chia thành 3 giai đoạn – Stage.



Hình 4.11: Tóm tắt về Pipeline của bộ tăng tốc

Cụ thể về từng giai đoạn trong Pipeline như sau:

- *Read Data – RDATA*: Nhiệm vụ của giai đoạn này là đọc dữ liệu cần cho tính toán từ bộ nhớ ngoài và lưu trữ vào các bộ nhớ đệm và thanh ghi bên trong bộ tăng tốc. Cụ thể hơn, giai đoạn *RDATA* thực hiện các hành vi như sau:

- Read Input: Đọc và lưu trữ dữ liệu đầu vào Input Activation hay Input Feature Map cho Input Buffer.
- Read Weight: Đọc và lưu trữ dữ liệu trọng số Weight hay Kernel cho Weight Buffer.
- Read Bias/Partial-Sum: Đọc và lưu trữ dữ liệu về đối số Bias hoặc tổng riêng phần Partial-Sum cho Bias and Partial-Sum Buffer.
- Load Input: Đưa dữ liệu từ Input Buffer vào Input Register (trường hợp Fully-Connected Layer hay Convolution Layer) hoặc từ Input Buffer vào Element-Wise Unit (trường hợp Pooling Layer) chuẩn bị cho quá trình tính toán
- Load Weight: Đưa dữ liệu từ Weight Buffer vào Weight Register chuẩn bị phục vụ cho quá trình tính toán.
- Load Bias/Partial-Sum: Đưa dữ liệu từ Bias and Partial-Sum Buffer vào Bias and Partial-Sum Register của Accumulate Matrix chuẩn bị phục vụ cho quá trinhfh tính toán.

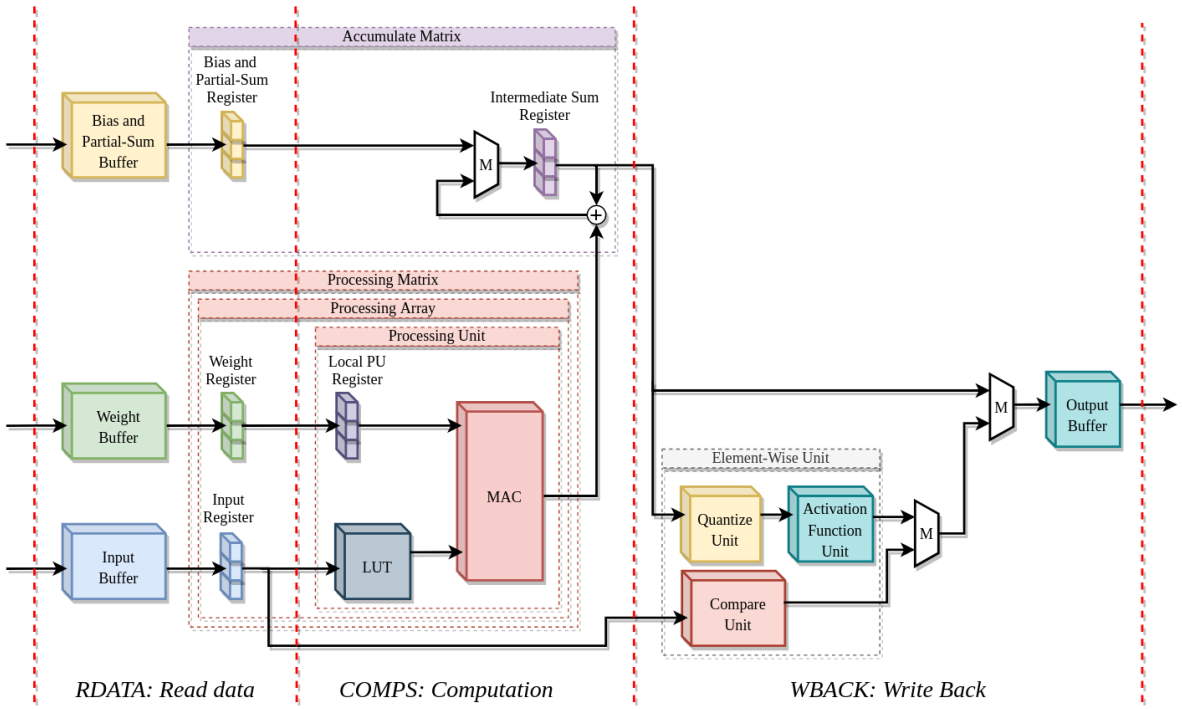
- *Computation – COMPS*: Nhiệm vụ của giai đoạn này là thực hiện các tính toán MAC đặc thù cho hai loại Layer là Fully-Connected và Convolution. Cụ thể hơn, giai đoạn *COMPS* thực hiện các hành vi như sau:
  - Multiply-ACcumulate Computation: Thực hiện tác vụ tính toán nhân và cộng tích lũy – MAC ở Fully-Connected Layer và Convolution Layer ở Processing Matrix từ các giá trị được lưu trữ trong bộ thanh ghi Weight và Input.
  - Accumulate Matrix Computation: Từ kết quả tính toán được ở Processing Matrix, các tổng riêng phần này được tiếp tục cộng lại với nhau và cộng với các giá trị đối số hoặc tổng riêng phần tương ứng.
- *Write Back – WBACK*: Nhiệm vụ của giai đoạn này là ghi các kết quả tạm thời (tổng riêng phần) hoặc kết quả đã tính toán xong trở lại bộ nhớ bên ngoài. Cụ thể hơn, giai đoạn *WBACK* thực hiện các hành vi như sau:
  - Compare: Tác vụ đặc thù của Max Pooling Layer. Các giá trị trong cùng một khối Patch sẽ được so sánh với nhau và cho ra kết quả cuối cùng là giá trị lớn nhất.
  - Quantize: Tác vụ đặc thù sau khi đã hoàn thành quá trình tính toán nhân và cộng tích lũy. Bởi vì, tổng khi tính toán xong thì kết quả là 32-bit nên chúng cần được làm tròn xuống 8-bit và chạy qua các hàm kích hoạt để cho ra kết quả cuối cùng.
  - Activation Function: Tác vụ đặc thù cho Fully-Connected Layer và Convolution Layer. Quá trình tính toán này chỉ được thực hiện khi đã nhân và cộng tích lũy xong. Các hàm kích hoạt sẽ được thực thi trên tổng này và cho ra kết quả cuối cùng. Thông thường, tác vụ này đi kèm với việc tính toán làm tròn trước đó.
  - Store Output: Lưu trữ kết quả tính toán được vào Output Buffer để chuẩn bị ghi vào bộ nhớ. Kết quả tính toán này có thể là tổng riêng phần từ Accumulate Matrix hoặc là kết quả

đầu ra từ Element-wise Unit.

- Write Memory: Ghi kết quả tính toán được từ Output Buffer vào bộ nhớ bên ngoài.

Để ý rằng, ở giai đoạn *COMPS*, không có bất kỳ tác vụ nào liên quan tới việc tính toán cho Pooling Layer. Do đó, ở trường hợp Layer được dùng là Pooling, Alpha Accelerator chỉ thực hiện Pipeline 2 giai đoạn là *RDATA* và *WBACK*.

Từ những đặc tả trên, Alpha Accelerator phân chia tài nguyên ở từng giai đoạn Pipeline như sau.



Hình 4.12: Phân chia tài nguyên Pipeline của Alpha Accelerator

Theo đó, mỗi giai đoạn Pipeline gồm các tài nguyên như sau:

- *RDATA*: Tài nguyên bao gồm:
  - Weight Buffer
  - Input Buffer
  - Bias and Partial-Sum Buffer
  - Một phần của Processing Matrix
    - Weight Register
    - Input Register
  - Một phần của Accumulate Matrix
    - Bias and Partial-Sum Register
- *COMPS*: Tài nguyên bao gồm:
  - Một phần của Processing Matrix
    - Local PU Register
    - LUT
    - MAC
  - Một phần của Accumulate Matrix
    - Intermediate Sum Register

        - Bộ Cộng

- *WBACK*: Tài nguyên bao gồm:
    - Toàn bộ Element-Wise Unit
        - Quantize Unit
        - Activation Function Unit
        - Compare Unit
    - Ouput Bufer

4.3.3 Máy trạng thái

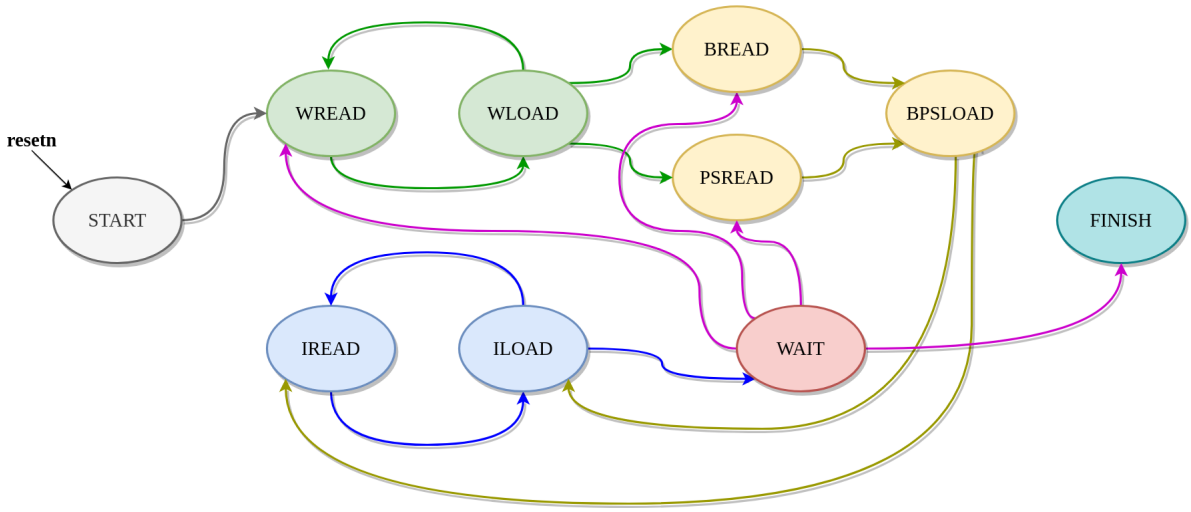
Bởi vì theo cơ chế Pipeline, Alpha Accelerator cần phải điều khiển các hoạt động của tài nguyên phù hợp với từng giai đoạn mà tài nguyên đó thuộc về. Để giải quyết vấn đề, một phương pháp có thể nghĩ đến là thiết kế 3 khối điều khiển ở 3 giai đoạn của Pipeline để phục vụ cho việc điều khiển luồng tính toán ở các thành phần tài nguyên tính toán của Alpha Accelerator.

Tùy vào loại hình của Layer mà có các máy trạng thái khác nhau cho phù hợp với phương pháp tính toán ưu tiên ở Layer đó.

4.3.3.1 Khối điều khiển RDATA

Khối điều khiển này được thiết kế cho việc đọc dữ liệu trên cả ba loại Layer là Convolution, Fully-Connected và Pooling ở giai đoạn RDATA.

A. Convolution Layer



Hình 4.13: Máy trạng thái ở giai đoạn RDATA của Convolution Layer

Mô hình tính toán được dùng cho Convolution Layer được gọi là *Kernel-Based Computation*. Nghĩa là việc tính toán ở Layer này dựa trên hai yếu tố là:

- Tái sử dụng các giá trị Kernel
- Tính cục bộ của Input Feature Map

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
START	*WREAD*	Tự động chuyển sang trạng thái tiếp theo mà không cần điều kiện đặc biệt gì cả.	*START* là trạng thái đầu tiên cũng là trạng thái khi máy trạng thái được cài lại. Nhiệm vụ của *START* là khởi tạo tất cả thanh ghi điều khiển của máy trạng thái chuẩn bị phục vụ cho các tác vụ tiếp theo.
WREAD	*WLOAD*	Hoàn thành quá trình đọc giá trị trọng số cho tất cả bộ nhớ đệm.	*WREAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới trọng số của giai đoạn RDATA. Nhiệm vụ của *WREAD* là đọc tất cả giá trị trọng số từ bộ nhớ ngoài vào Weight Buffer.
WLOAD	*WREAD*	Hoàn thành quá trình tải giá trị trọng số từ tất cả bộ nhớ đệm sang thanh ghi và số lượng giá trị trọng số phục vụ cho tính toán vẫn CHƯA đủ.	*WLOAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới trọng số của giai đoạn RDATA. Nhiệm vụ của *WLOAD* là tải tất cả giá trị trọng số từ Weight Buffer sang Weight Register bên trong Processing Unit chuẩn bị cho tác vụ tính toán.
	BREAD	Hoàn thành quá trình tải giá trị trọng số từ tất cả bộ nhớ đệm sang thanh ghi và tập giá trị cần cho tính toán là tập đối số – BIAS.	
	PSREAD	Hoàn thành quá trình tải giá trị trọng số từ tất cả bộ nhớ đệm sang thanh ghi và tập giá trị cần cho tính toán là tập tổng riêng phần – PARTIAL-SUM.	
BREAD	*BPSLOAD*	Hoàn thành quá trình đọc giá trị đối số vào tất cả bộ nhớ đệm.	*BREAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đối số và tổng riêng phần của giai đoạn RDATA. Nhiệm vụ của *BREAD* là đọc tất cả giá trị đối số từ bộ nhớ ngoài vào Bias and Partial-Sum Bufer cần cho tác vụ tính toán.
PSREAD	*BPSLOAD*	Hoàn thành quá trình đọc tổng riêng phần vào tất cả bộ nhớ đệm.	*PSREAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đối số và tổng riêng phần của giai đoạn RDATA. Nhiệm vụ của *PSREAD* là đọc tất cả tổng riêng phần từ bộ nhớ ngoài vào Bias and Partial-Sum Bufer cần cho tác vụ tính toán.

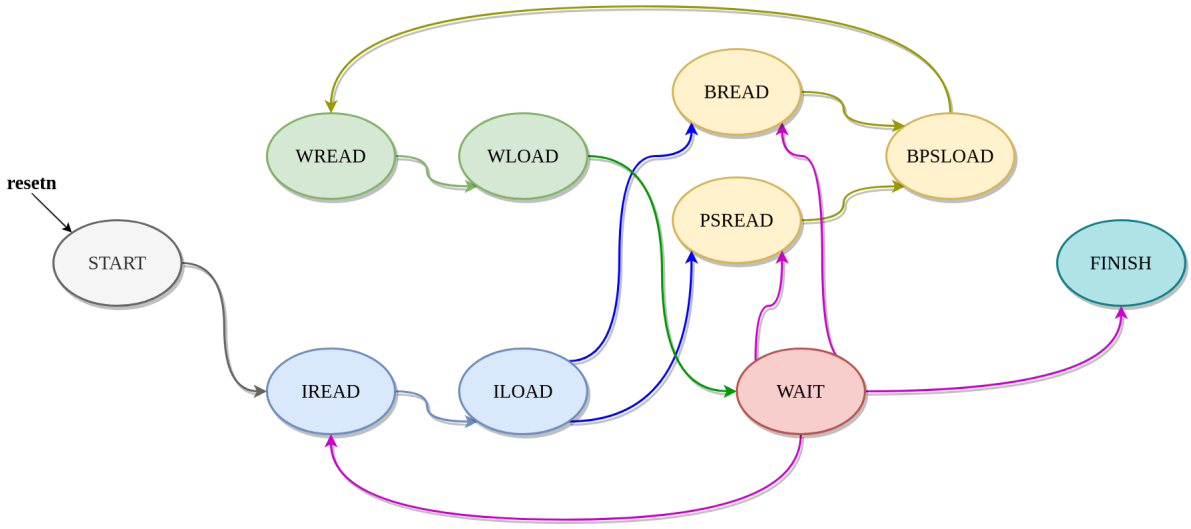
Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
BPSLOAD	*IREAD*	Hoàn thành quá trình đọc giá trị đối số hoặc tổng riêng phần từ tất cả bộ nhớ đệm sang thanh ghi và bộ nhớ đệm đầu vào CHƯA đủ giá trị cần thiết cho lượt tính toán tiếp theo.	*BPSLOAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đối số và tổng riêng phần của giai đoạn RDATA. Nhiệm vụ của *BPSLOAD* là tải tất cả giá trị đối số hoặc tổng riêng (tùy vào trạng thái trước đó) từ Bias and Partial-Sum Bufer sang Bias and Partial-Sum Register bên trong Acucmulate Matrix chuẩn bị cho tác vụ tính toán.
	ILOAD	Hoàn thành quá trình đọc giá trị đối số hoặc tổng riêng phần từ tất cả bộ nhớ đệm sang thanh ghi và bộ nhớ đệm đầu vào ĐÃ đủ giá trị cần thiết cho lượt tính toán tiếp theo.	
IREAD	*ILOAD*	Hoàn thành quá trình đọc giá trị đầu vào vào tất cả bộ nhớ đệm.	*IREAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đầu vào của giai đoạn RDATA. Nhiệm vụ của *IREAD* là đọc tất cả giá trị đầu vào từ bộ nhớ ngoài vào Input Buffer cần cho tác vụ tính toán.
ILOAD	*IREAD*	Hoàn thành quá trình tải giá trị đầu vào từ tất cả bộ nhớ đệm sang ghi và số lượng giá trị đầu vào phục vụ cho tinh toán vẫn CHƯA đủ.	*ILOAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đầu vào của giai đoạn RDATA. Nhiệm vụ của *ILOAD* là tải tất cả giá trị đầu vào từ Input Buffer sang Input Register bên trong Processing Matrix chuẩn bị cho tác vụ tính toán.
	WAIT	Hoàn thành quá trình tải giá trị đầu vào từ tất cả bộ nhớ đệm sang ghi và số lượng giá trị đầu vào phục vụ cho tinh toán vẫn ĐÃ đủ.	

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
WAIT	*WREAD*	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình tính toán ĐÃ hoàn thành trên tất cả giá trị đầu vào cần thiết.	*WAIT* là trạng thái mang tính chất đồng bộ hóa giữa các giai đoạn trong Pipeline. Nhiệm vụ của *WAIT* là để xác nhận rằng một chu kỳ hoạt động của giai đoạn RDATA đã xong và khối điều khiển chỉ việc chờ các khối điều khiển khác xác nhận để cùng chuyển qua chu kỳ làm việc tiếp theo.
	BREAD	Tất cả các khối điều khiển đều đạt trạng thái *WAIT*, quá trình tính toán CHƯA hoàn thành trên tất cả giá trị đầu vào cần thiết và chu kỳ làm việc tiếp theo cần tập đối số – BIAS.	
	PSREAD	Tất cả các khối điều khiển đều đạt trạng thái *WAIT*, quá trình tính toán CHƯA hoàn thành trên tất cả giá trị đầu vào cần thiết và chu kỳ làm việc tiếp theo cần tập tổng riêng phần – PARTIAL-SUM.	
	FINISH	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình tính toán ĐÃ hoàn thành trên tất cả giá trị đầu vào với giá trị trọng số cần của Layer làm việc.	
FINISH			*FINISH* là trạng thái kết thúc của máy trạng thái. Khi đạt được trạng thái này đồng nghĩa với việc tất cả tác vụ liên quan tới đọc dữ liệu của giai đoạn RDATA đã hoàn thành.

Bảng 4.1: Bảng mô tả trạng thái RDATA ở Convolution Layer

B. Fully-Connected Layer

Mô hình tính toán được dùng cho Fully-Connected Layer được gọi là *Input-Based Computation FSM*. Nghĩa là việc tính toán ở Layer này chỉ dựa trên yếu tố là tái sử dụng các giá trị đầu vào Input Activation.



Hình 4.14: Máy trạng thái ở giai đoạn RDATA của Fully-Connected Layer

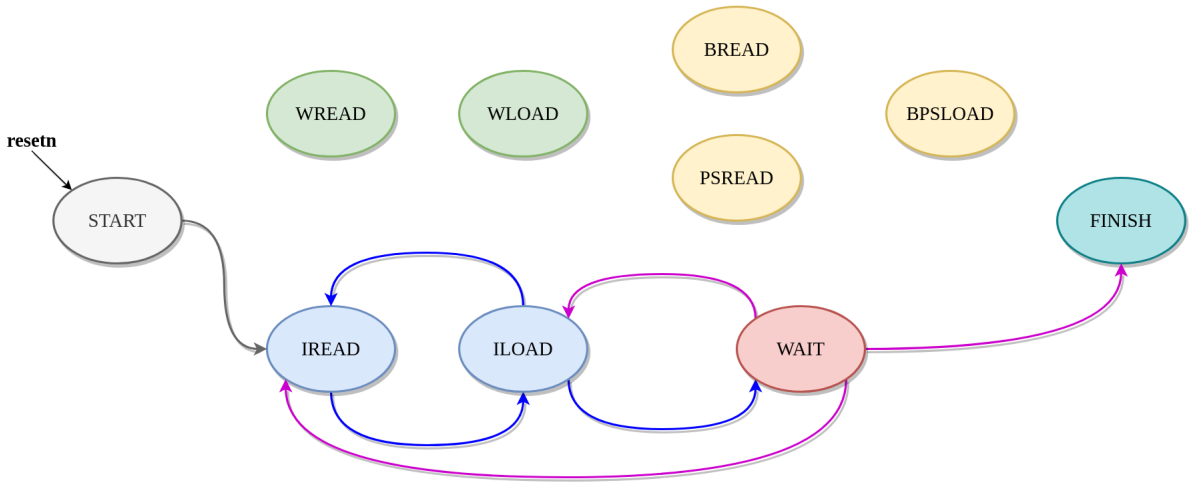
Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
START	*IREAD*	Tự động chuyển sang trạng thái tiếp theo mà không cần điều kiện đặc biệt gì cả.	*START* là trạng thái đầu tiên cũng là trạng thái khi máy trạng thái được cài lại. Nhiệm vụ của *START* là khởi tạo tất cả thanh ghi điều khiển của máy trạng thái chuẩn bị phục vụ cho các tác vụ tiếp theo.
IREAD	*ILOAD*	Hoàn thành quá trình đọc giá trị đầu vào vào tất cả bộ nhớ đệm.	*IREAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đầu vào của giai đoạn RDATA. Nhiệm vụ của *IREAD* là đọc tất cả giá trị đầu vào từ bộ nhớ ngoài vào Input Buffer cần cho tác vụ tính toán.
ILOAD	*BREAD*	Hoàn thành quá trình tải giá trị đầu vào từ tất cả bộ nhớ đệm sang thanh ghi và tập giá trị cần cho tính toán là tập đối số – BIAS.	*ILOAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đầu vào của giai đoạn RDATA. Nhiệm vụ của *ILOAD* là tải tất cả giá trị đầu vào từ Input Buffer sang Input Register bên trong Processing Matrix chuẩn bị cho tác vụ tính toán.
	PSREAD	Hoàn thành quá trình tải giá trị đầu vào từ tất cả bộ nhớ đệm sang thanh ghi và tập giá trị cần cho tính toán là tập tổng riêng phần – PARTIAL-SUM.	

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
BREAD	*BPSLOAD*	Hoàn thành quá trình đọc giá trị đối số vào tất cả bộ nhớ đệm.	*BREAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đối số và tổng riêng phần của giai đoạn RDATA. Nhiệm vụ của *BREAD* là đọc tất cả giá trị đối số từ bộ nhớ ngoài vào Bias and Partial-Sum Bufer cần cho tác vụ tính toán.
PSREAD	*BPSLOAD*	Hoàn thành quá trình đọc tổng riêng phần vào tất cả bộ nhớ đệm.	*PSREAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đối số và tổng riêng phần của giai đoạn RDATA. Nhiệm vụ của *PSREAD* là đọc tất cả tổng riêng phần từ bộ nhớ ngoài vào Bias and Partial-Sum Bufer cần cho tác vụ tính toán.
BPSLOAD	*WREAD*	Hoàn thành quá trình đọc giá trị đối số hoặc tổng riêng phần từ tất cả bộ nhớ đệm sang thanh ghi.	*BPSLOAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đối số và tổng riêng phần của giai đoạn RDATA. Nhiệm vụ của *BPSLOAD* là tải tất cả giá trị đối số hoặc tổng riêng (tùy vào trạng thái trước đó) từ Bias and Partial-Sum Bufer sang Bias and Partial-Sum Register bên trong Acucmulate Matrix chuẩn bị cho tác vụ tính toán.
WREAD	*WLOAD*	Hoàn thành quá trình đọc giá trị trọng số cho tất cả bộ nhớ đệm.	*WREAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới trọng số của giai đoạn RDATA. Nhiệm vụ của *WREAD* là đọc tất cả giá trị trọng số từ bộ nhớ ngoài vào Weight Buffer.

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
WLOAD	*WAIT*	Hoàn thành quá trình tải giá trị trọng số từ tất cả bộ nhớ đệm sang thanh ghi.	*WLOAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới trọng số của giai đoạn RDATA. Nhiệm vụ của *WLOAD* là tải tất cả giá trị trọng số từ Weight Buffer sang Weight Register bên trong Processing Unit chuẩn bị cho tác vụ tính toán.
WAIT	*IREAD*	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình tính toán ĐÃ hoàn thành trên tất cả giá trị trọng số cần thiết.	*WAIT* là trạng thái mang tính chất đồng bộ hóa giữa các giai đoạn trong Pipeline. Nhiệm vụ của *WAIT* là để xác nhận rằng một chu kỳ hoạt động của giai đoạn RDATA đã xong và khối điều khiển chỉ việc chờ các khối điều khiển khác xác nhận để cùng chuyển qua chu kỳ làm việc tiếp theo.
	BREAD	Tất cả các khối điều khiển đều đạt trạng thái *WAIT*, quá trình tính toán CHƯA hoàn thành trên tất cả giá trị trọng số cần thiết và chu kỳ làm việc tiếp theo cần tập đối số – BIAS.	
	PSREAD	Tất cả các khối điều khiển đều đạt trạng thái *WAIT*, quá trình tính toán CHƯA hoàn thành trên tất cả giá trị trọng số cần thiết và chu kỳ làm việc tiếp theo cần tập tổng riêng phần – PARTIAL-SUM.	
	FINISH	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình tính toán ĐÃ hoàn thành trên tất cả giá trị trọng số với giá trị đầu vào cần của Layer làm việc.	
FINISH			*FINISH* là trạng thái kết thúc của máy trạng thái. Khi đạt được trạng thái này đồng nghĩa với việc tất cả tác vụ liên quan tới đọc dữ liệu của giai đoạn RDATA đã hoàn thành.

Bảng 4.2: Bảng mô tả trạng thái RDATA ở Fully-Connected Layer

C. Pooling Layer



Hình 4.15: Máy trạng thái ở giai đoạn RDATA của Pooling Layer

Mô hình tính toán được dùng cho Pooling Layer được gọi là *Local-Based Computation*. Việc tính toán ở Layer này dựa trên chỉ dựa trên yếu tố là tận dụng tính cục bộ của Input Feature Map.

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
START	*IREAD*	Tự động chuyển sang trạng thái tiếp theo mà không cần điều kiện đặc biệt gì cả.	*START* là trạng thái đầu tiên cũng là trạng thái khi máy trạng thái được cài lại. Nhiệm vụ của *START* là khởi tạo tất cả thanh ghi điều khiển của máy trạng thái chuẩn bị phục vụ cho các tác vụ tiếp theo.
IREAD	*ILOAD*	Hoàn thành quá trình đọc giá trị đầu vào vào tất cả bộ nhớ đệm.	*IREAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đầu vào của giai đoạn RDATA. Nhiệm vụ của *IREAD* là đọc tất cả giá trị đầu vào từ bộ nhớ ngoài vào Input Buffer cần cho tác vụ tính toán.

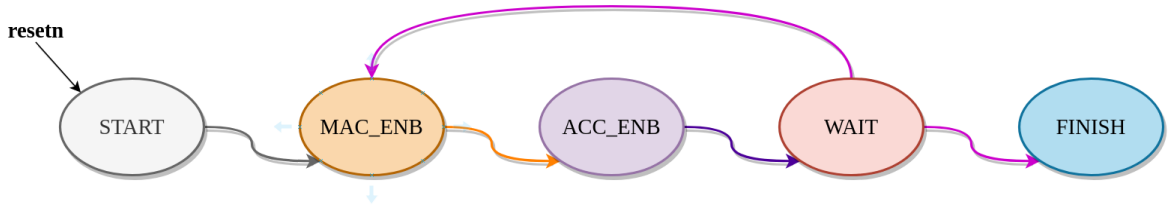
Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
ILOAD	*IREAD*	Hoàn thành quá trình tải giá trị đầu vào từ tất cả bộ nhớ đệm sang ghi và số lượng giá trị đầu vào phục vụ cho tinh toán vẫn CHƯA đủ.	*ILOAD* là trạng thái đặc thù thuộc vào bộ tác vụ liên quan tới đầu vào của giai đoạn RDATA. Nhiệm vụ của *ILOAD* là tải tất cả giá trị đầu vào từ Input Buffer sang Input Register bên trong Processing Matrix chuẩn bị cho tác vụ tính toán.
	WAIT	Hoàn thành quá trình tải giá trị đầu vào từ tất cả bộ nhớ đệm sang ghi và số lượng giá trị đầu vào phục vụ cho tinh toán vẫn ĐÃ đủ.	
WAIT	*IREAD*	Tất cả các khối điều khiển đều đạt trạng thái *WAIT*, quá trình tính toán CHƯA hoàn thành trên tất cả giá trị đầu vào cần thiết và bộ nhớ đệm đầu vào CHƯA đủ giá trị cần thiết cho lượt tính toán tiếp theo.	*WAIT* là trạng thái mang tính chất đồng bộ hóa giữa các giai đoạn trong Pipeline. Nhiệm vụ của *WAIT* là để xác nhận rằng một chu kỳ hoạt động của giai đoạn RDATA đã xong và khối điều khiển chỉ việc chờ các khối điều khiển khác xác nhận để cùng chuyển qua chu kỳ làm việc tiếp theo.
	ILOAD	Tất cả các khối điều khiển đều đạt trạng thái *WAIT*, quá trình tính toán CHƯA hoàn thành trên tất cả giá trị đầu vào cần thiết và bộ nhớ đệm đầu vào ĐÃ đủ giá trị cần thiết cho lượt tính toán tiếp theo.	
	FINISH	Tất cả các khối điều khiển đều đạt trạng thái *WAIT*, quá trình tính toán ĐÃ hoàn thành trên tất cả giá trị đầu vào cần thiết.	
FINISH			*FINISH* là trạng thái kết thúc của máy trạng thái. Khi đạt được trạng thái này đồng nghĩa với việc tất cả tác vụ liên quan tới đọc dữ liệu của giai đoạn RDATA đã hoàn thành.

Bảng 4.3: Bảng mô tả trạng thái RDATA ở Pooling Layer

4.3.3.2 Khối điều khiển COMPS

Khối điều khiển này được thiết kế cho việc thực hiện tác vụ MAC trên hai loại Layer là Convolution và Fully-Connected ở giai đoạn COMPS.

Máy trạng thái điều khiển ở giai đoạn này giống nhau cho cả hai loại Layer.



Hình 4.16: Máy trạng thái ở giai đoạn COMPS của Convolutionn và Fully-Connected Layer

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
START	*MAC_ENB*	Khối điều khiển RDATA đạt được một chu kỳ hoạt động và hiện đang ở trạng thái *WAIT*	*START* là trạng thái đầu tiên cũng là trạng thái khi máy trạng thái được cài lại. Nhiệm vụ của *START* là khởi tạo tất cả thanh ghi điều khiển của máy trạng thái chuẩn bị phục vụ cho các tác vụ tiếp theo.
MAC_ENB	*ACC_ENB*	Processing Matrix thông báo đã hoàn thành quá trình tính toán MAC.	*MAC_ENB* là trạng thái đặc thù liên quan tới tác vụ nhân và cộng tích lũy – MAC ở giai đoạn COMPS. Nhiệm vụ của *MAC_ENB* là từ tất cả giá trị về đầu vào, trọng số, đối số hoặc tổng riêng phần từ giai đoạn trước để thực hiện các tính toán nhân và cộng dồn để cho ra kết quả là tổng cuối cùng.
ACC_ENB	*WAIT*	Accumulate Matrix thông báo đã hoàn thành quá trình tính toán cộng.	*ACC_ENB* là trạng thái đặc thù liên quan tới tác vụ cộng tích lũy tổng riêng phần ở giai đoạn COMPS. Nhiệm vụ của *ACC_ENB* là từ tất cả giá trị tính toán được từ Processing Matrix cộng lại các tổng riêng phần thuộc cùng phần kết quả đầu ra để tạo ra các tổng riêng phần lớn hoặc là tổng hoàn chỉnh.

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
WAIT	*MAC_ENB*	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình tính toán CHƯA hoàn thành trên tất cả giá trị đầu vào, trọng số và tổng riêng phần hoặc đối số cần thiết được lưu trữ trong thanh ghi.	*WAIT* là trạng thái mang tính chất đồng bộ hóa giữa các giai đoạn trong Pipeline. Nhiệm vụ của *WAIT* là để xác nhận rằng một chu kỳ hoạt động của giai đoạn COMPS đã xong và khối điều khiển chỉ việc chờ các khối điều khiển khác xác nhận để cùng chuyển qua chu kỳ làm việc tiếp theo.
	FINISH	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình tính toán ĐÃ hoàn thành trên tất cả giá trị đầu vào, trọng số và tổng riêng phần hoặc đối số cần thiết được lưu trữ trong thanh ghi.	
FINISH			*FINISH* là trạng thái kết thúc của máy trạng thái. Khi đạt được trạng thái này đồng nghĩa với việc tất cả tác vụ liên quan tới tính toán của giai đoạn COMPS đã hoàn thành.

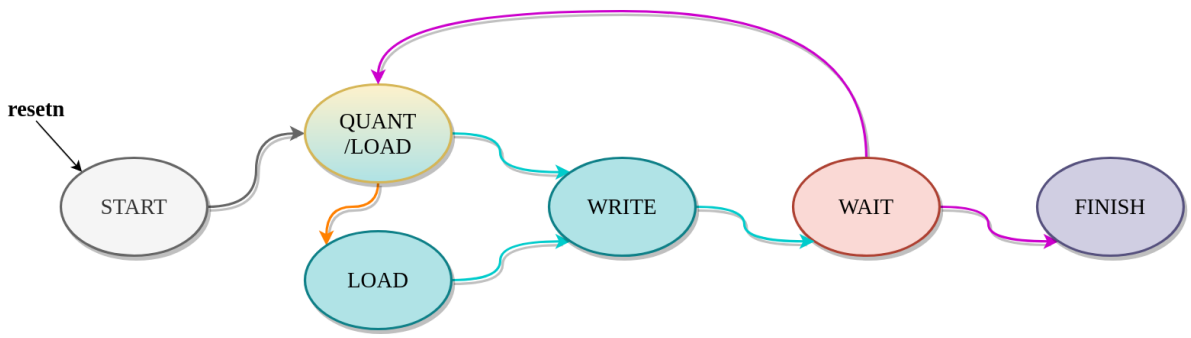
Bảng 4.4: Bảng mô tả trạng thái COMPS ở Convolution Layer và Fully-Connected Layer

4.3.3.3 Khối điều khiển WBACK

Khối điều khiển này được thiết kế cho việc đọc dữ liệu trên cả ba loại Layer là Convolution, Fully-Connected và Pooling ở giai đoạn WBACK.

Máy trạng thái điều khiển ở giai đoạn này có sự giống nhau giữa Convolution và Fully-Connected Layer.

A. Convolution và Fully-Connected Layer



Hình 4.17: Máy trạng thái ở giai đoạn WBACK của Convolution và Fully-Connected Layer

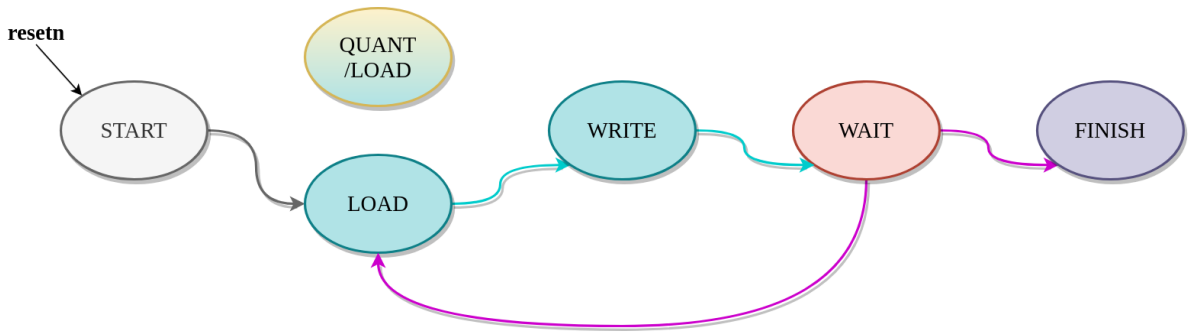
Giai đoạn WBACK ở Convolution và Fully-Connected Layer còn bao gồm thêm tác vụ làm kết quả từ 32-bit về 8-bit và thực thi các hàm kích hoạt trên các kết quả đó.

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
START	*QUANT/LOAD*	Khối điều khiển RDATA đạt được hai chu kỳ hoạt động, khối điều khiển COMPS đạt được một chu kỳ hoạt đọng và hiện cả hai đều đang ở trạng thái *WAIT*	*START* là trạng thái đầu tiên cũng là trạng thái khi máy trạng thái được cài lại. Nhiệm vụ của *START* là khởi tạo tất cả thanh ghi điều khiển của máy trạng thái chuẩn bị phục vụ cho các tác vụ tiếp theo.
QUANT/LOAD	*LOAD*	Hành vi của trạng thái là *QUANT* – kết quả tính toán được xác định là tổng cuối cùng và Element-Wise Unit thông báo đã hoàn thành quá trình làm tròn và thực hiện hàm kích hoạt.	*QUANT/LOAD* là trạng thái đặc biệt khi mà tùy vào kết quả tính toán từ chu kỳ hiện tại mà quyết định hành vi của nó. Nếu kết quả tính toán được xác định là tổng riêng phần thì hành vi của trạng thái là *LOAD* – nhiệm vụ là tải dữ liệu vào trong Output Buffer. Còn nếu kết quả tính toán được xác định là tổng cuối cùng thì hành vi của trạng thái là *QUANT* – nhiệm vụ là làm tròn dữ liệu về 8-bit và thực hiện hàm kích hoạt.
	WRITE	Hành vi của trạng thái là *LOAD* – kết quả tính toán được xác định là tổng riêng phần và hoàn thành quá trình tải giá trị tải tổng riêng phần vào tất cả bộ nhớ đệm.	
LOAD	*WRITE*	Hoàn thành quá trình tải giá trị kết quả đầu ra vào tất cả bộ nhớ đệm.	*LOAD* là trạng thái đặc thù thuộc tác vụ ghi dữ liệu ra bộ nhớ ngoài. Nhiệm vụ của *LOAD* là tải dữ liệu vào trong Output Buffer chuẩn bị cho việc ghi ra bộ nhớ ngoài.
WRITE	*WAIT*	Hoàn thành quá trình ghi dữ liệu từ bộ nhớ đệm vào bộ nhớ ngoài.	*WRITE* là trạng thái đặc thù thuộc tác vụ ghi dữ liệu ra bộ nhớ ngoài. Nhiệm vụ của *WRITE* là ghi dữ liệu từ Output Buffer vào bộ nhớ ngoài.

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
WAIT	*QUANT/LOAD*	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình ghi dữ liệu CHƯA hoàn thành trên tất cả giá trị kết quả đầu ra của Layer.	*WAIT* là trạng thái mang tính chất đồng bộ hóa giữa các giai đoạn trong Pipeline. Nhiệm vụ của *WAIT* là để xác nhận rằng một chu kỳ hoạt động của giai đoạn WBACK đã xong và khối điều khiển chỉ việc chờ các khối điều khiển khác xác nhận để cùng chuyển qua chu kỳ làm việc tiếp theo.
	FINISH	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình ghi dữ liệu ĐÃ hoàn thành trên tất cả giá trị kết quả đầu ra của Layer.	
FINISH			*FINISH* là trạng thái kết thúc của máy trạng thái. Khi đạt được trạng thái này đồng nghĩa với việc tất cả tác vụ liên quan tới ghi dữ liệu của giai đoạn WBACK đã hoàn thành.

Bảng 4.5: Bảng mô tả trạng thái WBACK ở Convolution Layer và Fully-Connected Layer

B. Pooling Layer



Hình 4.18: Máy trạng thái ở giai đoạn WBACK của Pooling Layer

Tác vụ đơn nguyên (so sánh hoặc trung bình cộng) của Pooling Layer được thực thi ở giai đoạn WBACK thông qua Element-Wise Unit.

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
START	*LOAD*	Khối điều khiển RDATA đạt được một chu kỳ hoạt động và hiện đang ở trạng thái *WAIT*. (Pooling Layer chỉ thực hiện Pipeline trên 2 Stage)	*START* là trạng thái đầu tiên cũng là trạng thái khi máy trạng thái được cài lại. Nhiệm vụ của *START* là khởi tạo tất cả thanh ghi điều khiển của máy trạng thái chuẩn bị phục vụ cho các tác vụ tiếp theo.
LOAD	*WRITE*	Hoàn thành quá trình tải giá trị kết quả đầu ra vào tất cả bộ nhớ đệm.	*LOAD* là trạng thái đặc thù thuộc tác vụ ghi dữ liệu ra bộ nhớ ngoài. Nhiệm vụ của *LOAD* là tải dữ liệu vào trong Output Buffer chuẩn bị cho việc ghi ra bộ nhớ ngoài.
WRITE	*WAIT*	Hoàn thành quá trình ghi dữ liệu từ bộ nhớ đệm vào bộ nhớ ngoài.	*WRITE* là trạng thái đặc thù thuộc tác vụ ghi dữ liệu ra bộ nhớ ngoài. Nhiệm vụ của *WRITE* là ghi dữ liệu từ Output Buffer vào bộ nhớ ngoài.
WAIT	*LOAD*	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình ghi dữ liệu CHƯA hoàn thành trên tất cả giá trị kết quả đầu ra của Layer.	*WAIT* là trạng thái mang tính chất đồng bộ hóa giữa các giai đoạn trong Pipeline. Nhiệm vụ của *WAIT* là để xác nhận rằng một chu kỳ hoạt động của giai đoạn WBACK đã xong và khối điều khiển chỉ việc chờ các khối điều khiển khác xác nhận để cùng chuyển qua chu kỳ làm việc tiếp theo.
	FINISH	Tất cả các khối điều khiển đều đạt trạng thái *WAIT* và quá trình ghi dữ liệu ĐÃ hoàn thành trên tất cả giá trị kết quả đầu ra của Layer.	
FINISH			*FINISH* là trạng thái kết thúc của máy trạng thái. Khi đạt được trạng thái này đồng nghĩa với việc tất cả tác vụ liên quan tới ghi dữ liệu của giai đoạn WBACK đã hoàn thành.

Bảng 4.6: Bảng mô tả trạng thái WBACK ở Convolution Layer và Fully-Connected Layer

4.3.4 Quy trình tính toán

Dưới đây là mô tả cụ thể về quy trình tính toán ở từng loại Layer được thiết kế và hiện thực ở Alpha Accelerator.

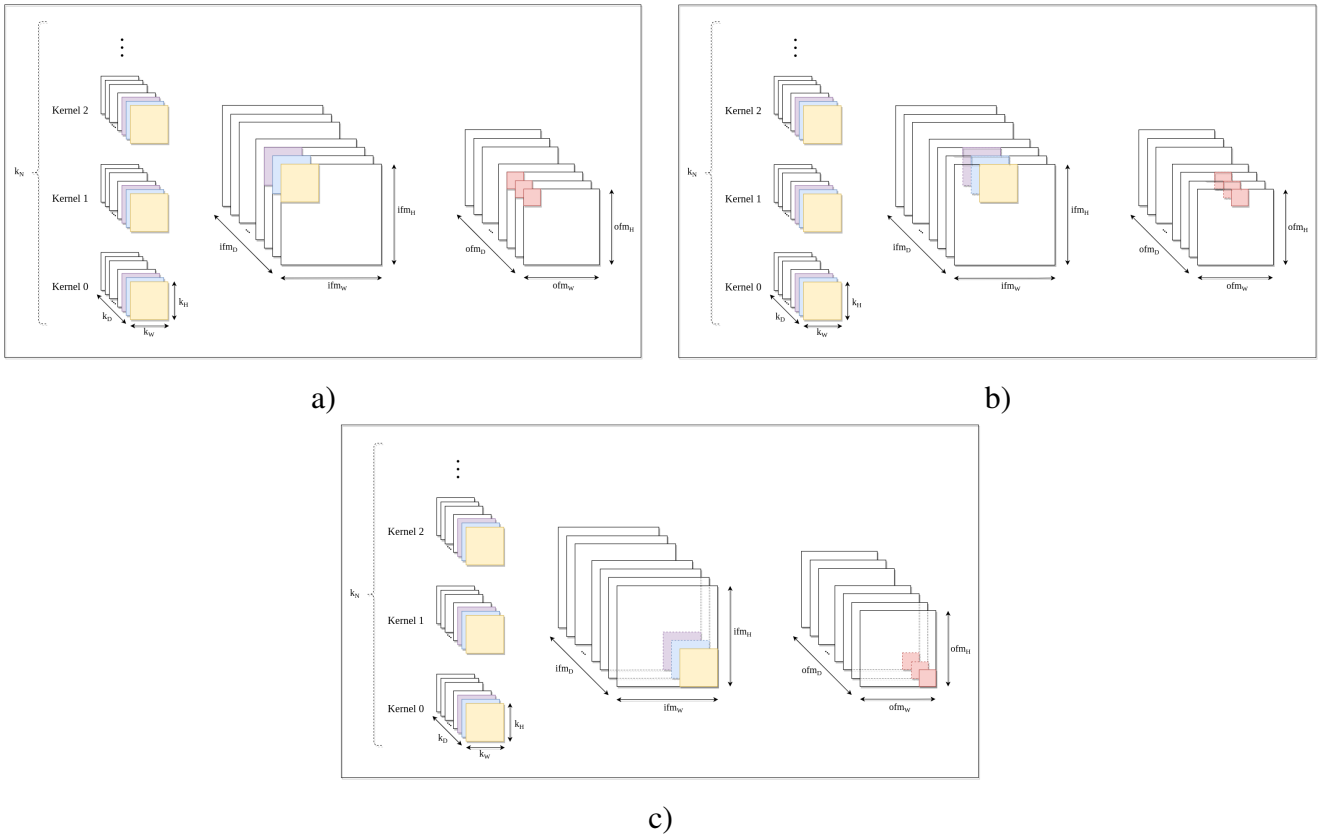
4.3.4.1 Convolution Layer

Như đã trình bày, mô hình tính toán được dùng ở Convolution Layer là *Kernel-Based Computation* được phát triển từ việc tận dụng tính tái sử dụng ở Kernel và tính cục bộ ở Input Feature Map trong phép tích chập.

Trước khi bắt đầu, lưu ý rằng một lần tính toán cho phép:

- Tính toán với một tập Kernel có kích thước là $N_1 = 3$ phần tử. Ví dụ tập Kernel là tập $(6, 7, 8)$.
- Tính toán với mức độ sâu là một tập hợp có kích thước là $N_3 = 3$ phần tử. Ví dụ mức độ sâu là tập $(9, 10, 11)$.

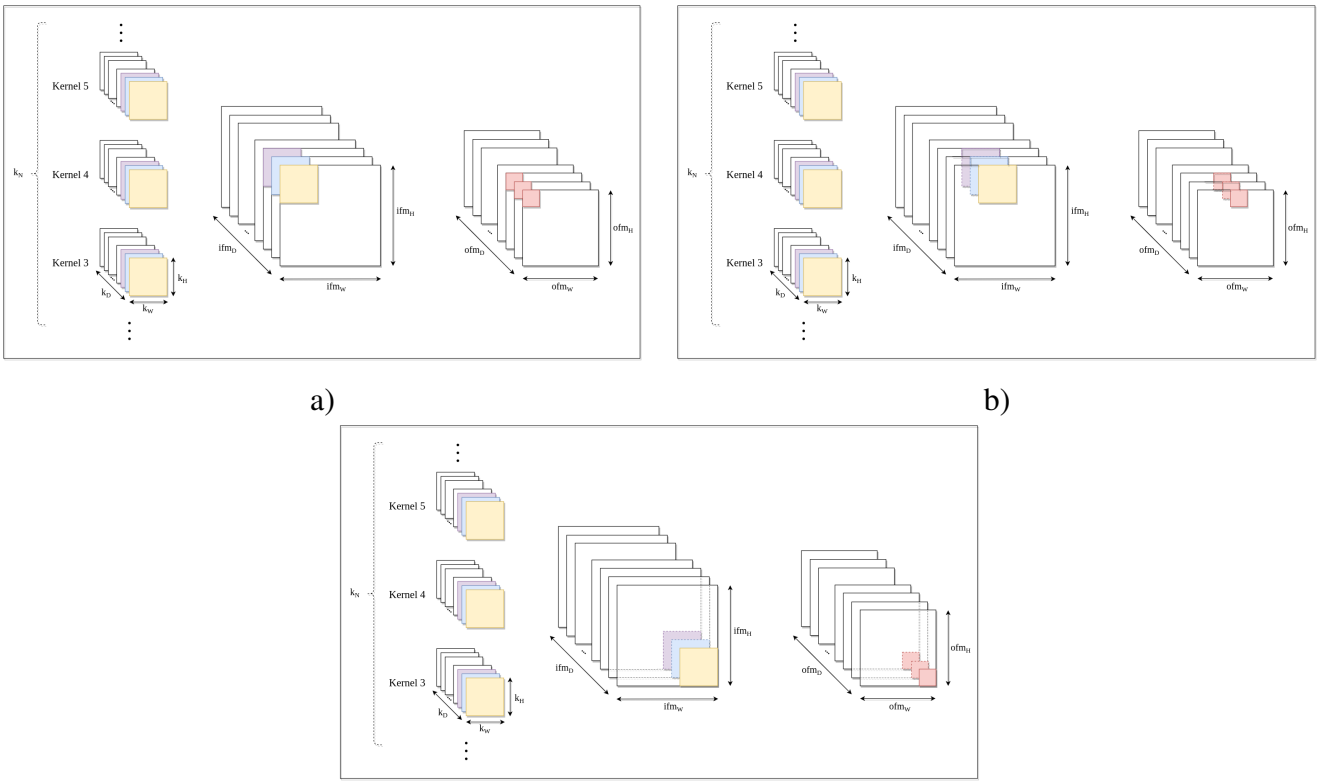
Quá trình tính toán trên Convolution Layer được thực hiện tuần tự như sau.



Hình 4.19: Quá trình tính toán với tập Kernel là $(0, 1, 2)$ ở mức độ sâu là $(0, 1, 2)$

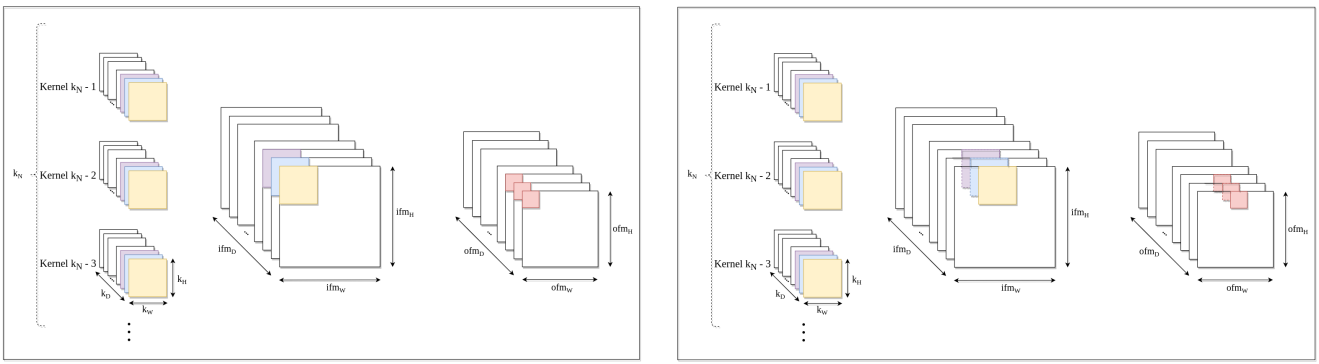
Bước đầu tiên, quy trình tính toán được thực hiện ở tập Kernel là $(0, 1, 2)$ với các giá trị đầu vào Input Feature Map bắt đầu ở vị trí khởi đầu là phía trái trên cùng ở mức độ sâu là $(0, 1, 2)$ (thể hiện ở hình 1.19a). Tiếp theo, các giá trị đầu vào Input Feature Map tiếp theo được đưa vào tính toán còn tập Kernel vẫn giữ nguyên (thể hiện ở hình 1.19b). Quá trình thay thế Input Feature Map được tiếp tục

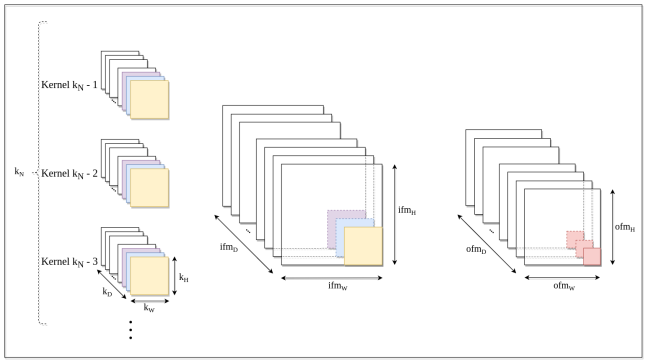
cho đến những giá trị cuối cùng ở mức độ sâu là $(0, 1, 2)$ (thể hiện ở hình 1.19c).



Hình 4.20: Quá trình tính toán với tập Kernel là $(3, 4, 5)$ ở mức độ sâu là $(0, 1, 2)$

Bước tiếp theo, quy trình tính toán được lặp lại khi giữ nguyên mức độ sâu là $(0, 1, 2)$, đưa giá trị đầu vào trở lại vị trí khởi đầu ở phía trái trên cùng và chuyển qua tập Kernel tiếp theo là tập $(4, 5, 6)$ (thể hiện ở hình 1.20a). Quá trình cũng tương tự ở bước đầu tiên khi các giá trị đầu vào Input Feature Map tiếp theo được đưa vào tính toán còn tập Kernel vẫn giữ nguyên (thể hiện ở hình 1.19b). Quá trình thay thế Input Feature Map được tiếp tục cho đến những giá trị cuối cùng ở mức độ sâu là $(4, 5, 6)$.

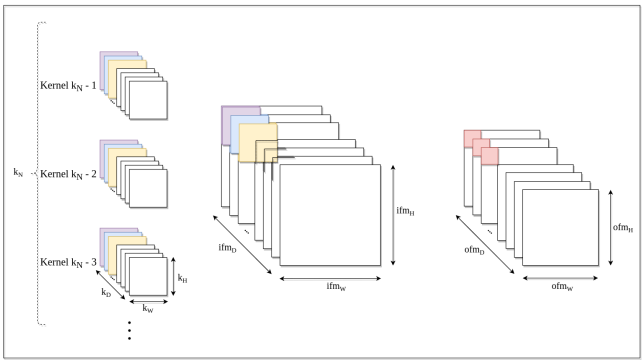




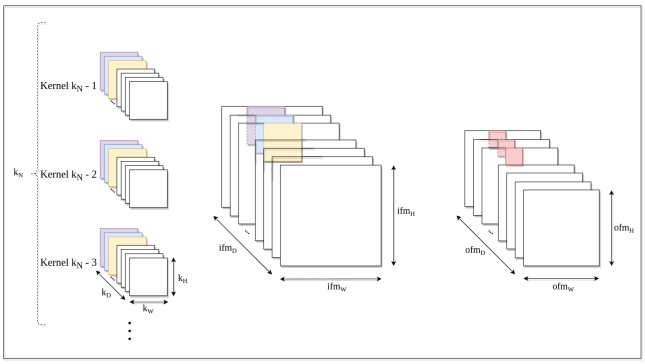
c)

Hình 4.21: Quá trình tính toán với tập Kernel là $(k_N - 3, k_N - 2, k_N - 1)$ ở mức độ sâu là $(0, 1, 2)$

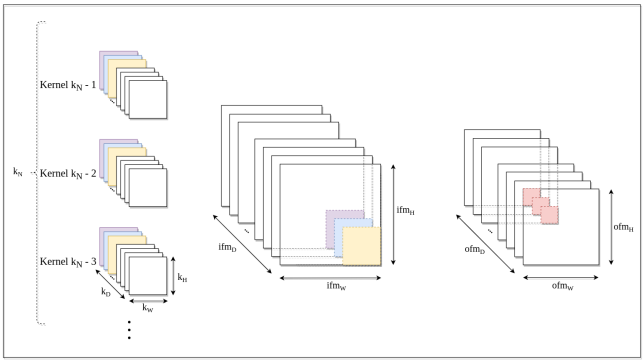
Quá trình tăng tập Kernel diễn ra cho tới khi đạt tập $(k_N - 3, k_N - 2, k_N - 1)$. Quy trình tính toán trên tập Kernel $(k_N - 3, k_N - 2, k_N - 1)$ vẫn diễn ra như thường lệ (thể hiện ở các hình 1.21a, 1.21b và 1.21c) ở mức độ sâu là $(0, 1, 2)$. Sau khi thực hiện xong việc tính toán bộ giá trị cuối cùng của Input Feature Map, mức độ sâu cần tiến toán cho bước tiếp theo tăng lên thành $(4, 5, 6)$. Quá trình tính toán diễn ra giống như ban đầu với mức độ sâu trở $(4, 5, 6)$.



a)



b)



c)

Hình 4.22: Quá trình tính toán với tập Kernel là $(k_N - 3, k_N - 2, k_N - 1)$ ở mức độ sâu là $(k_D - 3, k_D - 2, k_D - 1)$

Cho tới khi độ sâu đạt $(k_D - 3, k_D - 2, k_D - 1)$, tập Kernel đạt $(k_N - 3, k_N - 2, k_N - 1)$ và hoàn thàn một vòng tính toán trên Input Feature Map (thể hiện ở các hình 1.22a, 1.22b và 1.22c), quy trình tính toán trên Convolution Layer xem như đã hoàn thành.

TỔNG KẾT: Quá trình tính toán được hiểu như sau:

1. Giữ nguyên tập Kernel và mức độ sâu tính toán, tiến hành thay đổi giá trị đầu vào Input Feature Map.
2. Khi tính toán xong tất cả giá trị đầu vào Input Feature Map thì thay đổi tập Kernel.
3. Khi tính toán xong tất cả tập Kernel thì thay đổi mức độ sâu tính toán.
4. Khi tính toán xong tất cả mức độ sâu tính toán thì việc tính toán ở Convolution Layer hoàn tất

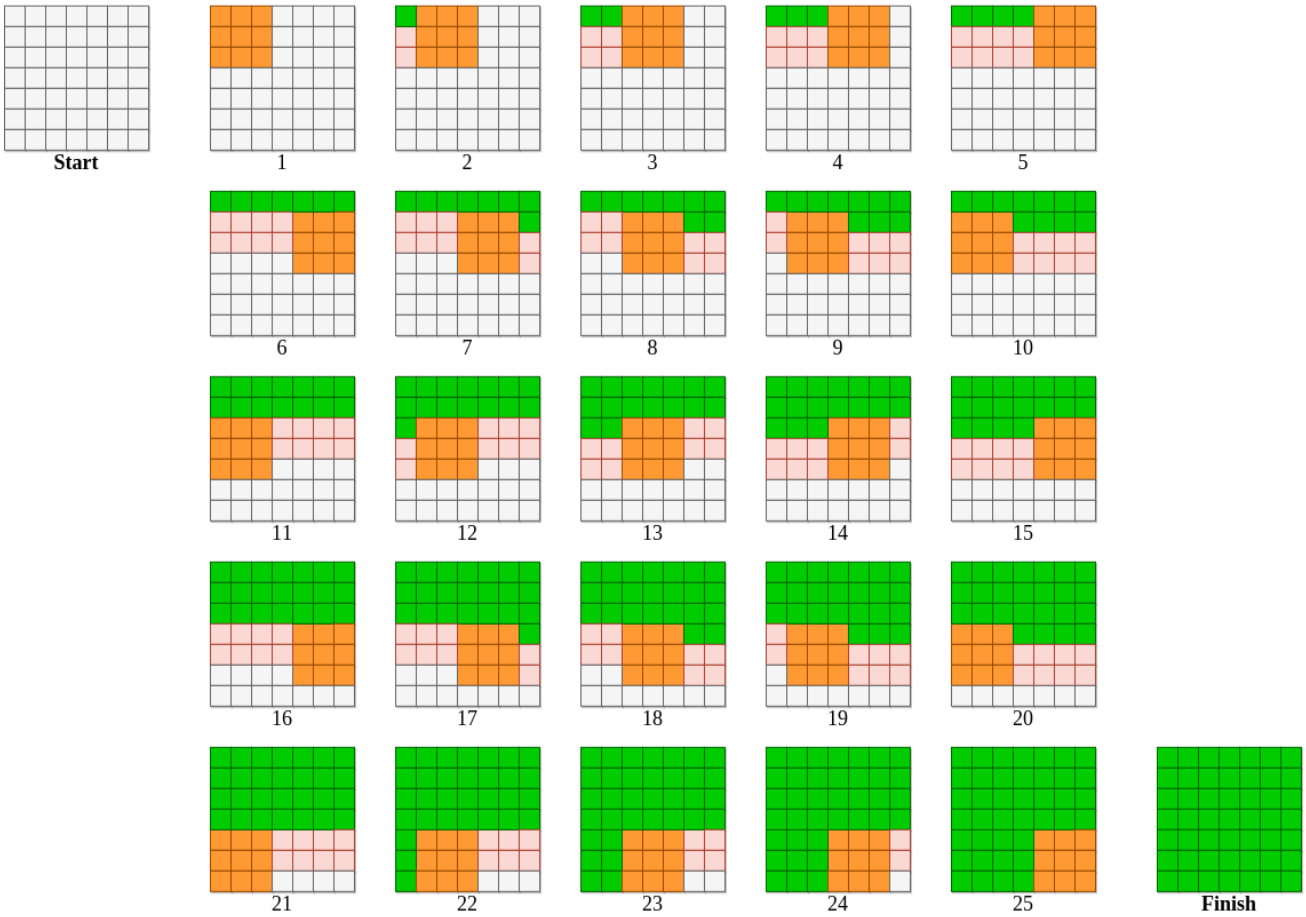
Mô hình thay đổi giá trị trên Input Feature Map ở Convolution Layer

Để tận dụng tính cục bộ của Input Feature Map trong phép tích chập, thuật toán thay đổi bộ giá trị đầu vào như sau:

1. Bắt đầu, tập giá trị Input Feature Map ở vị trí bắt đầu là góc trái trên cùng (giả thiết).
2. Dịch sang phải một lượng bằng đơn vị bước – $Stride_W$. Nếu như không thể dịch thêm thì chuyển sang bước 3, ngược lại thì ở lại bước 2.
3. Dịch xuống dưới một lượng bằng đơn vị bước – $Stride_H$. Nếu như không thể dịch xuống thì chuyển sang bước 6, ngược lại thì chuyển sang bước 4.
4. Dịch sang trái một lượng bằng đơn vị bước – $Stride_W$. Nếu như không thể dịch thêm thì chuyển sang bước 5, ngược lại thì ở lại bước 4.
5. Dịch xuống dưới một lượng bằng đơn vị bước – $Stride_H$. Nếu như không thể dịch xuống thì chuyển sang bước 6, ngược lại thì chuyển sang bước 2.
6. Kết thúc, toàn bộ Input Feature Map đã cung cấp đủ cho quá trình tính toán.

Để ý rằng, chỉ có ở bước 1 là cần phải mang toàn bộ dữ liệu Input Feature Map của một lần tính toán có kích thước là $k_W \times k_H$. Ở các bước dịch sang trái hoặc dịch sang phải, số lượng dữ liệu cần lấy thêm là $S_W \times k_H$. Còn ở các bước dịch xuống, số lượng dữ liệu cần lấy thêm là $k_W \times S_H$. Các thông số k_W, k_H, S_W và S_H là thông tin của Convolution Layer đã được định nghĩa ở những phần trước đó.

Bên dưới là ví dụ minh hoạt về thuật toán thay đổi giá trị tính toán trên Input Feature Map có kích thước 7×7, Kernel có kích thước 3×3 và Stride có giá trị là $Stride_W = 1$ và $Stride_H = 1$.



Hình 4.23: Ví dụ về thuật toán thay đổi giá trị tính toán trên Input Feature Map ở Convolution Layer

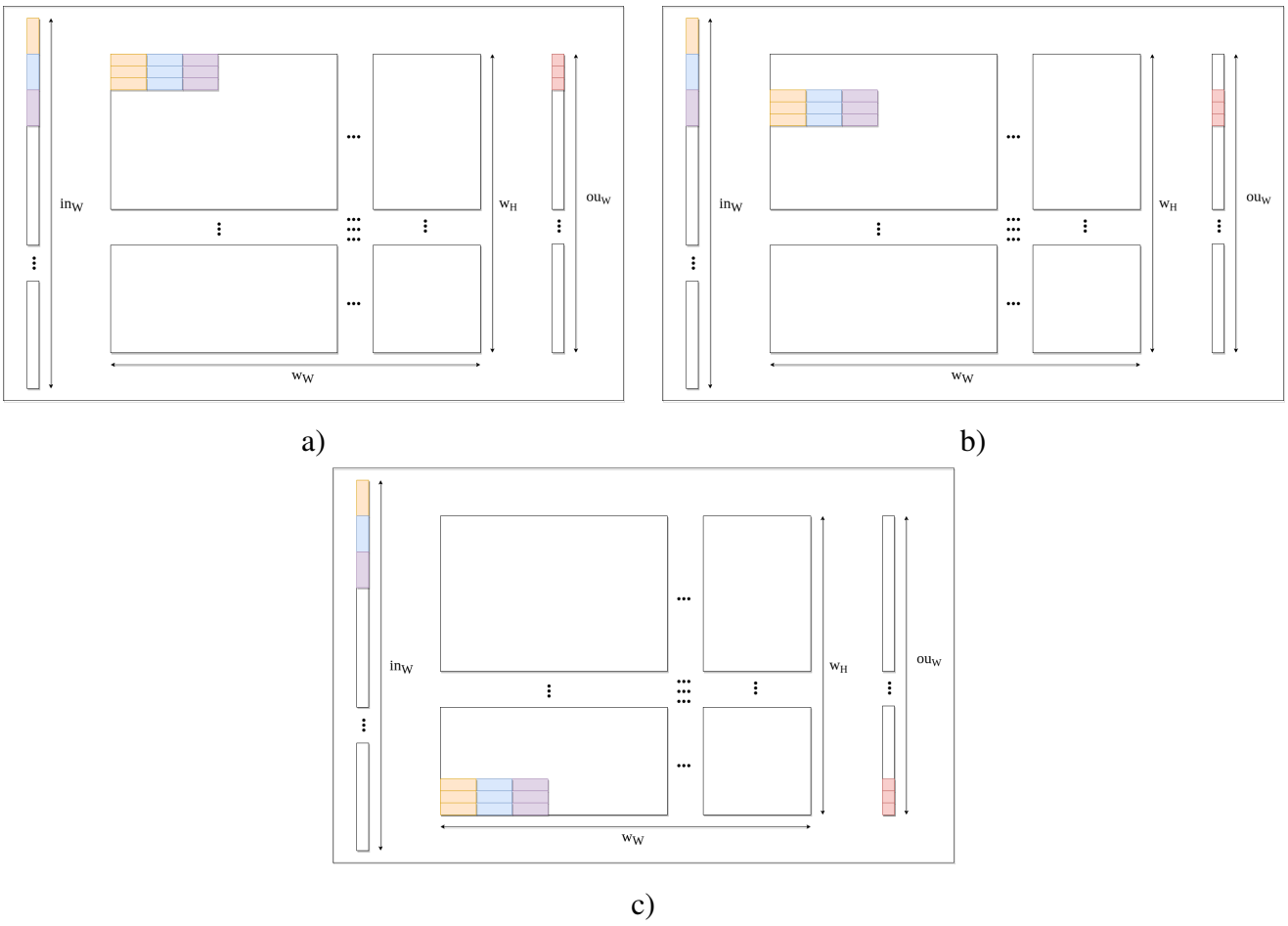
4.3.4.2 Fully-Connected Layer

Như đã trình bày, mô hình tính toán được dùng ở Fully-Connected Layer là *Input-Based Computation* được phát triển từ việc tận dụng tính tái sử dụng ở Input Activation.

Trước khi bắt đầu. lưu ý rằng một lần tính toán cho phép:

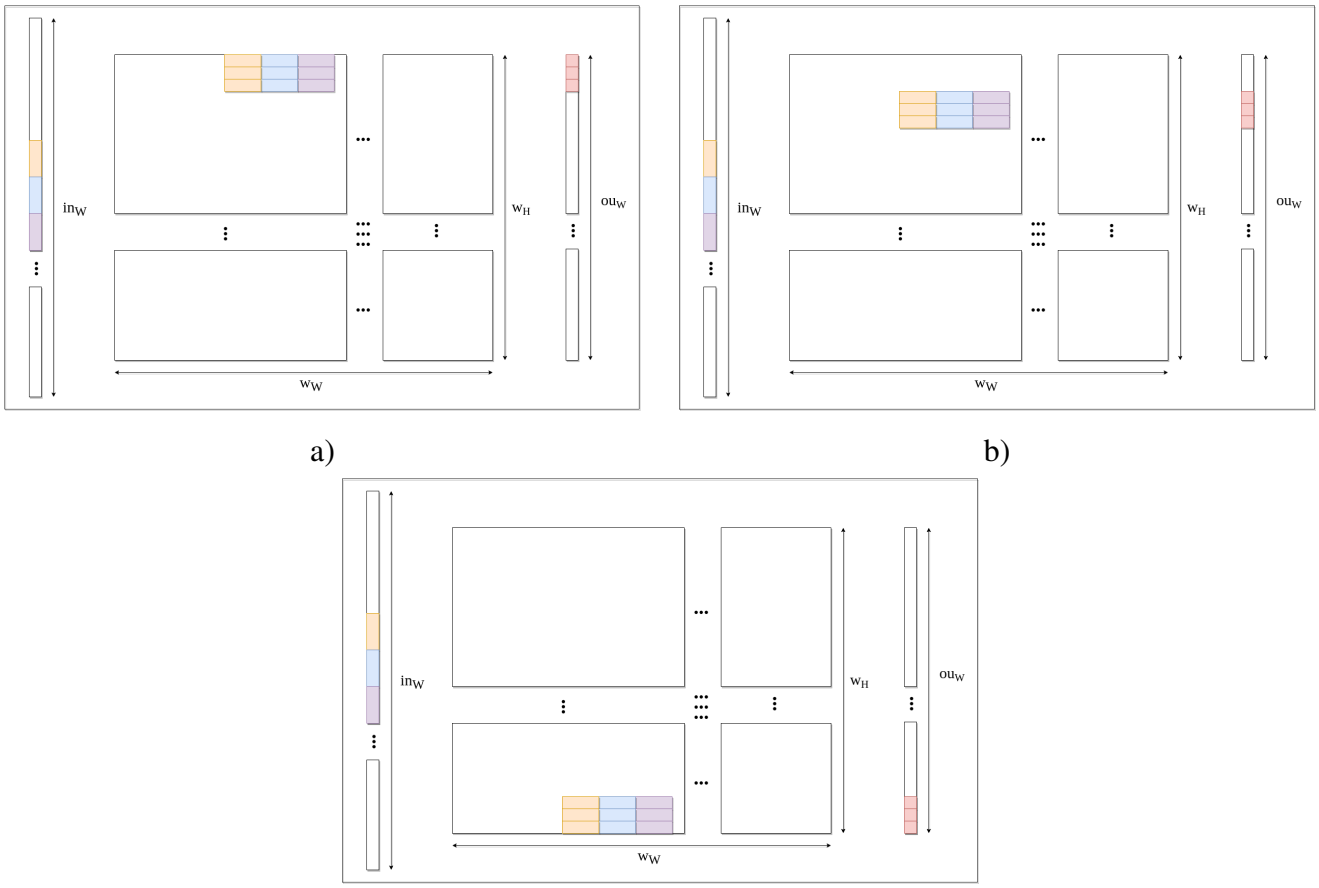
- Tính toán với một tập Input Activation có kích thước nhất định.
- Tính toán với một tập Weight có kích thước gấp $N_1 = 3$ lần kích thước Input Activation.

Quá trình tính toán trên Fully-Connected Layer được thực hiện tuần tự như sau:



Hình 4.24: Quá trình tính toán với ở tập Input Activation đầu tiên

Quy trình tính toán ở Fully-Connected Layer đơn giản hơn nhiều lần ở Convolution Layer. Đầu tiên, tính toán được thực hiện trên tập Input Activation đầu tiên và tập giá trị Weight tương ứng với Input Activation đó và tương ứng với tập Output Activation $(0, 1, 2)$ (thể hiện ở hình 1.24a). Tiếp theo, giữ nguyên tập Input Activation và thay đổi tập giá trị Weight tương ứng với Input Activation đó và tương ứng với tập Output Activation tiếp theo là tập $(3, 4, 5)$ (thể hiện ở hình 1.24b). Quá trình thay thế Weight này được tiếp tục cho đến những giá trị cuối cùng của Weight tương ứng với Input Activation ban đầu và tương ứng với tập Output Activation cuối cùng (thể hiện ở hình 1.24c).

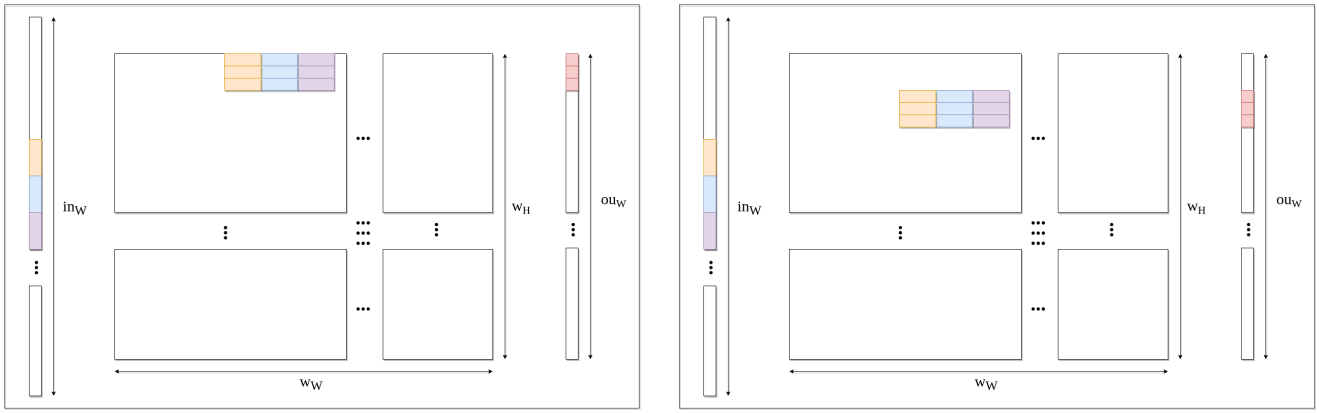


a) b)

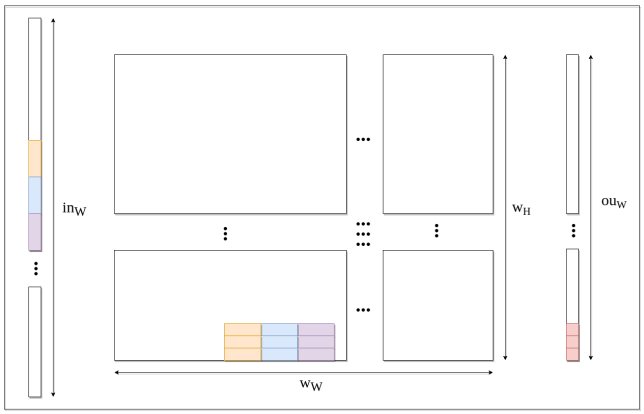
c)

Hình 4.25: Quá trình tính toán với ở tập Input Activation thứ hai

Bước tiếp theo, quy trình tính toán được lặp lại khi tập Input Activation được chuyển sang tập tiếp theo và tập giá trị Weight tương ứng với tập Input Activation đó và tương ứng với tập Output Activation ban đầu là $(0, 1, 2)$ (thể hiện ở hình 1.25a). Quá trình tính toán tương tự diễn ra khi tiếp tục thay đổi tập giá trị Weight tương ứng ở các tập Output Activation tiếp theo là $(3, 4, 5)$ (thể hiện ở hình 1.25b) và cho tới tập Weight tương ứng với tập Output Activation cuối cùng (thể hiện ở hình 1.25c).



a) b)



c)

Hình 4.26: Quá trình tính toán với ở tập Input Activation cuối cùng

Cho tới khi tập Input Activation tham gia quá trình tính toán, tập giá trị Weight tương ứng với tập Input Activation cuối cùng được thay đổi tương ứng với tập Output Activation từ đầu cho tới cuối (thể hiện ở hình 1.26a, 1.26b và 1.26c). Khi đó, quy trình tính toán trên Fully-Connected Layer xem như đã hoàn thành.

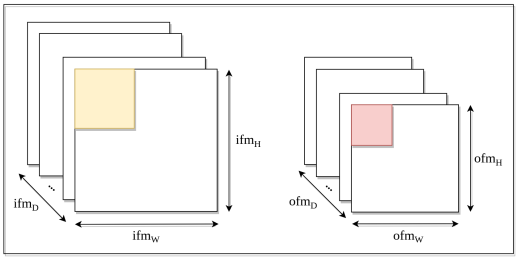
TỔNG KẾT: Quá trình tính toán được hiểu như sau:

1. Giữ nguyên tập Input Activation, tiến hành thay đổi tập giá trị Weight tương ứng với Input Activation.
2. Khi tính toán xong tất cả giá trị Weight tương ứng với Input Activation thì thây đổi tập Input Activation.
3. Khi tính toán xong tất cả giá trị Input Activation thì việc tính toán ở Fully-Connected Layer hoàn tất.

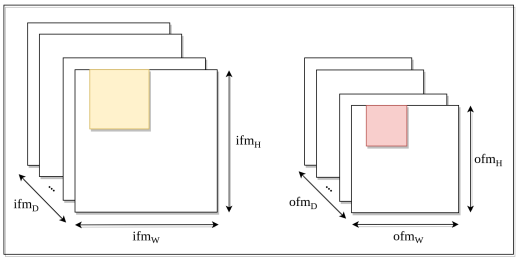
4.3.4.3 Pooling Layer

Mô hình tính toán ở Pooling Layer tương tự với Convolution Layer. Điểm khác biệt duy nhất là ở Pooling Layer không có tập giá trị Kernel nên trong quy trình tính toán bỏ qua các bước có liên quan tới Kernel.

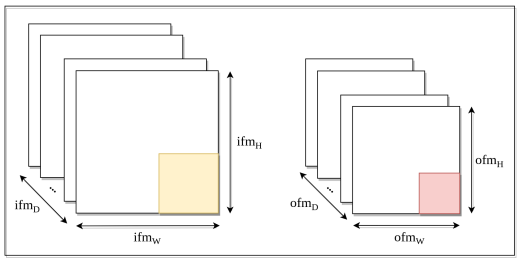
Quá trình tính toán trên Pooling Layer được thực hiện tuần tự như sau:



a)



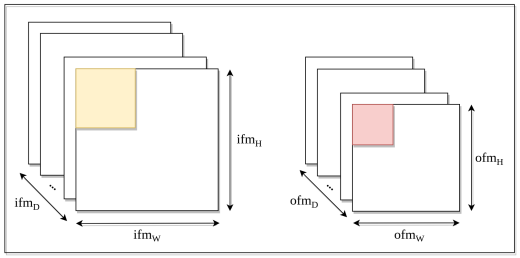
b)



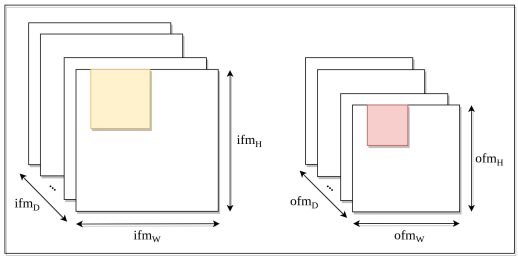
c)

Hình 4.27: Quá trình tính toán với ở mức độ sâu tính toán đầu tiên

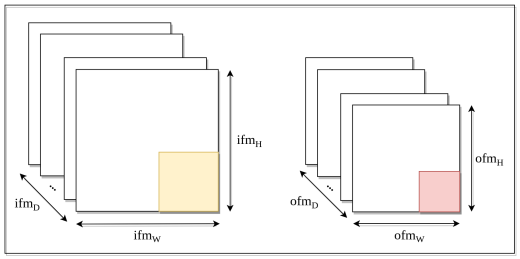
Đầu tiên, tính toán được thực hiện trên tập Input Feature Map đầu tiên ở góc trái trên cùng (giả thiết) (thể hiện ở hình 1.27a). Tiếp theo, các giá trị Input Feature Map được thay đổi sau tập giá trị tiếp theo (thể hiện ở hình 1.27b) cho tới tập giá trị cuối cùng (thể hiện ở hình 1.27c).



a)



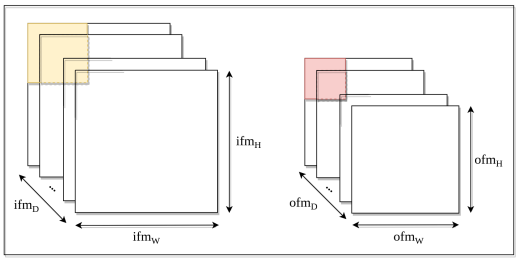
b)



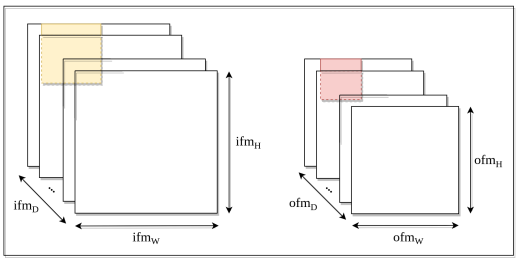
c)

Hình 4.28: Quá trình tính toán với ở mức độ sâu tính toán tiếp theo

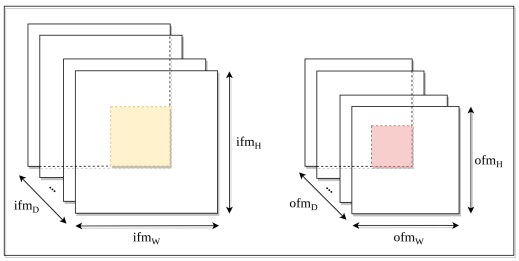
Tiếp theo, quá trình tính toán được lặp lại khi tăng mức độ sâu giá trị tiếp theo và đưa tập Input Feature Map về vị trí khởi đầu ở góc trái trên cùng (giả thiết) (thể hiện ở hình 1.28a). Tương tự, quy trình tính toán được thực hiện trên các giá trị Inptu Feature Map thay đổi từ tập giá trị tiếp theo tập đầu (thể hiện ở hình 1.28b) cho tới tập giá trị cuối cùng (thể hiện ở hình 1.28c).



a)



b)



c)

Hình 4.29: Quá trình tính toán với ở mức độ sâu tính toán cuối cùng

Quá trình tính toán tiếp tục cho đến khi đạt mức độ sâu tính toán cuối cùng. Tương tự, quy trình được lặp lại ở tập Input Feature Map từ tập giá trị ban đầu cho tới tập giá trị cuối cùng (thể hiện ở hình 1.29a, 1.29b và 1.29c). Sau khi thực hiện xong tính toán này, quy trình tính toán trên Pooling Layer xem như đã hoàn thành.

TỔNG KẾT: Quá trình tính toán được hiểu như sau:

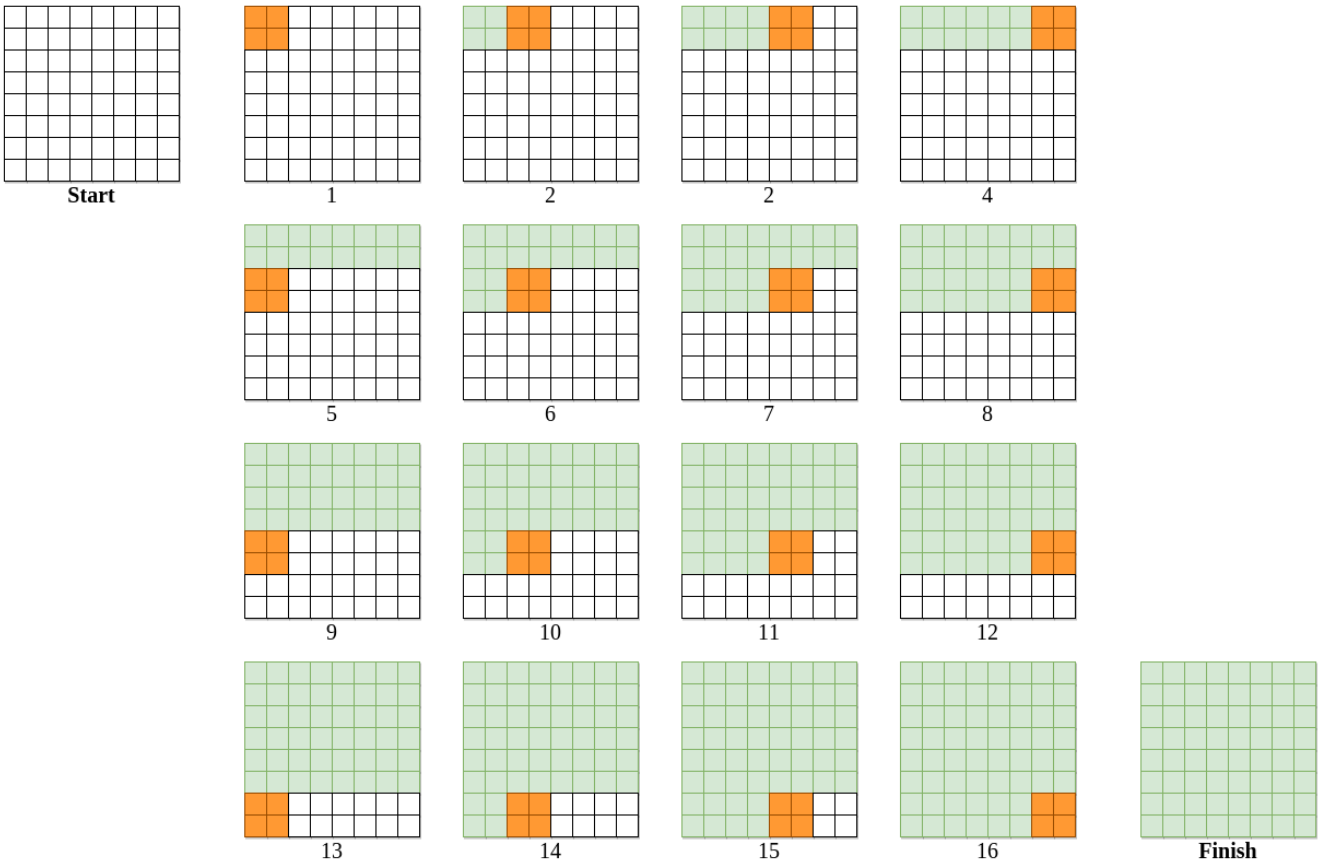
1. Giữ nguyên mức độ sâu tính toán, tiến hành thay đổi giá trị đầu vào Input Feature Map.
2. Khi tính toán xong tất cả giá trị đầu vào Input Feature Map thì thay đổi mức độ sâu tính toán.
3. Khi tính toán xong tất cả mức độ sâu tính toán thì việc tính toán ở Pooling Layer hoàn tất

Mô hình thay đổi giá trị trên Input Feature Map ở Pooling Layer

Ở Pooling Layer, có rất ít tính cục bộ của Input Feature Map có thể khai thác. Do đó, thuật toán thay đổi bộ giá trị đầu vào trở nên đơn giản hơn so với Convolution Layer như sau:

1. Bắt đầu, tập giá trị Input Feature Map ở vị trí bắt đầu là góc trái trên cùng (giả thiết)
2. Dịch sang phải một lượng bằng đơn vị bước – $Stride_W$. Nếu như không thể dịch thêm thì chuyển sang bước 3, ngược lại thì ở lại bước 2.
3. Đưa về đầu hàng đang làm việc và dịch xuống một lượng bằng đơn vị bước – $Stride_H$. Nếu như không thể dịch xuống thì chuyển sang bước 4, ngược lại thì chuyển bước 2.
4. Kết thúc, toàn bộ Input Feature Map đã cung cấp đủ cho quá trình tính toán.

Bên dưới là ví dụ minh hoạt về thuật toán thay đổi giá trị tính toán trên Input Feature Map có kích thước 8×8, Patch có kích thước là 2×2 và Stride có giá trị là $Stride_W = 2$ và $Stride_H = 2$.



Hình 4.30: Ví dụ về thuật toán thay đổi giá trị tính toán trên Input Feature Map ở Pooling Layer

Chương 5

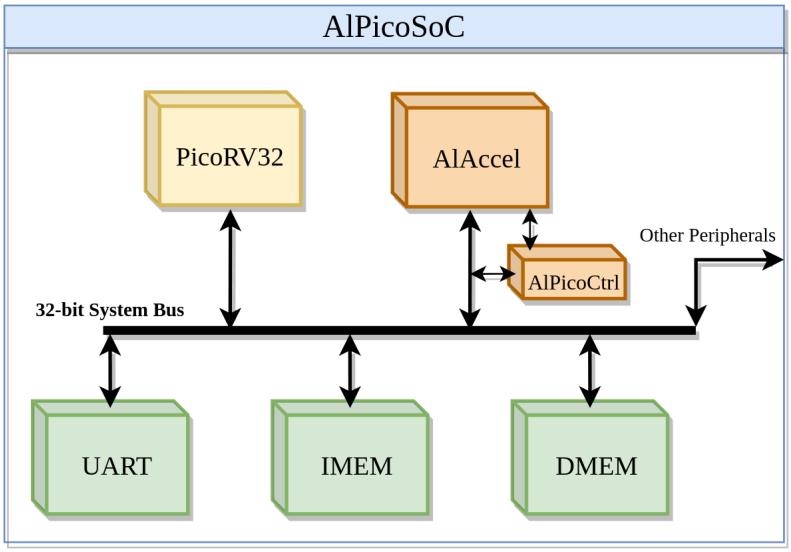
THIẾT KẾ HỆ THỐNG

5.1 Kiến trúc phần cứng

Hệ thống – được gọi là AlPicoSoC – lấy nền tảng dựa trên PicoSoC được tinh chỉnh cho phù hợp, sau đó tích thêm bộ tăng tốc AI có tên là Al Accelerator vào.

Về PicoSoC, đây là một SoC đơn giản lấy lõi RISC-V PicoRV32 làm CPU để chạy mã lệnh. PicoSoC chỉ có ngoại vi là UART để giao tiếp với thiết bị khác và bộ giao tiếp SPI chỉ phục vụ cho việc đọc dữ liệu từ SPI Flash. Thêm nữa, PicoSoC chạy mã lệnh trực tiếp trên bộ nhớ SPI Flash nên tốc độ đọc lệnh tương đối chậm. Nhìn chung, SoC này tương đối đơn giản, dễ hiểu, phù hợp cho việc tinh chỉnh và có thể phát triển thêm.

Bên dưới là hình minh họa về kiến trúc của AlPicoSoC.



Hình 5.1: Sơ đồ khối về kiến trúc tổng quát của hệ thống AlPicoSoC

AlPicoSoC bao gồm:

- *PicoRV32*: Đây là lõi RISC-V đóng vai trò chính trong việc điều khiển hoạt động của SoC bao gồm việc thực hiện các tính toán thông thường, quản lý bộ nhớ và các điều khiển hoạt động giao

tiếp của ngoại vi.

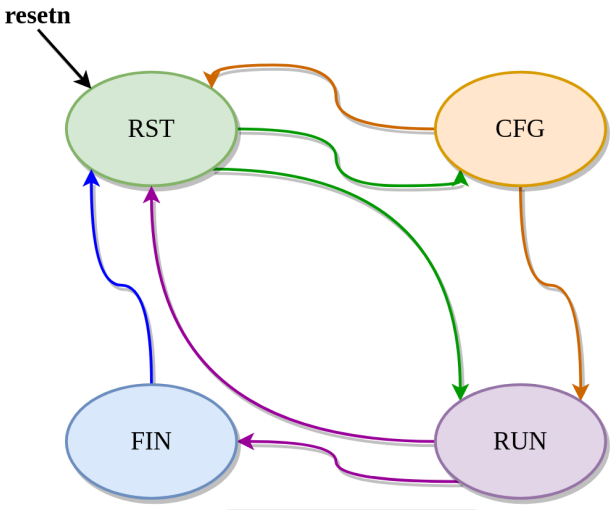
- *Alpha Accelerator*: Đây là thành phần tính toán đặc thù cho tác vụ suy luận của mô hình Deep Learning. Khối này có khả năng tính toán cấp Layer – nghĩa là một quá trình tính toán ở khối này diễn ra trên một Layer của mô hình Deep Learning.
- *IMEM*: Đây là bộ nhớ lưu trữ các lệnh thuộc RISC-V ISA để cho lõi RISC-V *PicoRV32* chạy trên đây. Đối với các khối chức năng bên trong AlPicoSoC thì đây là bộ nhớ chỉ đọc, việc ghi dữ liệu phục vụ cho việc thực thi được thực hiện bằng các thành phần bên ngoài AlPicoSoC.

  Việc thay thế bộ nhớ lệnh từ việc đọc trực tiếp từ SPI Flash sang đọc giá trị *IMEM* được hiện thực như Block RAM giúp giảm số chu kỳ xung clock cần thiết để đọc và ghi dữ liệu.
- *DMEM*: Đây là bộ nhớ bay hơi dùng để lưu trữ dữ liệu trong quá trình thực thi chương trình của lõi *PicoRV32*. Hiện tại, *DMEM* thiết kế theo dạng Dual-Port RAM cho phép đọc và ghi dữ liệu diễn ra song song với nhau.
- *UART*: Đây là ngoại vi duy nhất của AlPicoSoC đóng vai trò thực hiện các giao tiếp UART với các thiết bị kết nối bên ngoài SoC có thể là cảm biến, thiết bị điều khiển, một SoC khác,...
- *32-bit System Bus*: Đóng vai trò kết nối tất cả thành phần của SoC với nhau. Hiện tại, đây chỉ là một dạng bus đơn giản với *PicoRV32* đóng vai trò điều khiển quá trình đọc và ghi dữ liệu với các thành phần khác. Có thể thay thế bus hiện tại hơn bằng cách dạng khác như AXI bus,...

Ngoài ra, ở AlPicoSoC còn có thêm một khối điều khiển đặc thù cho bộ tăng tốc *Alpha Accelerator*. Khối này là *Alpha Accelerator Pico Controller* đóng vai trò đơn giản hóa việc giao tiếp giữa lõi *PicoRV32* và *Alpha Accelerator*.

5.1.1 Khối Alpha Accelerator Pico Controller

Khối Alpha Accelerator Pico Controller đóng vai trò là cầu nối giao tiếp giữa lõi PicoRV32 và Alpha Accelerator. Việc điều khiển bộ tăng tốc được thực hiện thông qua việc đọc và ghi một vùng địa chỉ của những thanh ghi đặc biệt của khối điều kiển này.



Hình 5.2: Máy trạng thái hoạt động của khối Alpha Accelerator Pico Controller

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
RST	*CFG*	Thanh ghi *AL_ACCEL_CTRL* của khối điểu khiển này được ghi giá trị **CFG** (tương ứng với giá trị 1)	*RST* là trạng thái bắt đầu của máy trạng thái. Nhiệm vụ của trạng thái *RST* là cài lại toàn bộ giá trị của bộ nhớ đệm, thanh ghi và các máy trạng thái bên trong các khối chức năng và khối điều khiển (trừ tập thanh ghi điều khiển) của Alpha Accelerator.
	RUN	Thanh ghi *AL_ACCEL_CTRL* của khối điểu khiển này được ghi giá trị **RUN** (tương ứng với giá trị 2)	
CFG	*RST*	Thanh ghi *AL_ACCEL_CTRL* của khối điểu khiển này được ghi giá trị **RST** (tương ứng với giá trị 0)	Nhiệm vụ của trạng thái *CFG* là thiết đặt các giá trị cho tập thanh ghi điều khiển của Alpha Accelerator tùy thuộc vào yêu cầu và mong muốn của người dùng về Layer làm việc.
	RUN	Thanh ghi *AL_ACCEL_CTRL* của khối điểu khiển này được ghi giá trị **RUN** (tương ứng với giá trị 2)	
RUN	*RST*	Thanh ghi *AL_ACCEL_CTRL* của khối điểu khiển này được ghi giá trị **RST** (tương ứng với giá trị 0)	Nhiệm vụ của trạng thái *RUN* là thiết đặt cho bộ tăng tốc Accelerator bắt đầu thực hiện tác vụ tính toán ở cấp độ Layer của mình.
	FIN	Alpha Accelerator thông báo hoàn thành công việc tính toán của mình.	
FIN	*RST*	Thanh ghi *AL_ACCEL_CTRL* của khối điểu khiển này được ghi giá trị **RST** (tương ứng với giá trị 0)	Nhiệm vụ của trạng thái *FIN* là thông báo rằng công việc tính toán đã xong và đặt Alpha Accelerator về trạng thái nghỉ.

Bảng 5.1: Bảng mô tả trạng thái của khối Alpha Accelerator Pico Controller

Nhờ có khối điều khiển này mà công việc lập trình, sử dụng và giao tiếp với bộ tăng tốc trở nên đơn giản, dễ dàng và dễ tiếp cận hơn.

5.1.2 Không gian địa chỉ

Bên dưới làm bảng mô tả về không gian địa chỉ của hệ thống AlPicoSoC. Những khoảng địa chỉ không được đề cập trong bảng là những địa chỉ không dùng tới trong hệ thống AlPicoSoC.

Nhóm	Khoảng địa chỉ	Mô tả
DMEM	0x0000_0000 .. 0x00FF_FFFF	Khoảng địa chỉ lưu trữ của bộ nhớ dữ liệu DMEM.
IMEM	0x0100_0000 .. 0x01FF_FFFF	Khoảng địa chỉ lưu trữ của bộ nhớ lệnh IMEM.
UART	0x0200_0004 .. 0x0200_0007	Địa chỉ thanh ghi điều khiển việc chia tần số phục vụ cho tác vụ giao tiếp ở các BAUDRATE khác nhau của khối UART.
	0x0200_0004 .. 0x0200_0007	Địa chỉ thanh ghi lưu trữ dữ liệu được gửi/nhận của khối UART.
AL_ACCEL	0x0200_1000 .. 0x0200_1003	Địa chỉ thanh ghi cấu hình 0 của Alpha Accelerator.
	0x0200_1004 .. 0x0200_1007	Địa chỉ thanh ghi cấu hình 1 của Alpha Accelerator.
	0x0200_1008 .. 0x0200_100B	Địa chỉ thanh ghi cấu hình 2 của Alpha Accelerator.
	0x0200_100C .. 0x0200_100F	Địa chỉ thanh ghi cấu hình 3 của Alpha Accelerator.
	0x0200_1010 .. 0x0200_1013	Địa chỉ thanh ghi cấu hình 4 của Alpha Accelerator.
	0x0200_1014 .. 0x0200_1017	Địa chỉ thanh ghi cấu hình 5 của Alpha Accelerator.
	0x0200_1018 .. 0x0200_101B	Địa chỉ thanh ghi cấu hình 6 của Alpha Accelerator.
	0x0200_101C .. 0x0200_101F	Địa chỉ thanh ghi cấu hình 7 của Alpha Accelerator.
	0x0200_1020 .. 0x0200_1023	Địa chỉ thanh ghi cấu hình 8 của Alpha Accelerator.
	0x0200_1024 .. 0x0200_1027	Địa chỉ thanh ghi cấu hình 9 của Alpha Accelerator.
	0x0200_1028 .. 0x0200_102B	Địa chỉ thanh ghi cấu hình 10 của Alpha Accelerator.
	0x0200_102C .. 0x0200_102F	Địa chỉ thanh ghi cấu hình 11 của Alpha Accelerator.
	0x0200_1030 .. 0x0200_1033	Địa chỉ thanh ghi cấu hình 12 của Alpha Accelerator.
	0x0200_1034 .. 0x0200_1037	Địa chỉ thanh ghi cấu hình 13 của Alpha Accelerator.
	0x0200_1038 .. 0x0200_103B	Địa chỉ thanh ghi cấu hình 14 của Alpha Accelerator.
	0x0200_103C .. 0x0200_103F	Địa chỉ thanh ghi cấu hình 15 của Alpha Accelerator.
	0x0200_1040 .. 0x0200_1043	Địa chỉ thanh ghi cấu hình 16 của Alpha Accelerator.
	0x0200_1050 .. 0x0200_1053	Địa chỉ thanh ghi điều khiển của Alpha Accelerator Pico Controller.
IO	0x0300_0000 .. 0xFFFF_FFFF	Khoảng địa chỉ dùng cho các thiết bị ngoại vi của người dùng.

Bảng 5.2: Bảng mô tả không gian địa chỉ được sử dụng trong AlPicoSoC

Dựa trên bảng không gian địa chỉ này, người dùng có thể dựa trên đó để xây dựng các chương trình phần mềm phù hợp với mục đích sử dụng của mình.

5.2 Nền tảng phần mềm

Xây dựng một hệ thống SoC phục vụ cho mục đích đặc thù của người dùng không chỉ dừng lại ở việc phát triển hệ thống phần cứng mà còn cần phải phát triển một hệ thống phần mềm đi kèm phục vụ cho phần cứng của SoC.

Nền tảng phần mềm của AlPicoSoC phục vụ cho tác vụ suy luận trên mô hình Deep Learning bao gồm các thành phần chính như sau:

- Tập công cụ phần mềm liên quan tới tác vụ đào tạo mô hình Deep Learning.
- Tập công cụ phần mềm liên quan tới việc biên dịch ngôn ngữ C về ngôn ngữ máy.
- Tập thư viện ngôn ngữ C liên quan tới tác vụ suy luận mô hình Deep Learning.
- Tập thư viện ngôn ngữ C phục vụ cho AlPicoSoC về tác vụ cơ bản thay thế cho một số thư viện C chuẩn (ví dụ như là tác vụ IO).

5.2.1 Công cụ đào tạo mô hình Deep Learning

Trong nền tảng phần mềm của AlPicoSoC, công cụ đào tạo mô hình Deep Learning được dùng là TensorFlow Lite.

TensorFlow Lite là một tập các công cụ cho phép việc phát triển học máy trên các thiết bị di động, thiết bị nhúng và thiết bị biên bằng việc hỗ trợ người dùng triển khai các mô hình trên đó [23].

Đặc tính nổi trội của TensorFlow Lite:

- Tối ưu hóa cho việc phát triển học máy cấp thiết bị dựa trên 5 yếu tố chính:
  - Độ trễ: Không cần thời gian phải giao tiếp với các mô hình trên server để lấy kết quả, thay vào đó mô hình được chạy trên thiết bị.
  - Tính riêng tử: Không có dữ liệu cá nhân rời thiết bị khi thực hiện chạy mô hình trên thiết bị.
  - Tính kết nối: Không cần phải kết nối với mạng để chạy mô hình.
  - Kích thước: Có hỗ trợ giảm kích thước của mô hình và tập nhị phân chạy chương trình trên thiết bị.
  - Năng lượng: suy luận trên mô hình một cách hiệu và không cần kết nối mạng làm giảm năng lượng tiêu hao.
- Hỗ trợ cho nhiều loại bao gồm thiết bị Android/IoS, thiết bị nhúng hỗ trợ Linux và thiết bị vi xử lý.
- Hỗ trợ cho nhiều loại ngôn ngư bao gồm Java, Swift, C/C++ và Python.
- Hiệu suất cao bằng việc sử dụng tăng tốc phần cứng và tối ưu hóa mô hình.
- Nhiều ví dụ có sẵn cho các tác vụ thông dụng trong học máy như phân biệt hình ảnh, nhận diện đối tượng, phân loại chữ viết,...

Nguyên nhân để lựa chọn TensorFlow Lite:

- Thứ nhất, các thế mạnh kể trên của TensorFlow Lite phù hợp với yêu cầu của hệ thống AlPicoSoC.
- Thứ hai, TensorFlow Lite có một cộng đồng người dùng rộng lớn hỗ trợ hiệu quả trong việc phát triển các mô hình Deep Learning cũng như giúp đỡ các vấn đề khi sử dụng TensorFlow Lite.
- Thứ ba, TensorFlow Lite hỗ trợ huấn luyện mô hình Deep Learning lượng tự hóa về 8-bit phù hợp với yêu cầu mô hình trên hệ thống AlPicoSoC.

5.2.2 Công cụ biên dịch chương trình

Để chương trình C có thể chạy trên hệ thống, cần phải có công cụ để biên dịch chương trình C về dạng mã máy phù hợp với ISA của hệ thống đó. Đối với AlPicoSoC sử dụng lõi là PicoRV32, dĩ nhiên RISC-V GNU Compiler Toolchain là công cụ cần thiết và cực kỳ quan trọng để biên dịch chương trình C về ngôn ngữ máy của RISC-V.

RISC-V GNU Compiler Toolchain là trình biên dịch ngôn ngữ C và C++ về dạng mã máy chạy trên lõi RISC-V. Đây là công cụ miễn phí mã nguồn mở phù hợp tất cả thiết bị sử dụng tập ISA là RISC-V. RISC-V GNU Compiler Toolchain hỗ trợ hai chế độ hoạt động: một tập công cụ chung ELF/Newlib hoặc là một tập công cụ phức tập Linux-ELF/glibc [24].

5.2.3 Tập thư viện suy luận trên mô hình Deep Learning

Để có thể thực hiện tác vụ suy luận trên mô hình Deep Learning trên hệ thống AlPicoSoC, cần xây dựng một tập thư viện cung cấp các giao thức cơ bản phục vụ cho mô hình Deep Learning bằng ngôn ngữ C. Việc này có thể được thực hiện bằng cách sử dụng một tập phần mềm có sẵn như TensorFlow Lite, Cafe,... Tuy nhiên, vì có những chỉnh sửa cần thiết để thay đổi cách thức thực thi cấp Layer bằng bộ tăng tốc Alpha Accelerator. Do đó, để đơn giản và dễ kiểm soát, một tập thư viện tối giản phục vụ cho tác vụ suy luận ở mô hình Deep Learning được xây dựng.

Tập thư viện này được tham khảo và xây dựng dựa trên hai nguồn chính:

- *TensorFlow Lite*: Tham khảo tập thư viện C/C++ của TensorFlow Lite về cách hiện thực các Layer và thực hiện tính toán trên dữ liệu đượcp lượng tự hóa về 8-bit số nguyên [23].
- *gemmlowp*: Tham khảo tập thư viện C/C++ thực hiện tính toán liên quan tới phép tích chập trên ma trận ở độ chính xác thấp [25].

Hiện tại, tập thư viện này hỗ trợ tính toán ở cấp Layer với tập giá trị trọng số và đầu vào được biểu diễn theo kiểu dữ liệu $int8_t$ (số nguyên 8-bit) và cho ra kết quả tính toán là $int8_t$ (số nguyên 8-bit) ở 3 loại Layer là Convolution, Fully-Connected và Max Pooling ở hai chế độ có hỗ trợ và không có hỗ trợ từ Alpha Accelerator. Những tính toán này được hiện thực ở mức tối giản đúng theo lý thuyết tính toán ở từng loại Layer Deep Learning.

5.2.4 Tập thư viện đặc thù của AlPicoSoC

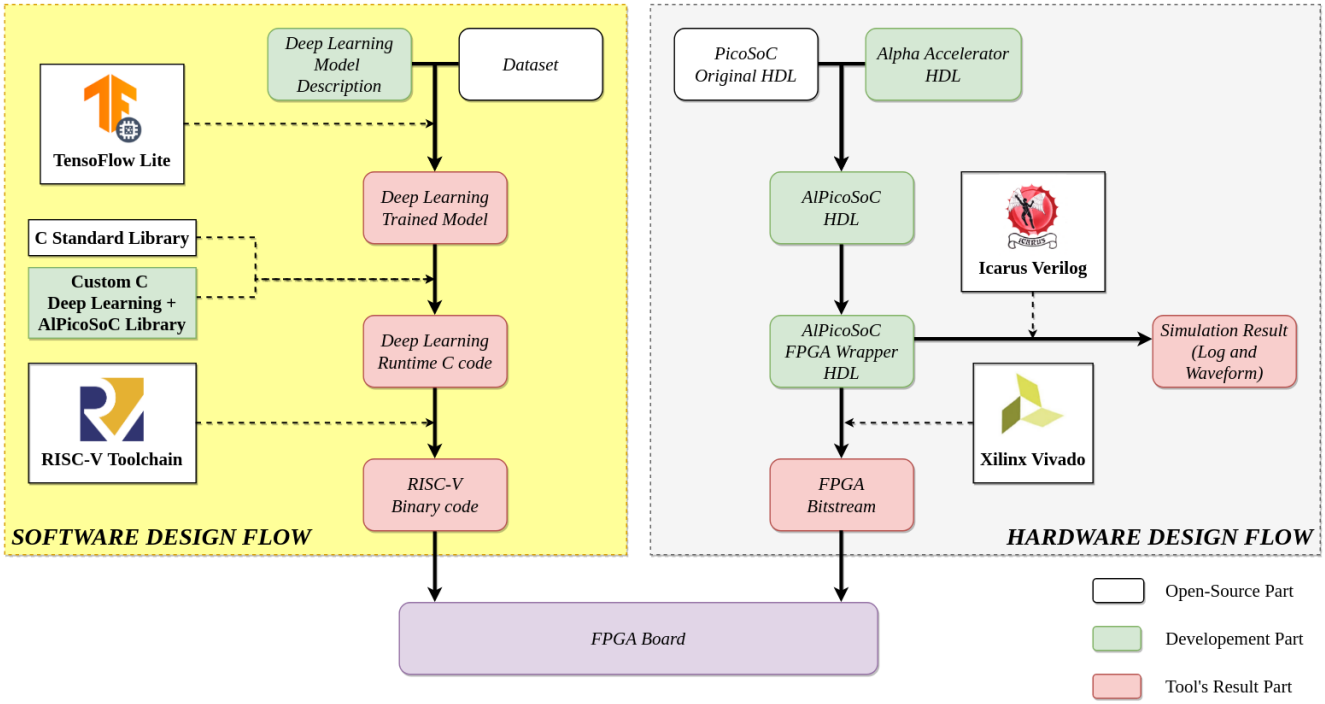
Một số thư viện chuẩn của ngôn ngữ C sẽ không thể thực hiện được vai trò của mình trên hệ thống AlPicoSoC. Cụ thể nhất là thư viện IO của C. Do đó, cần phải có một tập thư viện phục vụ cho các tác vụ đặc thù của AlPicoSoC.

Hiện tại, tập thư viện này mới chỉ hỗ trợ thay thế cho việc giao tiếp IO của hệ thống. Cụ thể, dữ liệu IO được lấy từ khối UART của AlPicoSoC hoặc thiết bị ngoài vi của người dùng bằng cách truy xuất đọc/ghi trên cá khoảng địa chỉ phù hợp được thể hiện ở bảng mô tả không gian địa chỉ.

Ngoài ra, tập thư viện này còn hỗ trợ cho việc cấu hình, điều khiển và sử dụng bộ tăng tốc Alpha Accelerator phục vụ cho các tính toán ở mô hình Deep Learning ở cấp độ Layer.

5.3 Thiết kế phần cứng–phần mềm

Quy trình thiết kế song song của phần cứng và phần mềm của hệ thống AlPicoSoC được thể hiện như hình bên dưới.



Hình 5.3: Quy trình đồng thiết kế phần cứng và phần mềm

Lưu ý ở trong hình có 3 màu với những ý nghĩa riêng:

- Thành phần có màu trắng là những thành phần có sẵn và miễn phí sử dụng.
- Thành phần có màu hồng là kết quả sau khi thực hiện của các công cụ tương ứng kết hợp với sự tinh chỉnh của người phát triển
- Thành phần có màu xanh là những thành phần tự thiết kế và phát triển.

Mô tả bằng lời, quá trình thiết kế giữa phần cứng và phần mềm diễn ra song song như sau:

- Quy trình phát triển phần cứng:
  - Đầu tiên, từ nền tảng gốc là hệ thống PicoSoC mã nguồn mở kết hợp với khối tăng tốc Alpha Accelerator, kiến trúc phần cứng của AlPicoSoC được tạo ra. Ở bước này, AlPicoSoC mới chỉ ở mức mã ngôn ngữ đặc tả phần cứng.
  - Tiếp theo, tùy theo loại FPGA được dùng để triển khai mà xây dựng một bộ đóng góp phù hợp cho AlPicoSoC – tạm gọi là AlPicoSoC Wrapper. ở bước này, AlPicoSoC Wrapper cũng chỉ mới ở mức mức mã ngôn ngữ đặc tả phần cứng.

    Ngoài ra, AlPicoSoC Wrapper có thể được mô phỏng bằng phần mềm để kiểm thử và đánh giá hành vi trước khi thực hiện các bước tiếp theo dựa trên các testbench tương ứng. Phần mềm mô phỏng được dùng là Icarus Verilog và kết quả thu được khi mô phỏng bao gồm

log file (thực chất là kết quả được in ra màn hình khi chạy testbench) và waveform file (file có đuôi là .vcd).

- Sau đó, AlPicoSoC Wrapper được phân tích – Elaborate, tổng hợp – Synthesize và hiện thực cấp vật lý – Implementation bằng phần mềm Xilinx Vivado. Các kết quả thu được bao gồm các báo cáo về thời gian, tài nguyên, năng lượng,... và Bitstream phục vụ cho việc triển khai AlPicoSoC Wrapper xuống FPGA tương ứng.

  Tới đây, quá trình phát triển trên phần cứng coi như đã hoàn thành.

- Quy trình phát triển phần mềm:
  - Đầu tiên, từ một tập dữ liệu ban đầu (có thể là MNIST, CIFAR, ImageNet,...) và các mô tả của mô hình Deep Learning (có thể ở dạng mã Python), TensorFlow Lite được sử dụng để tiến hành huấn luyện và lượng tử hóa về dạng 8-bit mô hình Deep Learning.
  - Tiếp theo, mô hình được huấn luyện ở bước trước đó được biến đổi và đóng gói dưới dạng mã ngôn ngữ C dựa trên tập thư viện C chuẩn và tập thư viện C của AlPicoSoC.
  - Sau đó, RISC-V Compiler Toolchain tiến hành biên dịch chương trình C chứa đoạn mã thực hiện tác vụ suy luận thành dạng tệp mã nhị phân RISC-V.

    Tới đây, quá trình phát triển trên phần mềm coi như đã hoàn thành

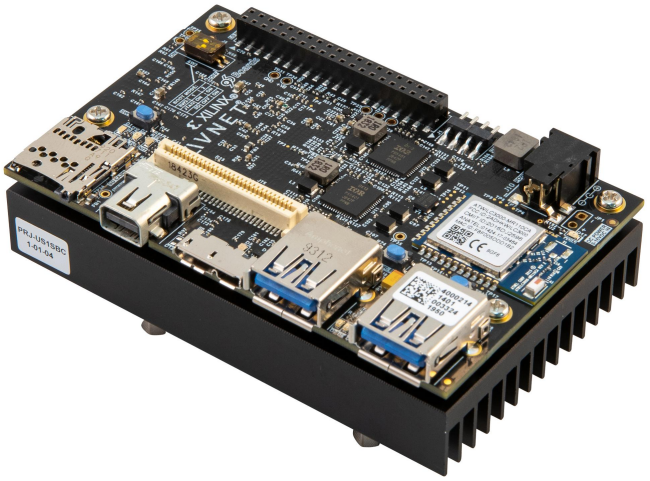
Các kết quả thu được từ quá trình đồng phát triển phần cứng và phần mềm được triển khai trên FPGA. Đầu tiên, Bitstream để ghi vào để cấu hình cho FPGA theo kiến trúc hệ thống của AlPicoSoC. Tiếp theo, tệp mã nhị phân chứa mô hình Deep Learning được ghi vào bộ nhớ lệnh của hệ thống AlPicoSoC. Khi đó, quá trình thiết kế được xem là hoàn tất.

Chương 6

KẾT QUẢ VÀ ĐÁNH GIÁ

6.1 Phần cứng triển khai

Phần cứng được dùng để triển khai đánh giá kiến trúc phần cứng được lựa chọn là FPGA Ultra96-V2.



Hình 6.1: Ngoại hình của Board FPGA Ultra96-V2

Giới thiệu về Ultra96-V2, đây là FPGA Single Board Computer giá thành thấp được sử dụng cho nhiều mục đích ứng dụng. Về tài nguyên, Ultra96-V2 hỗ trợ các tính năng như sau:

- Xilinx Zynq UltraScale+ MPSoC ZU3EG SBVA484
  - Processor System (PS)
    - Quad-core ARM Cortex-A53 MPCore
    - Dual-core ARM Cortex-R5
    - 256KB On-Chip Memory w/ECC
    - ...
  - Programmable Logic (PL)

        * System Logic Cells: 154,350
        * CLB Flip-Flops: 141,120
        * CLB LUTs: 70,560
        * Block RAM: 216 (7.6 Mb)
        * DSP Slices: 360
        * ...
- Bộ nhớ
  - Bộ nhớ LPDDR4 2GB
  - Bộ nhớ MicroSD 16GB
- Microchip Wi-Fi/Bluetooth
- Mini DisplayPort (
- 1x USB 3.0 Type Micro-B, 2x USB 3.0 Type A
- ...

Với lượng tài nguyên dồi dào ở cả phần PS lẫn PL, sự hỗ trợ rộng lớn của cộng đồng người và nhiều yếu tố khác, có thể thấy rằng FPGA Ultra96-V2 là phù hợp để triển khai và đánh giá kiến trúc phần cứng.

6.2 Hiện thực bộ tăng tốc Alpha Accelerator

Bộ tăng tốc Alpha Accelerator được hiện thực với các thông số tính toán N_0, N_1, N_2 và N_3 đều có giá trị là 3. Tất cả tài nguyên về bộ tăng tốc đều được thực hiện theo đúng phần mô tả ở chương thiết kế phần cứng. Ngôn ngữ được dùng để hiện thực là Verilog.

Kết quả hiện thực ở Alpha Accelerator phải đảm bảo yêu cầu về chức năng đã đặt ra ban đầu.

6.2.1 Kết quả mô phỏng

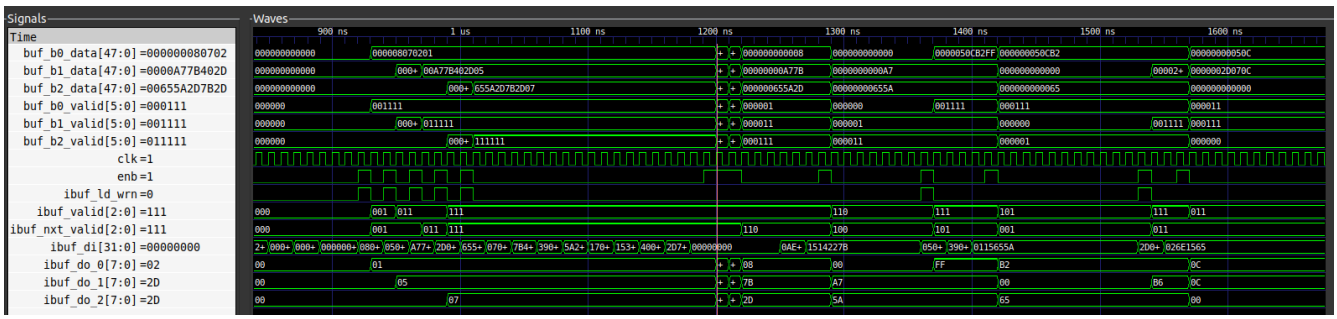
Dưới đây là các kết quả mô phỏng quá trình hoạt động của Alpha Accelerator ở tất cả các khối chức năng và khối điều khiển.

Ngữ cảnh được dùng trong đó là quá trình tính toán ở Convolutional Layer với các thông số:

- Tập giá trị đầu vào Input Feature Map có kích thước $7 \times 7 \times 3$
- Tập giá trị trọng số Kernel có kích thước $3 \times 3 \times 3 \times 6$
- Tập giá trị đầu ra Output Feature Map có kích thước $5 \times 5 \times 6$
- Tập giá trị đối số Bias có kích thước là 6.
- Địa chỉ lưu trữ vùng nhớ Input Feature Map là 1000.
- Địa chỉ lưu trữ vùng nhớ Kernel là 2000.
- Địa chỉ lưu trữ vùng nhớ Output Feature Map là 3000.
- Địa chỉ lưu trữ vùng nhớ Bias là 4000.
- Địa chỉ lưu trữ vùng nhớ Partial-Sum (kích thước là $5 \times 5 \times 6$) là 5000.
- Hàm kích hoạt là ReLU.
- Giá trị hai tham số làm tròn được áp dụng trên tất cả giá trị đầu ra là $M_0 = 1073742347$ và $n = 7$.

Thành phần chức năng:

- *Input Buffer*: có 3 khối được hiện thực trong Alpha Accelerator.



Hình 6.2: Waveform của Input Buffer

Các tín hiệu cần chú ý:

– Dữ liệu đầu vào được lưu trữ trong 3 vùng $buf_b * _data$ tương ứng với số lượng giá trị có thể tính toán cùng lúc của thành phần xử lý. Dữ liệu được tải vào các thanh ghi $buf_b * _data$ và khi dữ liệu có ý nghĩa thì các bit tương ứng với vùng dữ liệu sử dụng ở thanh ghi $buf_b * _valid$ được cài đặt giá trị là 1.

– Tín hiệu dữ liệu đầu vào $ibuf_di$ tương ứng với kích thước 32-bit của RAM và tín hiệu dữ liệu đầu ra $ibuf_do_*$ tương ứng với kích thước tính toán là 8-bit, số lượng bằng với $N_0 = 3$ và có giá trị tương ứng với thành phần đầu tiên trong vùng dữ liệu $buf_b * _data$ tương ứng.

– Tín hiệu phản hồi $ibuf_valid$ và $ibuf_nxt_valid$ phục vụ cho việc xác định có cần thiết để tải thêm dữ liệu vào *Input Buffer* hay không.

– Tín hiệu điều khiển gồm có clk và enb là cặp tín hiệu bắt buộc để điều khiển hoạt động của *Input Buffer*. Đây là tín hiệu bắt buộc chung ở tất cả khối nên không giới thiệu thêm ở các khối khác.

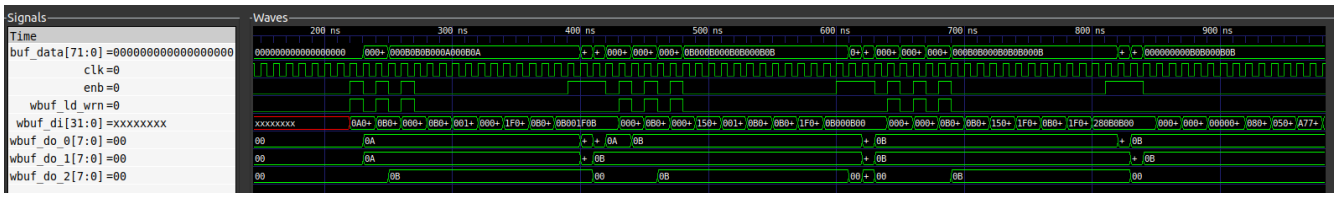
Quá trình hoạt động của *Input Buffer*:

– Giai đoạn từ 900 ns tới 1100 ns, quá trình đọc đầu tiên được thực hiện, dữ liệu tuần tự đi vào 3 vùng lưu trữ.

– Giai đoạn từ 1200 ns tới 1250 ns, quá trình ghi dữ liệu được thực hiện, dữ liệu được chuyển ra ngoài phục vụ cho tính toán.

– Giai đoạn từ 1250 ns đến 1600 ns, chỉ những dữ liệu còn thiếu mới được đọc vào và quá trình ghi cũng diễn ra đều đặn.

- *Weight Buffer*: có 3 khối được hiện thực trong Alpha Accelerator.



Hình 6.3: Waveform của Weight Buffer

Các tín hiệu cần chú ý:

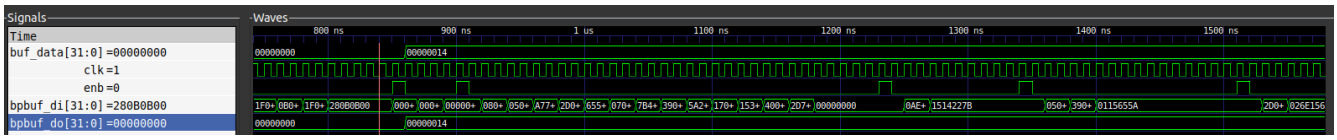
– Dữ liệu đầu vào được lưu trữ trong một vùng buf_data duy nhất về bản chất nhưng về lý thuyết vẫn chia thành 3 vùng (0 -> 2, 3 -> 5 và 6 -> 8) để phục vụ lưu trữ giá trị đối số phục vụ cho tính toán. Mục đích là để dữ liệu có thể được dịch theo chuỗi do dữ liệu trong trọng số thường nằm liền kề nhau.

– Tín hiệu dữ liệu đầu vào $wbuf_di$ tương ứng với kích thước 32-bit của RAM và tín hiệu dữ liệu đầu ra $wbuf_do_*$ tương ứng với kích thước tính toán là 8-bit, số lượng bằng với $N_0 = 3$ và có giá trị tương ứng với thành phần dữ liệu thứ 0, 3 và 6 trong chuỗi dữ liệu buf_data

Quá trình hoạt động của *Weight Buffer*:

- Giai đoạn từ 200 ns tới 300 ns, quá trình đọc đầu tiên được thực hiện, dữ liệu đi vào cho tới khi đầy vùng lưu trữ.
- Giai đoạn từ 380 ns tới 420 ns, quá trình ghi dữ liệu được thực hiện, dữ liệu được chuyển ra ngoài phục vụ cho tính toán. Dữ liệu ở *Weight Buffer* được đọc toàn bộ và không còn sót lại.
- Giai đoạn từ 420 ns tới 900 ns, lặp lại quá trình đọc và ghi dữ liệu như trên.

- *Bias and Partial-Sum Buffer*: có 3 khối được hiện thực trong Alpha Accelerator.



Hình 6.4: Waveform của Bias and Partial-Sum Buffer

Các tín hiệu cần chú ý:

- Dữ liệu đầu vào được lưu trữ trong một vùng buf_data có kích thước đúng bằng 32-bit. Do đó, bộ nhớ đệm này chỉ đệm một giá trị ở một thời điểm mà thôi.
- Tín hiệu dữ liệu đầu vào $bpuf_di$ tương ứng với kích thước 32-bit của RAM và tín hiệu dữ liệu đầu ra $bpuf_do$ tương ứng với kích thước tính toán là 32-bit tương ứng với dữ liệu lưu trong buf_data.

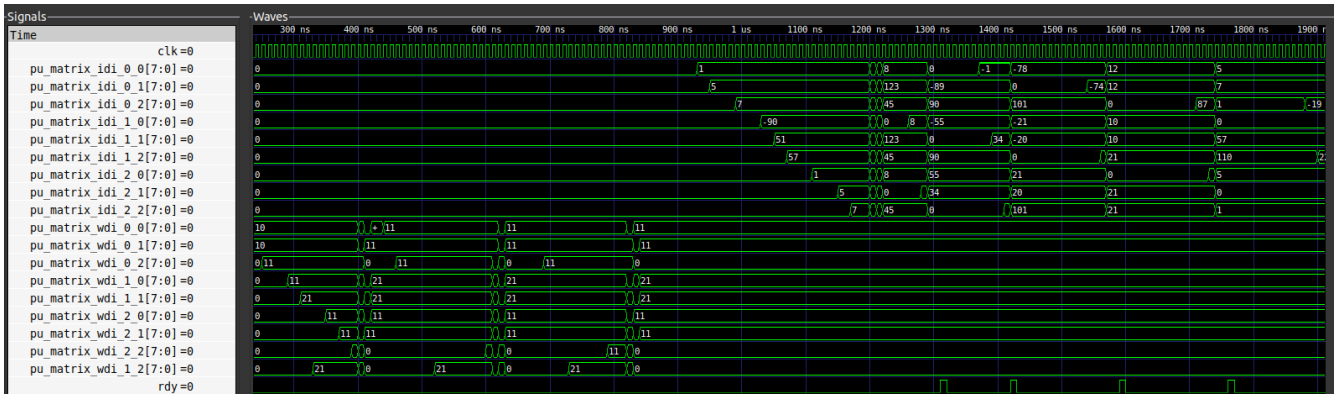
Quá trình hoạt động của *Bias and Partial-Sum Buffer*:

- Giai đoạn từ 850 ns tới 900 ns, quá trình đọc đầu tiên được thực hiện, duy nhất một dữ liệu đi vào.
- Giai đoạn từ 900 ns tới 1500 ns, quá trình ghi dữ liệu được thực hiện, tuy nhiên dữ liệu không hề bị xóa mà vẫn ở lại trong bộ nhớ đệm. Do giá trị này được dùng cho nhiều lần tính toán tiếp theo nên không cần phải xóa, chỉ tới khi cần giá trị mới thì mới thay thế giá trị này.

- *Processing Matrix*: có 1 khối được hiện thực trong Alpha Accelerator.

Các tín hiệu cần chú ý:

- Dữ liệu giá trị đầu vào được đi vào thông qua $pu_matrix_idi_*_*$, giá trị trọng số được đi vào thông qua $pu_matrix_wdi_*_*$ và lưu trữ vào các thanh ghi bên trong *Processing Matrix*. Có tổng cộng 27 giá trị đầu vào và 81 giá trị trọng số tham gia tính toán.
- Dữ liệu giá trị đầu ra được ra ngoài thông qua $pu_matrix_odo_*_*_*$ phục vụ cho quá trình tính toán tiếp theo. Có tổng cộng 27 tổng riêng phân được tính toán.
- Tín hiệu phản hồi rdy báo hiệu rằng chu kỳ tính toán ở *Processing Unit* đã hoàn thành.



Hình 6.5: Waveform thứ nhất của Processing Matrix



Hình 6.6: Waveform thứ hai của Processing Matrix

Quá trình hoạt động của *Processing Matrix*:

- Giai đoạn từ 200 ns tới 900 ns, dữ liệu về trọng số được di chuyển từ bộ nhớ đệm vào các thanh ghi bên trong.
- Giai đoạn từ 900 ns tới 1250 ns, dữ liệu về đầu vào được di chuyển từ bộ nhớ đệm vào các thanh ghi bên trong.
- Giai đoạn từ 1250 ns tới 1310 ns, hoàn thành vòng tính toán đầu tiên và cho ra kết quả tính toán.
- Giai đoạn từ 1310 ns tới 1900 ns, lặp lại quá trình di chuyển dữ liệu giá trị đầu vào và tính toán. Nguyên nhân là do trên Convolution Layer, mô hình tính toán tái sử dụng Kernel và tận dụng tính cục bộ ở Input Feature Map được vận dụng.

- *Accumulate Matrix*: có 1 khối được hiện thực trong Alpha Accelerator.

  Các tín hiệu cần chú ý:

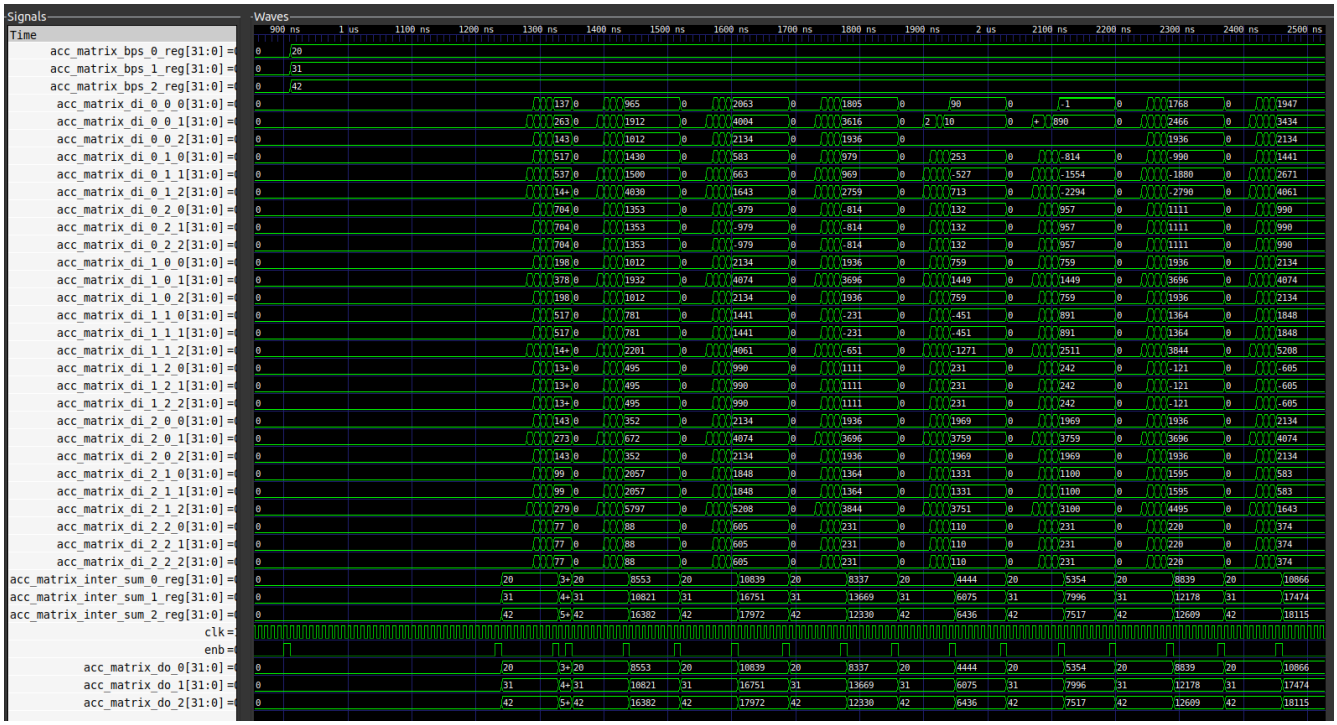
  - Dữ liệu tổng riêng phần đi vào thông qua $acc_matrix_di_*_*_*$ và được cộng tại đây.

Có tổng cộng 27 đầu vào tương ứng với 27 tổng riêng phần kết quả ở Processing Matrix.

– Thanh ghi $acc_matrix_bps_*_reg$ lưu trữ giá trị đối số hoặc tổng riêng phần trước đó.

– Thanh ghi $acc_matrix_inter_sum_bps_*_reg$ lưu trữ kết quả sau khi đã cộng các tổng riêng phần và các giá trị khởi đầu lại.

– Tín hiệu $acc_matrix_do_*$ đưa giá trị tổng lưu trong $acc_matrix_inter_sum_bps_*_reg$ ra ngoài phục vụ cho các công việc tiếp theo.



Hình 6.7: Waveform của Accumulate Matrix

Quá trình hoạt động của *Processing Matrix*:

– Giai đoạn từ 900 ns tới 950 ns, dữ liệu được ghi vào thanh ghi lưu giá trị khởi động (giá trị được ghi là đối số).

– Giai đoạn từ 1220 ns tới 1250 ns, giá trị dữ liệu khởi động (đối số) được ghi vào thanh ghi tổng riêng phần.

– Giai đoạn từ 1280 ns tới 1350 ns, sau khi đã hoàn thành tính toán các tổng riêng phần, các tổng này tiếp tục được cộng vào thanh ghi tổng riêng phần.

– Giai đoạn từ 1350 ns tới 2500 ns, lặp lại quá trình ghi giá trị khởi động và cộng dồn kết quả vào thanh ghi tổng riêng phần.

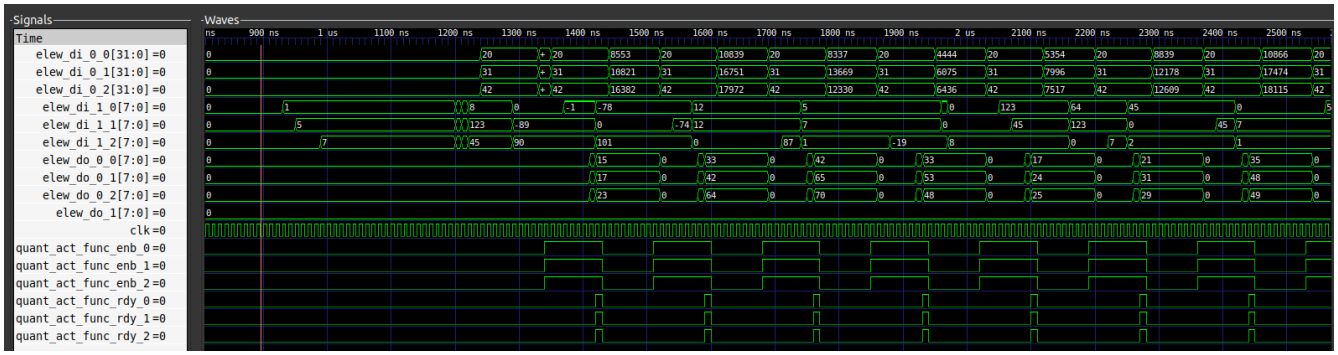
• *Element-Wise Unit*: có 1 khối được hiện thực trong Alpha Accelerator.

Các tín hiệu cần chú ý:

– Dữ liệu tổng từ Accumulate Matrix đi vào thông qua $elew_di_0_*$ và tín hiệu này chỉ sử dụng khi tổng này là tổng toàn phần. Có tổng cộng 3 đầu vào tương ứng với 3 tổng đầu ra

ở Accumulate Matrix.

- Dữ liệu về giá trị đầu vào đi vào ở $elew_di_1_*$. Tuy nhiên, vì đối tượng làm việc là Convolution Layer nên ta không cần quan tâm tín hiệu này.
- Tín hiệu đầu ra $elew_do_0_*$ cho ra kết quả sau khi lượng từ hóa và đi qua các hàm kích hoạt. Tương tự, cũng có 3 đầu ra ở đây.
- Tín hiệu phản hồi $quant_act_func_rdy_*$ báo hiệu quá trình làm lượng tử hóa đã hoàn thành.

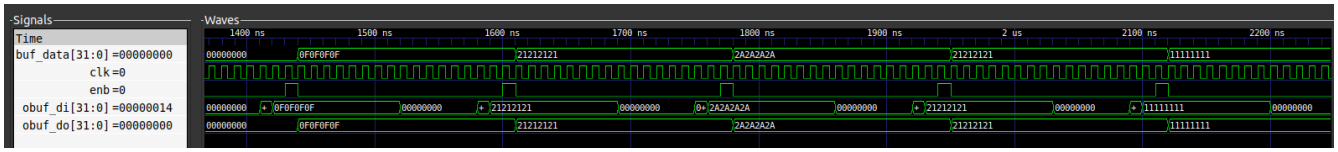


Hình 6.8: Waveform của Elment-Wise Unit

Quá trình hoạt động của *Element-Wise Unit*:

- Giai đoạn từ 1330 ns tới 1450 ns, dữ liệu được đưa vào lượng tử hóa và qua các hàm kích hoạt cho ra hàm cuối cùng.
- Giai đoạn từ 1450 ns tới 2400 ns, quá tình tương tự diễn ra.

- *Ouput Buffer*: có 3 khối được hiện thực trong Alpha Accelerator.



Hình 6.9: Waveform của Output Buffer

Các tín hiệu cần chú ý:

- Dữ liệu đầu vào được lưu trữ trong một vùng buf_data có kích thước đúng bằng 32-bit. Do đó, bộ nhớ đệm này chỉ đệm một giá trị ở một thời điểm mà thôi.
- Tín hiệu dữ liệu đầu vào $obuf_di$ tương ứng với kích thước 32-bit và tín hiệu dữ liệu đầu ra $obuf_do$ tương ứng với kích thước tính toán là 32-bit tương ứng với dữ liệu lưu trong buf_data.

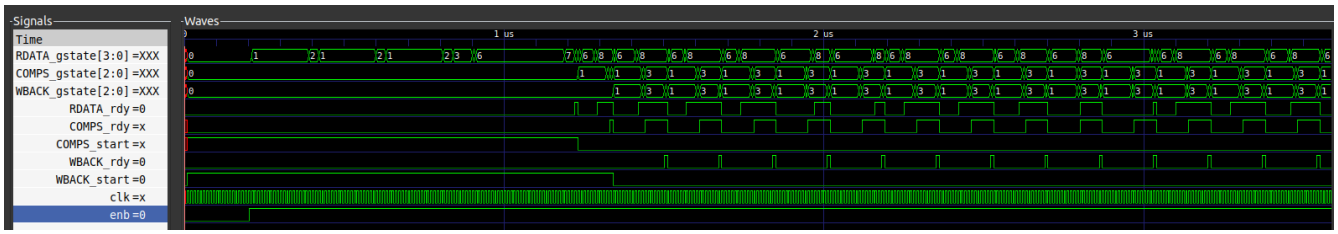
Quá trình hoạt động của *Bias and Partial-Sum Buffer*:

- Giai đoạn từ 1400 ns tới 1500 ns, dữ liệu tính toán được lưu trữ trong bộ nhớ đệm và ghi ra bên ngoài ngay sau đó.

– Giai đoạn từ 1600 ns tới 1700 ns, dữ liệu mới được lưu vào trong bộ nhớ đệm và ghi ra bên ngoài ngay sau đó.

– Giai đoạn từ 1700 ns tới 2200 ns, lặp lại quá trình lưu trữ và ghi dữ liệu này.

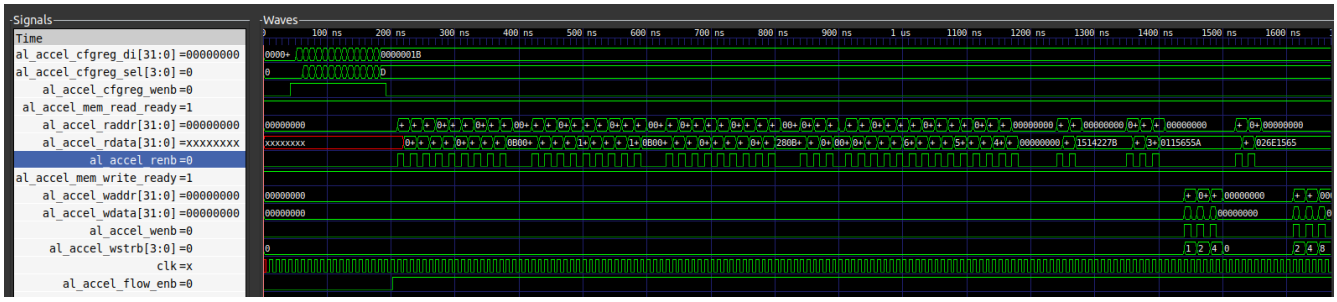
Thành phần điều khiển *Flow Controller*:



Hình 6.10: Waveform của Flow Controller

- Các tín hiệu cần chú ý:
  - Thanh ghi $RDATA_gstate$, $COMPS_gstate$ và $WBACK_gstate$ tương ứng với trạng thái hiện tại của các bộ điều khiển tương ứng $RDATA$, $COMPS$ và $WBACK$ với các giá trị về trạng thái đã được mã hóa.
  - Tín hiệu $RDATA_rdy$, $COMPS_rdy$ và $WBACK_rdy$ là tín hiệu đồng bộ hoạt động giữa 3 khối điều khiển này với nhau. Tín hiệu này kích hoạt khi bộ điều khiển hoàn thành xong chu kỳ tính toán và chờ các bộ điều khiển khác.
  - Tín hiệu $COMPS_start$ và $WBACK_start$ là tín hiệu đánh dấu chờ hoạt động của các trạng thái Pipeline trước đó để bắt đầu.
- Quá trình hoạt động của *Flow Controller*:
  - Giai đoạn từ 200 ns tới 1220 ns, bộ điều khiển $RDATA$ điều khiển thực hiện chu kỳ hoạt động đầu tiên và khi hoàn thành thì đánh dấu bắt đầu hoạt động của bộ điều khiển $COMPS$.
  - Giai đoạn từ 1220 ns tới 1340 ns, bộ điều khiển $RDATA$ điều khiển thực hiện chu kỳ hoạt động thứ hai, bộ điều khiển $COMPS$ điều khiển thực hiện chu kỳ hoạt động đầu tiên và khi hoàn thành thì đánh dấu bắt đầu hoạt động của bộ điều khiển $WBACK$.
  - Giai đoạn từ 1340 ns tới 3000 ns, cả ba bộ điều khiển hoạt động Pipeline với nhau và chờ khi tất cả bộ điều khiển đều hoàn thành chu kỳ hoạt động thì sẽ chuyển sang chu kỳ tiếp theo.

Mô phỏng quá trình hoạt động của Alpha Accelerator:



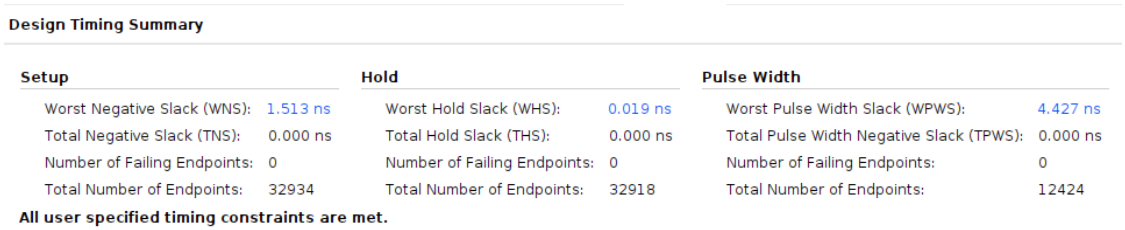
Hình 6.11: Waveform của Alpha Accelerator

- Đặc tả tín hiệu: Alpha Accelerator có hai vùng giao tiếp chủ yếu là vùng cấu hình và vùng tính toán.
  - Vùng cấu hình: Các tín hiệu có dạng $al_accel_cfgreg_*$ tham gia vào tác vụ cấu hình các thanh ghi của Alpha Accelerator.
  - Vùng tính toán: Các tín hiệu có dạng al_accel_r* tham gia vào tác vụ lấy giá trị từ bộ nhớ ngoài. Các tín hiệu có dạng al_accel_w* tham gia vào tác vụ ghi giá trị vào bộ nhớ ngoài. Ngoài ra, còn có các tín hiệu $al_accel_mem_*_ready$ để phản hồi quá trình đọc hoặc ghi ở bộ nhớ ngoài đã hoàn thành.
- Mô tả đơn giản về quá trình hoạt động:
  - Giai đoạn từ 0 ns tới 200 ns, Alpha Accelerator thực hiện cấu hình vào tập thanh ghi.
  - Giai đoạn từ 200 ns tới 1400 ns, Alpha Accelerator lấy dữ liệu phục vụ cho tác vụ tính toán của mình.
  - Giai đoạn từ 1400 ns tới 1800 ns, Alpha Accelerator đồng thời vừa lấy dữ liệu phục vụ cho tác vụ tính toán của mình và ghi các kết quả ra bộ nhớ ngoài.

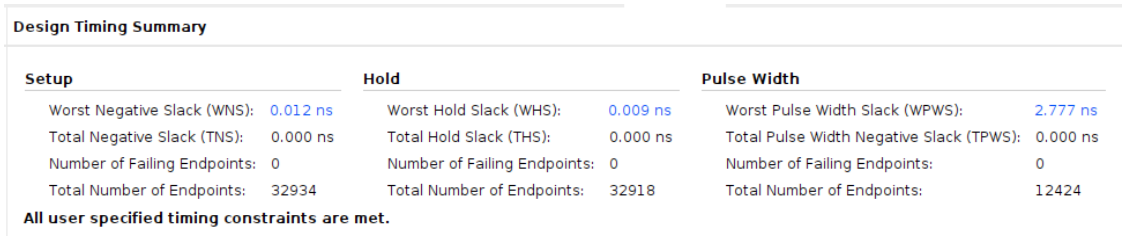
6.2.2 Kết quả hiện thực

Bộ tăng tốc Alpha Accelerator được phân tích – Elaborate, tổng hợp – Synthesize và hiện thực – Implementation bằng công cụ Xilinx Vivado. Kết quả cuối cùng được lấy ở bước sau khi đã hoàn thành quá trình hiện thực ở Vivado.

Tần số tối đa của Alpha Accelerator lên tới 150 MHz. Tần số làm việc thông thường của Alpha Accelerator là 100 MHz. Dưới đây là báo cáo về thời gian của Alpha Accelerator.



Hình 6.12: Kết quả báo cáo về thời gian khi tổng hợp ở tần số 100 MHz



Hình 6.13: Kết quả báo cáo về thời gian khi tổng hợp ở tần số 150 MHz

Đơn vị mà ta cần chú ý ở đây là Setup Slack. Đối với tần số 100 MHz, giá trị Slack này tương đối lớn khoảng 1.513 ns. Còn đối với tần số 150 MHz, giá trị Slack là 0.012 ns gần như bằng 0. Điều này có nghĩa là tương đối khó để tăng thêm tần số hoạt động nữa, nếu không giá trị này sẽ bị âm và vi phạm về Setup.

Ở các phần sau, tất cả kết quả thu được khi hiện thực Alpha Accelerator ở tần số 100 MHz.

Về tài nguyên sử dụng:

- Kết quả tổng quát:

Tài nguyên	Sử dụng	Có sẵn	Tỉ lệ %
LUT	18624	70560	26.39
FF	10653	141120	7.55

Bảng 6.1: Bảng tổng kết tổng quát tài nguyên sử dụng của Alpha Accelerator

Từ bảng tổng hợp, có thể thấy rằng lượng tài nguyên cần cho Alpha Accelerator là tương đối ít so với khả năng cung cấp của FPGA khi chỉ tốn chưa tới 28% Look-Up Table và chưa tới 8% Flip-Flops.

- Kết quả cụ thể:

Tên	CLB LUTs		CLB Registers		CLB	
al_accel	18624	100%	10653	100%	3504	100%
wbuf_2	12	0.06	72	0.68	28	0.80
wbuf_1	12	0.06	72	0.68	26	0.74
wbuf_0	12	0.06	72	0.68	25	0.71
pu_matrix	9112	48.93	6912	64.88	1549	44.21
obuf_2	0	0	32	0.30	9	0.26
obuf_1	0	0	32	0.30	10	0.29
obuf_0	0	0	32	0.30	9	0.26
ibuf_2	84	0.45	162	1.52	74	2.11
ibuf_1	84	0.45	162	1.52	81	2.31
ibuf_0	79	0.42	162	1.52	70	2.00
flow_ctrl	5398	28.98	508	4.77	1364	38.93
elw_unit	923	4.96	321	3.01	244	6.96
config_regs	3021	16.22	1826	17.14	999	28.51
bpbuf_2	0	0	32	0.30	17	0.49
bpbuf_1	0	0	32	0.30	18	0.51
bpbuf_0	0	0	32	0.30	14	0.40
acc_matrix	21	0.11	192	1.80	74	2.11

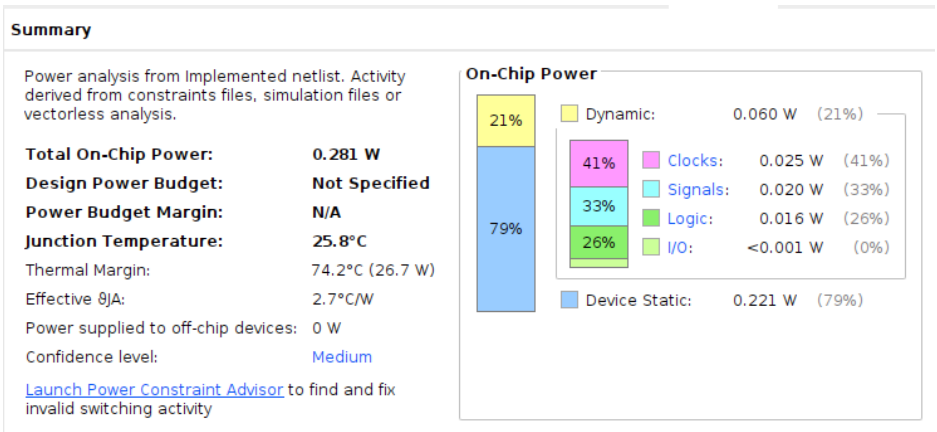
Bảng 6.2: Bảng tổng kết chi tiết tài nguyên sử dụng của Alpha Accelerator

Theo đó, khối tiêu hao nhiều tài nguyên nhất chính là *Processing Matrix* khi chiếm tới 48.93% số lượng CLB LUTs và 64.88% số lượng CLB Registers của Alpha Accelerator. Thành phần sử dụng lượng tài nguyên lớn tiếp theo là *Flow Controller* khi chiếm tới 28.98% số lượng CLB LUTs, 4.77% số lượng CLB Registers. Thứ ba là *Configuration Registers* với 16.22 số lượng CLB LUTS và 17.14 số lượng CLB Registers.

Các thành phần còn lại chiếm lượng tài nguyên tương đối ít, nhất là các loại $Buffer$ khi chiếm tài nguyên gần như không đáng kể.

Về năng lượng tiêu thụ:

- Kết quả tổng quát:



Hình 6.14: Kết quả báo cáo về năng lượng tổng quát được thực hiện bởi Vivado

Kết quả về năng lượng này chỉ mang tính tham khảo vì đây là hiện thực trên FPGA. Theo đó, năng lượng tĩnh của FPGA đã lên tới 0.221 W còn năng lượng tiêu thụ động chỉ có 0.060 W được phân bố như trên hình.

- Kết quả cụ thể:

Tên	Tổng (W)	Clocks (W)	Signal (W)	Data (W)	Logic (W)	BRAM (W)
al_accel	0.055	0.021	0.019	0.019	0.015	<0.001
flow_ctrl	0.025	0.001	0.014	0.014	0.01	<0.001
pu_matrix	0.022	0.014	0.004	0.004	0.004	<0.001
config_regs	0.004	0.004	<0.001	<0.001	<0.001	<0.001
elw_unit	0.002	0.001	0.001	0.001	0.001	<0.001
acc_matrix	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
ibuf_0	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
ibuf_1	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
ibuf_2	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
wbuf_0	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
wbuf_1	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
wbuf_2	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
obuf_0	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
obuf_1	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
obuf_2	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
bpbuf_0	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
bpbuf_1	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
bpbuf_2	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001

Bảng 6.3: Kết quả cụ thể về năng lượng tiêu thụ của Alpha Accelerator

Theo đó, ở tần số 100 MHz, bộ tăng tốc chỉ tiêu hao 0.055 W. Thành phần tiêu hao năng lượng nhiều nhất vẫn là khối *Flow Controller* khi chiếm tới 0.025 W, sau đó tới khối *Processing Matrix* khi chiếm 0.022 W.

6.2.3 Ràng buộc sử dụng

Có vài ràng buộc để sử dụng Alpha Accelerator:

- Thứ nhất, phải chuẩn bị thêm một vùng nhớ để lưu trữ dữ liệu cho Partial-Sum. Vùng nhớ này tùy theo loại Layer mà có kích thước khác nhau:
  - Đối với Convolution Layer, vùng nhớ tạm có kiểu dữ liệu là $int32_t$ và có kích thước bằng $ifm_W \times ifm_H \times 3$.
  - Đối với Fully-Connected Layer, vùng nhớ tạm có kiểu dữ liệu là $int32_t$ và có kích thước bằng kích thước của vùng nhớ giá trị đầu ra.
  - Đối với Pooling Layer, không cần vùng nhớ tạm.
- Thứ hai, tất cả đối tượng dữ liệu về giá trị trọng số, giá trị đầu vào, giá trị đầu ra, giá trị tổng riêng phần hay giá trị đối số đều được phải lưu liên tục trên bộ nhớ.

- Thứ ba, với cấu hình hiện tại của Alpha Accelerator (N_0, N_1, N_2 và N_3 đều bằng 3), dữ liệu cần phải được điều chỉnh để sao cho chiều kích thước liên quan tới các đối tượng này đề phải chia hết cho 3, nếu không phải thêm các giá trị 0 để thỏa mãn. Ví dụ, việc tính toán yêu cầu Kernel phải có kích thước theo chiều thứ nhất và chiều thứ hai là 3×3.

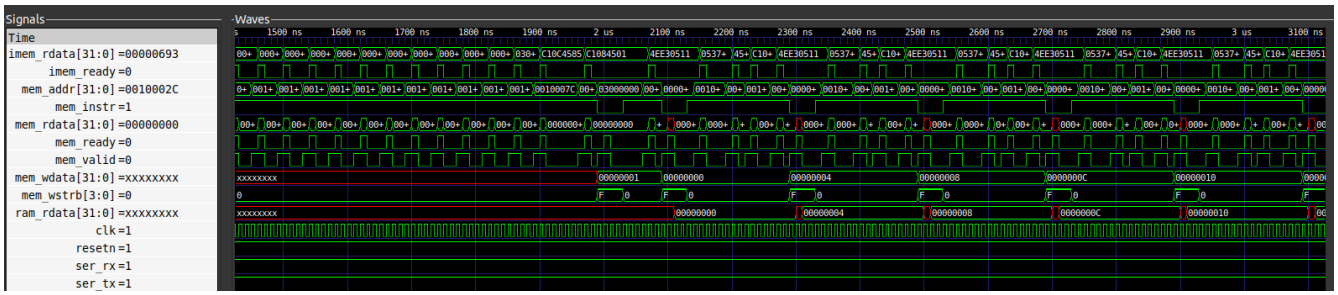
6.3 Hiện thực hệ thống PicoSoC và AlPicoSoC

6.3.1 Hiện thực và tinh chỉnh PicoSoC

PicoSoC là một SoC đơn giản chỉ bao gồm CPU là PicoRV32, RAM để lưu dữ liệu chạy chương trình, ngoại vi chỉ có UART và bộ nhớ lệnh được đọc trực tiếp từ SPI Flash. Ở đây, PicoSoC được tinh chỉnh để có thể cải thiện về tốc độ cũng như đơn giản về mặt tính toán. Theo đó, PicoSoC được tinh chỉnh lại với việc sử dụng Block RAM để chạy bộ nhớ lệnh, nhờ đó tăng tốc độ đọc lệnh của SoC.

6.3.1.1 Kết quả mô phỏng

Dưới đây là kết quả mô phỏng của PicoSoC được tinh chỉnh.



Hình 6.15: Waveform của PicoSoC

Về cơ bản, kết quả mô phỏng rất phức tạp khi rất nhiều tín hiệu đan xen nhau. Tuy nhiên, có thể hiểu đơn giản quá trình này là CPU đọc lệnh, sau đó lệnh được thực hiện rồi tiếp tục đọc lệnh và thực hiện. Mục đích của Waveform là để chứng minh rằng PicoSoC có khả năng thực hiện các lệnh trong chương trình và không bị lỗi dẫn đến dùng quá trình hoạt động của mình.

6.3.1.2 Kết quả hiện thực

PicoSoC được hiện thực cấp phần cứng với bộ nhớ lệnh có kích thước là 256 KB và bộ nhớ dữ liệu có kích thước là 128 KB. Ngoài ra, ở PicoSoC có cho phép sử dụng thêm khối $picorv32_pcpi_fast_mul$ để tăng tốc quá trình tính toán phép nhân. Tần số hoạt động của PicoSoC được để ở mức thông thường là 100 MHz.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	2.953 ns	Worst Hold Slack (WHS):	0.025 ns	Worst Pulse Width Slack (WPWS):	4.427 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	9894	Total Number of Endpoints:	9878	Total Number of Endpoints:	3108

All user specified timing constraints are met.

Hình 6.16: Kết quả báo cáo về thời gian khi tổng hợp PicoSoC ở tần số 100 MHz

Theo đó, thông số Setup Slack là 2.953 ns còn tương đối lớn nên có thể tăng thêm tần số hoạt động cho PicoSoC.

Về tài nguyên sử dụng:

- Kết quả tổng quát:

Tài nguyên	Sử dụng	Có sẵn	Tỉ lệ %
LUT	2485	70560	3.52
LUTRAM	40	28800	0.14
FF	1115	141120	0.79
BRAM	90	216	41.67
DSP	4	360	1.11

Bảng 6.4: Bảng tổng kết tổng quát tài nguyên sử dụng của PicoSoC

Từ bảng tổng hợp, có thể thấy rằng lượng tài nguyên cần cho PicoSoC là rất ít so với khả năng cung cấp của FPGA khi chỉ tốn chưa tới 4% Look-Up Table và chưa tới 1% Flip-Flops. Nguyên nhân là vì PicoSoC có cấu trúc cực kỳ đơn giản chỉ bao gồm một CPU PicoRV32, một bộ nhớ dữ lệnh, một bộ nhớ lệnh và một thiết bị ngoại vi là UART.

- Kết quả cụ thể:

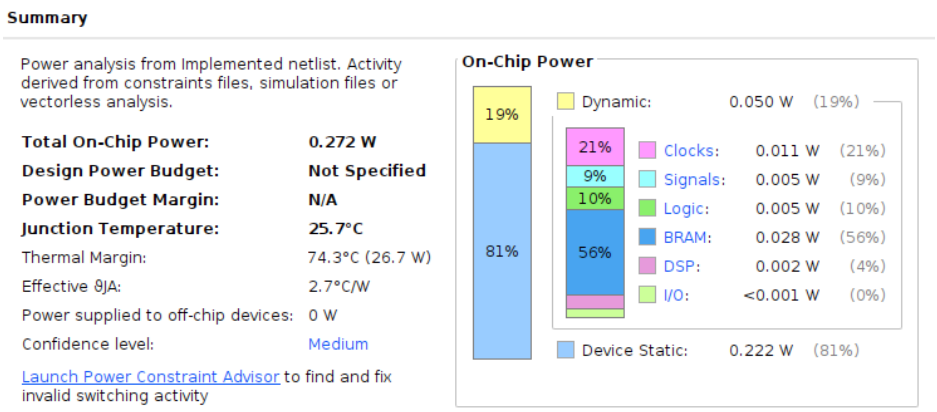
Tên	CLB LUTs		CLB Registers		CLB	
picosoc	2485	100%	1115	100%	503	100%
cpu	2244	90.30	980	87.89	436	86.68
dmem	18	0.72	0	0	9	1.79
imem	15	0.60	2	0.18	13	2.58
simpleuart	115	4.62	131	11.75	45	8.95

Bảng 6.5: Bảng tổng kết chi tiết tài nguyên sử dụng của PicoSoC

Toàn bộ tài nguyên của PicoSoC gần như chỉ thuộc về khối CPU PicoRV32 khi chiếm tới 90.30% số lượng CLB LUTs, 87.89% số lượng CLB Registers và 86.68% CLB. Nhắc lại một lần nữa về nguyên nhân là do thiếu hụt số lượng thiết bị ngoại vi ở PicoSoC. Các thành phần khác gần như chiếm tài nguyên lượng tài nguyên không đáng kể.

Về năng lượng tiêu thụ:

- Kết quả tổng quát:



Hình 6.17: Kết quả báo cáo về năng lượng tổng quát của PicoSoC được thực hiện bởi Vivado

Tổng năng lượng tiêu thụ trên chip là 0.272 W. Trong đó, năng lượng tĩnh của FPGA đã lên tới

0.222 W còn năng lượng tiêu thụ động chỉ có 0.050 W được phân bố như trên hình.

- Kết quả cụ thể:

Tên	Tổng (W)	Clocks (W)	Signal (W)	Data (W)	Logic (W)	BRAM (W)	DSP (W)
picosoc	0.044	0.005	0.004	0.004	0.004	0.028	0.002
imem	0.019	<0.001	<0.001	<0.001	<0.001	0.018	<0.001
cpu	0.014	0.004	0.004	0.004	0.004	<0.001	0.002
dmem	0.010	<0.001	<0.001	<0.001	<0.001	0.010	<0.001
simpleuart	0.001	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001

Bảng 6.6: Kết quả cụ thể về năng lượng tiêu thụ của PicoSoC

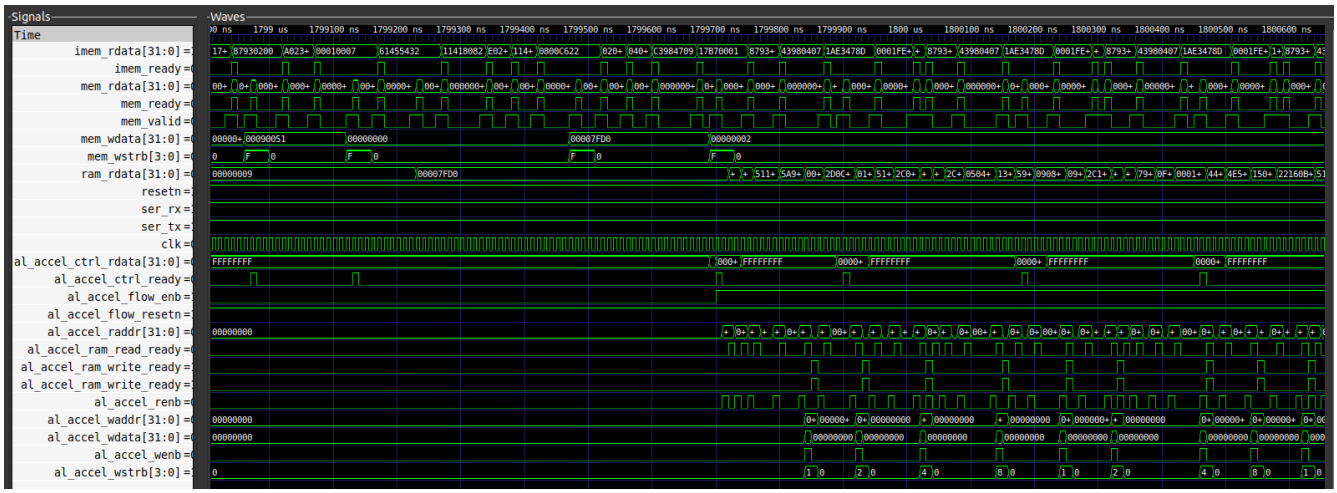
Theo đó, ở tần số 100 MHz, PicoSoC chỉ tiêu hao năng lượng là 0.044 W. Thành phần tiêu hao năng lượng nhiều nhất là bộ nhớ lệnh khi chiếm tới 0.019 W.

6.3.2 Hiện thực AlPicoSoC

AlPicoSoC là sự kết hợp giữa PicoSoC đã tinh chỉnh ở phần trước đó và bộ tăng tốc Alpha Accelerator. Do đó, chắc chắn AlPicoSoC sử dụng tài nguyên nhiều hơn so với PicoSoC.

6.3.2.1 Kết quả mô phỏng

Dưới đây là kết quả mô phỏng của AlPicoSoC.

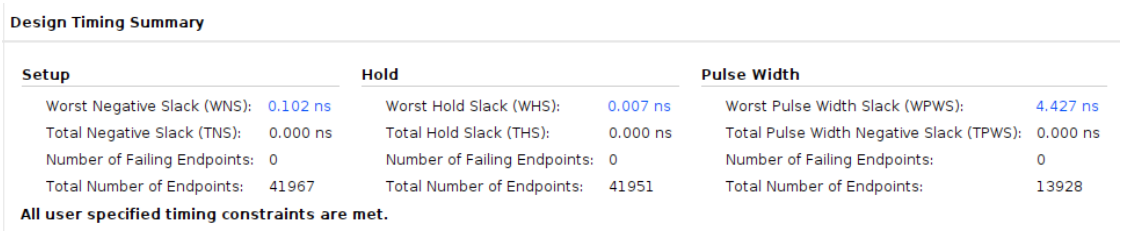


Hình 6.18: Waveform của AlPicoSoC

Về cơ bản, kết quả mô phỏng rất phức tạp khi rất nhiều tín hiệu đan xen nhau. Tuy nhiên, có thể hiểu đơn giản quá trình này là CPU đọc lệnh, sau đó lệnh được thực hiện rồi tiếp tục đọc lệnh và thực hiện. Thêm nữa, CPU còn có thể điều khiển hành vi của bộ tăng tốc dựa trên tập thanh ghi và vùng địa chỉ tương ứng. Mục đích của Waveform là để chứng minh rằng AlPicoSoC có khả năng thực hiện các lệnh trong chương trình cũng như có thể điều khiển hoạt động của bộ tăng tốc mà không gây lỗi cho hệ thống.

6.3.2.2 Kết quả hiện thực

AlPicoSoC được hiện thực cấp phần cứng tương tự như PicoSoC với bộ nhớ lệnh có kích thước là 256 KB và bộ nhớ dữ liệu có kích thước là 128 KB. Tần số hoạt động của AlPicoSoC được để ở mức thông thường là 100 MHz.



Hình 6.19: Kết quả báo cáo về thời gian khi tổng hợp AlPicoSoC ở tần số 100 MHz

Thông số Setup Slack là 0.102 ns – giá trị tương đối nhỏ. Do đó, có thể tăng thêm tương đối tần số làm việc của hệ thống AlPicoSoC.

Về tài nguyên sử dụng:

- Kết quả tổng quát:

Tài nguyên	Sử dụng	Có sẵn	Tỉ lệ %
LUT	21555	70560	30.55
LUTRAM	40	28800	0.14
FF	11940	141120	8.46
BRAM	90	216	41.67

Bảng 6.7: Bảng tổng kết tổng quát tài nguyên sử dụng của AlPicoSoC

Từ bảng tổng hợp, có thể thấy rằng lượng tài nguyên cần cho AlPicoSoC là tương đối lớn khi cần tới 30.55% Look-Up Table và 8.46% Flip-Flops.

- Kết quả cụ thể:

Tên	CLB LUTs		CLB Registers		CLB	
picosoc	21555	100%	11940	100%	4153	100%
accel_ctrl	4	0.02	4	0.03	4	0.10
al_accel	18990	88.10	10643	89.14	3683	88.68
cpu	2357	10.93	1150	9.63	509	12.26
dmem	17	0.08	0	0	8	0.19
imem	96	0.45	2	0.02	77	1.85
simpleuart	87	0.40	131	1.10	38	0.92

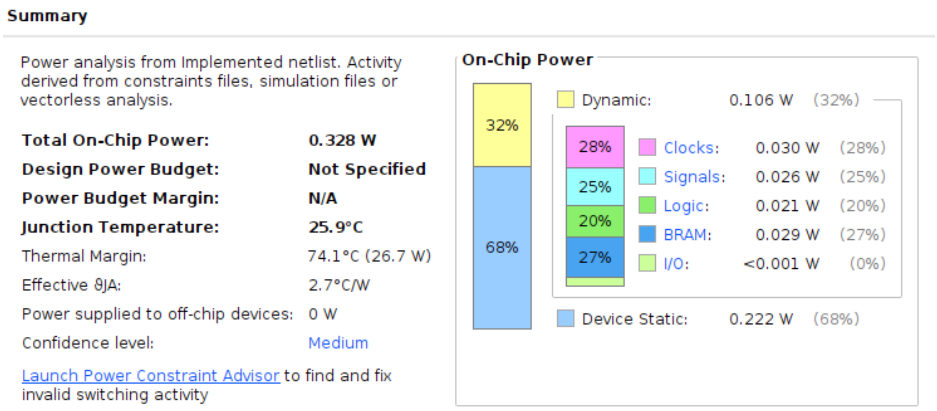
Bảng 6.8: Bảng tổng kết chi tiết tài nguyên sử dụng của AlPicoSoC

Theo đó, bộ tăng tốc giữ phần lớn tài nguyên của AlPicoSoC khi chiếm tới 88.10% số lượng CLB LUTs, 89.14% số lượng CLB Registers và 88.68% CLB. Tiếp theo, thành phần chiếm tài nguyên lớn thứ hai là CPU tuy nhiên ít hơn rất nhiều so với Alpha Accelerator khi chỉ chiếm

10.93% số lượng CLB LUTs, 9.63% số lượng CLB Registers và 12.26% CLB. Các thành phần còn lại chiếm tài nguyên gần như không đáng kể khi so sánh với hai thành phần trên.

Về năng lượng tiêu thụ:

- Kết quả tổng quát:



Hình 6.20: Kết quả báo cáo về năng lượng tổng quát của AlPicoSoC được thực hiện bởi Vivado

Tổng năng lượng tiêu thụ trên chip là 0.328 W. Trong đó, năng lượng tĩnh của FPGA đã lên tới 0.222 W còn năng lượng tiêu thụ động chỉ có 0.106 W được phân bố như trên hình.

- Kết quả cụ thể:

Tên	Tổng (W)	Clocks (W)	Signal (W)	Data (W)	Logic (W)	BRAM (W)
alpicosoc	0.101	0.027	0.026	0.025	0.021	0.029
al_accel	0.062	0.023	0.022	0.022	0.017	<0.001
cpu	0.010	0.003	0.004	0.004	0.004	<0.001
imem	0.019	<0.001	<0.001	<0.001	<0.001	0.019
dmem	0.010	<0.001	<0.001	<0.001	<0.001	0.010
simpleuart	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001
accel_ctrl	<0.001	<0.001	<0.001	<0.001	<0.001	<0.001

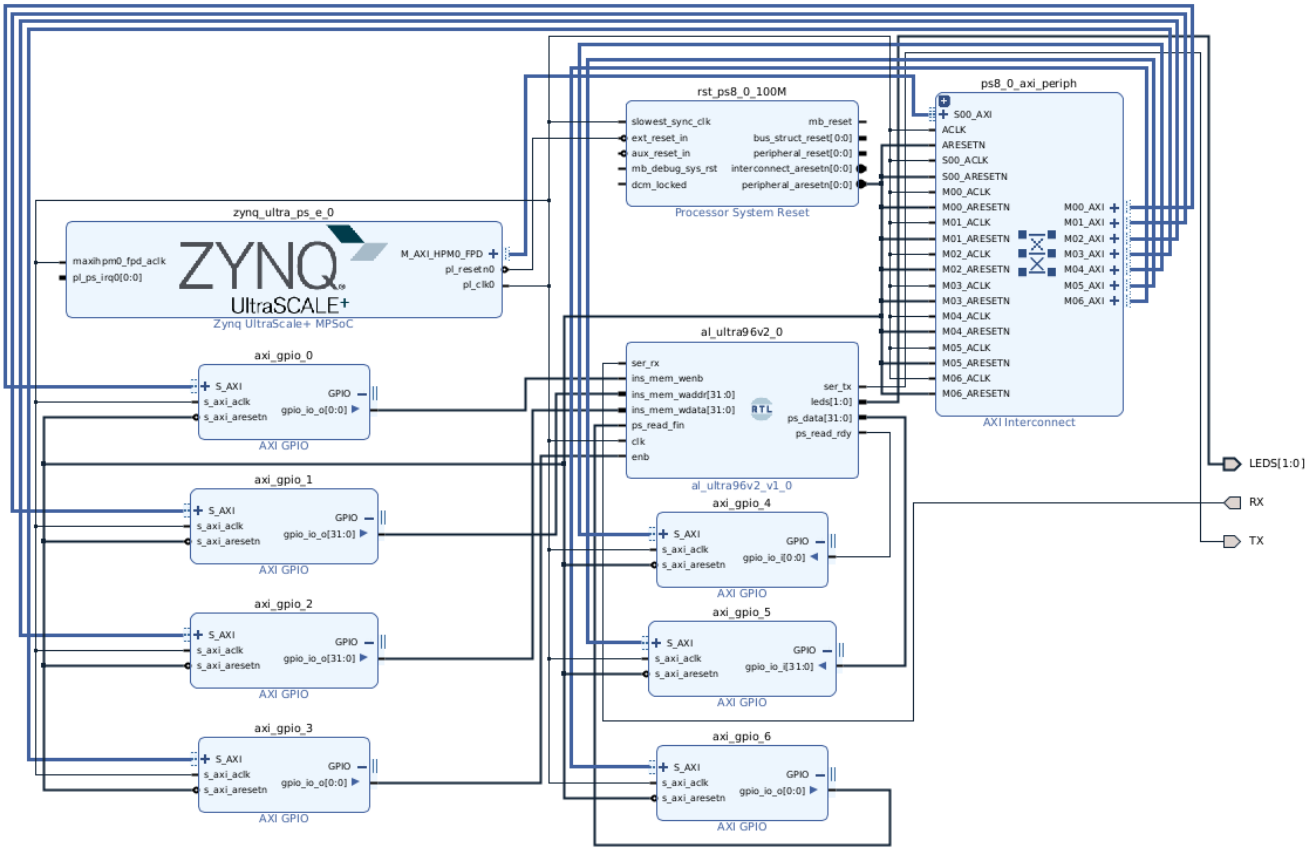
Bảng 6.9: Kết quả cụ thể về năng lượng tiêu thụ của AlPicoSoC

Ở tần số 100 MHz, AlPicoSoC tiêu hao năng lượng là 0.066 W. Trong đó, thành phần tiêu hao năng lượng nhiều nhất là Alpha Accelerator khi chiếm tới 0.047 W.

6.4 Triển khai và đánh giá trên Ultra96-V2 FPGA

Vấn đề lớn nhất đối với hiện thực ở FPGA là dữ liệu để chạy chương trình của PicoSoC và AlPicoSoC được lưu trữ ở đâu và lập trình như thế nào. Vì vậy, có thể tận dụng thành phần Processor System – PS của FPGA để lưu trữ và lập trình cho PicoSoC và AlPicoSoC. Thêm nữa, có thể dùng PS để điều chỉnh hoạt động của các SoC này thông qua các tín hiệu điều khiển quan trọng.

Ta xây dựng sơ đồ kiến trúc của mô hình kiểm thử như sau.



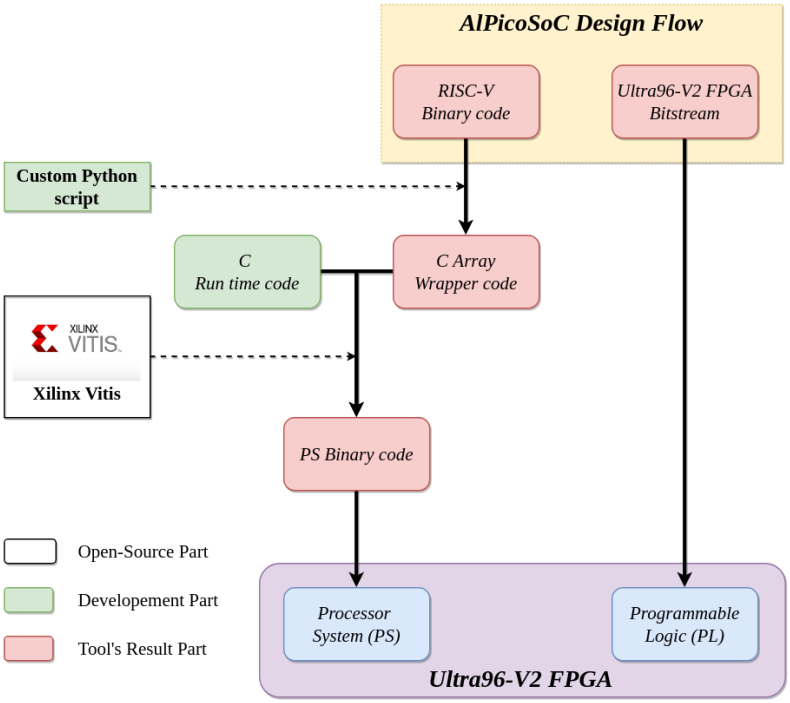
Hình 6.21: Mô hình kiểm thử trên Ultra96-V2 FPGA

PicoSoC và AlPicoSoC đều sử dụng chung mô hình như trên. Trong đó, Zynq UltraScale+ MPSoC chịu trách nhiệm thực hiện những vai trò chính kết hợp với các khối hỗ trợ tương ứng:

- Giai đoạn đầu, Zynq UltraScale+ MPSoC kết hợp với các AXI_GPIO 0, 1 và 2 tiến hành ghi mã nhị phân của chương trình RISC-V sau khi đã được biên dịch vào trong bộ nhớ lệnh của PicoSoC hoặc AlPicoSoC.
- Giai đoạn hai, Zynq UltraScale+ MPSoC sử dụng AXI_GPIO 3 để bật tín hiệu cho phép PicoSoC/AlPicoSoC bắt đầu hoạt động.
- Giai đoạn tiếp theo, Zynq UltraScale+ MPSoC cùng với AXI_GPIO 4, 5 và 6 thực hiện nhiệm vụ chờ dữ liệu được ghi vào AXI_GPIO 5 rồi thông qua giao thức UART để in giá trị đó ra màn hình.

Ngoài những hoạt động trên, Zynq UltraScale+ MPSoC không can thiệp vào bất kỳ hoạt động nào của PicoSoC/AlPicoSoC.

Kết hợp thêm quá trình này, ta có được quy trình thiết kế phần cứng và phần mềm như sau.



Hình 6.22: Quy trình triển khai trên Ultra96-V2 FPGA

Sau khi đã phát triển theo quy trình thiết kế AlPicoSoC và thu được kết quả là Bitstream của Ultra96-V2 và mã nhị phân chương trình RISC-V, quy trình tiếp theo được triển khai như sau:

- Về Bitstream của Ultra96-V2, kết quả này được đưa tới phần Programmable Logic (PL) của FPGA để tiến hành lập trình tạo ra phần cứng tương ứng.
- Về mã nhị phân chương trình RISC-V, kết quả này thông một đoạn mã đơn giản bằng Python để chuyển về dạng mã của mảng C tương ứng. Sau đó, đoạn mã này kết hợp với đoạn mã thực hiện chức năng ghi bộ nhớ, đánh dâu bắt đầu và đọc dữ liệu (được trình bày ở phần trước) để tạo ra mã nhị phân cho thành Processor System (PS) của FPGA để tiến hành lập trình cho Zynq UltraScale+ MPSoC.

Sau khi hoàn thành, bằng việc khởi động chương trình lưu trữ ở Zynq UltraScale+ MPSoC, chương trình RISC-V sẽ được chạy trên hệ thống PicoSoC/AlPicoSoC và độc lập với Zynq UltraScale+ MPSoC.

6.4.1 Kết quả kiểm thử tốc độ tính toán cấp Layer

6.4.1.1 Convolution Layer

A. Mô tả

Tập test case được dùng để đánh giá có số lượng là 7 và tuân theo bảng sau.

Tên	Input Feature Map	Kernel	Stride	Output Feature Map	Bias
Test Case 0	$7 \times 7 \times 3$	$3 \times 3 \times 3 \times 6$	1×1	$5 \times 5 \times 6$	6
Test Case 1	$7 \times 7 \times 6$	$3 \times 3 \times 6 \times 6$	1×1	$5 \times 5 \times 6$	6
Test Case 2	$7 \times 7 \times 9$	$3 \times 3 \times 9 \times 9$	1×1	$5 \times 5 \times 9$	9
Test Case 3	$7 \times 7 \times 3$	$3 \times 3 \times 3 \times 6$	2×2	$3 \times 3 \times 6$	6
Test Case 4	$7 \times 7 \times 6$	$3 \times 3 \times 6 \times 6$	2×2	$3 \times 3 \times 6$	6
Test Case 5	$7 \times 7 \times 9$	$3 \times 3 \times 9 \times 9$	2×2	$3 \times 3 \times 9$	9
Test Case 6	$28 \times 28 \times 3$	$3 \times 3 \times 3 \times 3$	1×1	$26 \times 26 \times 3$	3

Bảng 6.10: Bảng mô tả về test case ở Convolution Layer

B. Hệ thống PicoSoC

Kết quả triển khai các test case của Convolution Layer của hệ thống PicoSoC trên Ultra96-V2 như sau.

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	*Dec*	*Hex*	*Dec*
Test Case 0	0x0022267d	2238077	0x00058b91	363409
Test Case 1	0x003fdab7	4184759	0x000a4bed	674797
Test Case 2	0x008c5273	9196147	0x0016922f	1479215
Test Case 3	0x000c4d0f	806159	0x0001ff57	130903
Test Case 4	0x0016feb5	1506997	0x0003b547	243015
Test Case 5	0x003287e4	3311588	0x0008212f	532783
Test Case 6	0x03dfd16e	64999790	0x0095e0af	9822383

Bảng 6.11: Bảng kết quả về tốc độ hoàn thành test case của PicoSoC ở Convolution Layer

C. Hệ thống AlPicoSoC

Kết quả triển khai các test case của Convolution Layer của hệ thống AlPicoSoC trên Ultra96-V2 như sau.

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	*Dec*	*Hex*	*Dec*
Test Case 0	0x00000d98	3480	0x00000267	615
Test Case 1	0x000010ea	4330	0x00000311	785
Test Case 2	0x00001c1a	7194	0x0000053d	1341
Test Case 3	0x00000b8b	2955	0x000001fe	510
Test Case 4	0x00000db1	3505	0x0000026c	620
Test Case 5	0x000014f9	5369	0x000003d0	976
Test Case 6	0x000068ce	26830	0x000014a5	5285

Bảng 6.12: Bảng kết quả về tốc độ hoàn thành test case của AlPicoSoC ở Convolution Layer

6.4.1.2 Fully-Connected Layer

A. Mô tả

Tập test case được dùng để đánh giá có số lượng là 7 và tuân theo bảng sau.

Tên	Input Activation	Weight	Output Activation	Bias
Test Case 0	18	18×9	9	9
Test Case 1	27	27×9	9	9
Test Case 2	27	27×12	12	12
Test Case 3	54	54×9	9	9
Test Case 4	54	54×12	12	12
Test Case 5	18	18×27	27	27
Test Case 6	54	54×27	27	27

Bảng 6.13: Bảng mô tả về test case ở Fully-Connected Layer

B. Hệ thống PicoSoC

Kết quả triển khai các test case của Fully-Connected Layer của hệ thống PicoSoC trên Ultra96-V2 như sau.

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	*Dec*	*Hex*	*Dec*
Test Case 0	0x0000b359	45913	0x00001d97	7575
Test Case 1	0x0000edd9	60889	0x000026be	9918
Test Case 2	0x0001a0d2	106706	0x00003075	12405
Test Case 3	0x00019d77	105847	0x00004247	16967
Test Case 4	0x000226ca	141002	0x0000584a	22602
Test Case 5	0x000216f4	136948	0x0000583f	22591
Test Case 6	0x0004d543	316739	0x0000c64d	50765

Bảng 6.14: Bảng kết quả về tốc độ hoàn thành test case của PicoSoC ở Fully-Connected Layer

C. Hệ thống AlPicoSoC

Kết quả triển khai các test case của Fully-Connedted Layer của hệ thống AlPicoSoC trên Ultra96-V2 như sau.

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	*Dec*	*Hex*	*Dec*
Test Case 0	0x00000535	1333	0x000000f0	240
Test Case 1	0x00000599	1433	0x00000104	260
Test Case 2	0x000005e4	1508	0x00000113	275
Test Case 3	0x000006ac	1708	0x0000013b	315
Test Case 4	0x0000075b	1883	0x0000015e	350
Test Case 5	0x00000693	1683	0x00000136	310
Test Case 6	0x00000ac6	2758	0x0000020d	525

Bảng 6.15: Bảng kết quả về tốc độ hoàn thành test case của AlPicoSoC ở Fully-Connected Layer

6.4.1.3 Pooling Layer

A. Mô tả

Tập test case được dùng để đánh giá có số lượng là 7 và tuân theo bảng sau.

Tên	Input Feature Map	Patch	Stride	Output Feature Map
Test Case 0	$9 \times 9 \times 3$	3×3	3×3	$3 \times 3 \times 3$
Test Case 1	$7 \times 7 \times 3$	3×3	2×2	$3 \times 3 \times 3$
Test Case 2	$8 \times 8 \times 2$	2×2	2×2	$4 \times 4 \times 2$
Test Case 3	$9 \times 18 \times 3$	3×3	3×3	$3 \times 6 \times 3$
Test Case 4	$9 \times 17 \times 3$	3×3	2×2	$4 \times 8 \times 3$
Test Case 5	$8 \times 16 \times 2$	2×2	2×2	$4 \times 8 \times 2$
Test Case 6	$28 \times 28 \times 1$	2×2	2×2	$14 \times 14 \times 3$

Bảng 6.16: Bảng mô tả về test case ở Pooling Layer

B. Hệ thống PicoSoC

Kết quả triển khai các test case của Pooling Layer của hệ thống PicoSoC trên Ultra96-V2 như sau.

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	*Dec*	*Hex*	*Dec*
Test Case 0	0x00013aba	80570	0x00002fd5	12245
Test Case 1	0x00013aca	80586	0x00002fe5	12261
Test Case 2	0x0000cf04	52996	0x00001f2f	7983
Test Case 3	0x000273a0	160672	0x00005f62	24418
Test Case 4	0x0004596b	285035	0x0000a932	43314
Test Case 5	0x00019c33	105523	0x00003e0e	15886
Test Case 6	0x0005018a	328074	0x0000bff5	49141

Bảng 6.17: Bảng kết quả về tốc độ hoàn thành test case của PicoSoC ở Pooling Layer

C. Hệ thống AlPicoSoC

Kết quả triển khai các test case của Pooling Layer của hệ thống AlPicoSoC trên Ultra96-V2 như sau.

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	*Dec*	*Hex*	*Dec*
Test Case 0	0x000005c1	1473	0x00000106	262
Test Case 1	0x0000058f	1423	0x000000fc	252
Test Case 2	0x0000055d	1373	0x000000f2	242
Test Case 3	0x000006d4	1748	0x0000013d	317
Test Case 4	0x0000079c	1948	0x00000165	357
Test Case 5	0x0000060c	1548	0x00000115	277
Test Case 6	0x000008e1	2273	0x000001a6	422

Bảng 6.18: Bảng kết quả về tốc độ hoàn thành test case của AlPicoSoC ở Pooling Layer

6.4.2 Kết quả kiểm thử tốc độ tính toán cấp Model

6.4.2.1 Mô tả Model

Mô hình được dùng để kiểm thử là mô hình phân loại chữ viết tay dựa trên tập dữ liệu MNIST. Mô hình có dạng là một Convolution Neural Network đơn giản bao gồm 6 Layer cơ bản (2 Convolution, 2

Max Pooling và 2 Fully-Connected Layer) và một vài Layer dùng để chuyển đổi khác được mô tả như bảng bên dưới.

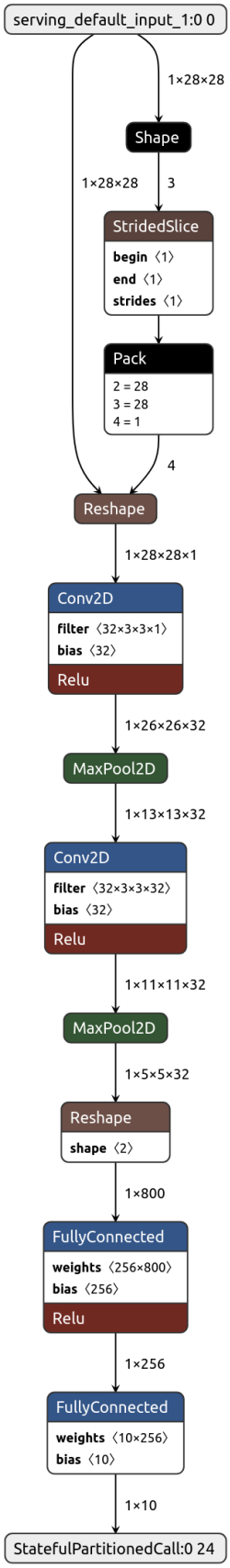
Tên	Loại Layer	Kích thước đầu ra	Số lượng tham số
reshape	Reshape	$28 \times 28 \times 1$	0
conv2d_0	Conv2D	$26 \times 26 \times 32$	320
max_pooling2d_0	MaxPooling2D	$13 \times 13 \times 32$	0
conv2d_1	Conv2D	$11 \times 11 \times 32$	9248
max_pooling2d_1	MaxPooling2D	$5 \times 5 \times 32$	0
flatten	Flatten	800	0
dense_0	Dense	256	205056
dense_1	Dense	10	2570

Bảng 6.19: Bảng tóm tắt thông tin của Model kiểm thử

Tổng số tham số của mô hình là 217194. Đây là mô hình tương đối nhỏ và phù hợp để kiểm thử cho hệ thống.

Thông tin cụ thể của từng Layer như sau:

- *reshape*: Reshape Layer dùng để thay đổi kích thước đầu vào cho phù hợp với mục đích sử dụng của người dùng. Layer này không chứa thông tin gì đặc biệt.
- *conv2d_0*: Convolution Layer thứ nhất có số lượng Kernel là 32, kích thước Kernel là 3×3, Stride là 1×1 và hàm kích hoạt là ReLU.
- *max_pooling2d_0*: Pooling Layer thứ nhất có kích thước Patch là 2×2 và Stride cũng là 2×2
- *conv2d_1*: Convolution Layer thứ hai có số lượng Kernel là 32, kích thước Kernel là 3×3, Stride là 1×1 và hàm kích hoạt là ReLU.
- *max_pooling2d_1*: Pooling Layer thứ hai có kích thước Patch là 2×2 và Stride cũng là 2×2
- *flatten*: Flatten Layer dùng để trải phẳng dữ liệu đầu ra của Layer trước đó để phục vụ như đầu vào cho Layer tiếp theo. Layer này không chứa thông tin gì đặc biệt.
- *dense_0*: Fully-Connected Layer thứ nhất có kích thước đầu ra là 256 và hàm kích hoạt là ReLU.
- *dense_1*: Fully-Connected Layer thứ hai có kích thước đầu ra là 10 tương đương với 10 chữ số trong tập MNIST.



Hình 6.23: Biểu diễn Model kiểm thử dạng sơ đồ

6.4.2.2 Độ chính xác của Model

Model sau khi được huấn luyện bằng TensorFlow Lite, được lượng tử hóa theo về dạng 8-bit số nguyên. Độ chính xác của mô hình như sau:

- 32-bit Floating-Point Model: **99.1200%**
- 8-bit Integer Model: **99.1000%**

Độ chính xác của mô hình được đánh giá trên tập kiểm thử MNIST bao gồm 10000 dữ liệu. Theo đó, mô hình được lượng tử hóa chỉ bị giảm 0.02% độ chính xác khi so với mô hình gốc.

6.4.2.3 Kiểm thử

A. Hệ thống PicoSoC

Kết quả triển khai Model lên hệ thống PicoSoC trên Ultra96-V2 như sau.

- Độ chính xác của mô hình khi kiểm thử trên 10000 dữ liệu từ tập kiểm thử MNIST: **98.5800%**
- Tốc độ thực hiện tác vụ suy luận trên 10 dữ liệu từ tập kiểm thử MNIST:

Tác vụ	**Chu kỳ**		**Lệnh thực thi**	
	Số lượng	*Tỉ lệ (%)*	*Số lượng*	*Tỉ lệ (%)*
Đọc dữ liệu	2,409,948	0.03	217,390	0.02
Tính toán	7,277,166,368	99.88	1,171,320,094	99.98
In kết quả	6,337,480	0.09	22,360	0.00
Tổng cộng:	7,285,913,796	100	1,171,559,844	100

Bảng 6.20: Bảng tổng kết về tốc độ hoàn thành 10 lần suy luận của PicoSoC đối với mô hình MNIST

B. Hệ thống AlPicoSoC

Kết quả triển khai Model lên hệ thống AlPicoSoC trên Ultra96-V2 như sau.

- Độ chính xác của mô hình khi kiểm thử trên 10000 dữ liệu từ tập kiểm thử MNIST: **98.5800%**
- Tốc độ thực hiện tác vụ suy luận trên 10 dữ liệu từ tập kiểm thử MNIST:

Tác vụ	**Chu kỳ**		**Lệnh thực thi**	
	Số lượng	*Tỉ lệ (%)*	*Số lượng*	*Tỉ lệ (%)*
Đọc dữ liệu	2,426,795	15.79	217,390	15.84
Tính toán	6,601,950	42.96	1,132,830	82.53
In kết quả	6,337,489	41.25	22,360	1.63
Tổng cộng:	15,366,234	100	1,372,580	100

Bảng 6.21: Bảng tổng kết về tốc độ hoàn thành 10 lần suy luận của AlPicoSoC đối với mô hình MNIST

C. Processor System của Ultra96V-2

Ngoài ra, khi kiểm thử cấp Model, tôi còn thực hiện thêm mô hình trên Processor System của Ultra96V-2 mà cụ thể là lõi ARM Cortex-A53 MPCore.

Kết quả triển khai Model lên Processor System của Ultra96-V2 như sau.

- Độ chính xác của mô hình khi kiểm thử trên 10000 dữ liệu từ tập kiểm thử MNIST: **98.5800%**
- Tốc độ thực hiện tác vụ suy luận trên 10 dữ liệu từ tập kiểm thử MNIST:

Tác vụ	**Chu kỳ của X_TIME**	
	Số lượng	*Tỉ lệ (%)*
Đọc dữ liệu	56,830	0.02
Tính toán	267,934,142	99.96
In kết quả	66,476	0.02
Tổng cộng:	268,057,448	100

Bảng 6.22: Bảng tổng kết về tốc độ hoàn thành 10 lần suy luận của Ultra96-V2 Processor System đối với mô hình MNIST

6.5 Đánh giá

Để đánh giá được ảnh hưởng của bộ tăng tốc Alpha Accelerator đối với toàn bộ hệ thống, ta tiến hành so sánh giữa PicoSoC (SoC gốc) và AlPicoSoC (SoC được tích hợp thêm bộ tăng tốc) trên các yếu tố như sau:

- Về tài nguyên,

Tài nguyên	PicoSoC	AlPicoSoC	Hệ số
LUT	2485	21555	8.67
LUTRAM	40	40	1
FF	1115	11940	10.71
BRAM	90	90	1

Bảng 6.23: Bảng so sánh tài nguyên giữa PicoSoC và AlPicoSoC

trong đó, hệ số được tính bằng cách lấy tài nguyên của AlPicoSoC chia cho PicoSoC.

AlPicoSoC sử dụng lượng tài nguyên nhiều hơn PicoSoC với 8.67 lần tài nguyên LUT và 10.71 lần tài nguyên Flip-Flop. Còn tài nguyên về LUTRAM và BRAM được sử dụng thì không đổi ở cả hai hệ thống vì tài nguyên này chỉ dùng ở PicoRV32. Nguyên nhân của việc sử dụng tài nguyên gấp nhiều lần như vậy là vì PicoSoC còn quá đơn giản khi chỉ bao gồm CPU tính toán PicoRV32, ngoại vi UART, bộ nhớ lệnh và bộ nhớ dữ liệu với lượng tài nguyên khiêm tốn khi chỉ có 2485 LUT và 1115 Flip-Flop.

- Về năng lượng,

Năng lượng	PicoSoC	AlPicoSoC	Hệ số
Tổng (W)	0.044	0.101	2.30
Clocks (W)	0.005	0.027	5.40
Signal (W)	0.004	0.026	6.50
Data (W)	0.004	0.025	6.25
Logic (W)	0.004	0.021	5.25
BRAM (W)	0.028	0.0029	1.04

Bảng 6.24: Bảng so sánh năng lượng giữa PicoSoC và AlPicoSoC

trong đó, hệ số được tính bằng cách lấy năng lượng ở AlPicoSoC chia cho PicoSoC.

AlPicoSoC có năng lượng tiêu thụ nhiều hơn PicoSoC với 2.30 lần tổng năng lượng tiêu thụ. Trong đó, năng lượng tiêu thụ ở Clocks của AlPicoSoC gấp 5.40 lần, ở Signal gấp 6.50, ở Data gấp 6.25 lần và ở Logic gấp 5.25 lần so với PicoSoC. Chỉ có năng lượng ở BRAM là tương tự nhau cho cả hai hệ thống. Nguyên nhân cũng tương tự như ở phần trước, là vì PicoSoC còn quá đơn giản.

- Về tốc độ tính toán ở cấp Layer,
  - Ở Convolution Layer,

Kiểm thử	**Chu kỳ**			**Lệnh thực thi**		
	PicoSoC	*AlPicoSoC*	*Hệ số*	*PicoSoC*	*AlPicoSoC*	*Hệ số*
Test case 0	2238077	3480	643.13	363409	615	590.91
Test case 1	4184759	4330	966.46	674797	785	859.61
Test case 2	9196147	7194	1278.31	1479215	1341	1103.07
Test case 3	806159	2955	272.81	130903	510	256.67
Test case 4	1506997	3505	429.96	243015	620	391.96
Test case 5	3311588	5369	616.80	532783	976	545.88
Test case 6	64999790	26830	2422.65	9822383	5285	1858.54

Bảng 6.25: Bảng so sánh tốc độ tính toán giữa PicoSoC và AlPicoSoC ở Convolution Layer

trong đó, hệ số được tính bằng cách lấy số chu kỳ/lệnh thực thi ở PicoSoC chia cho AlPicoSoC

Theo đó, hệ thống được tích hợp thêm Alpha Accelerator tăng tốc tốc độ tính toán ở Convolution Layer lên đến $272 \sim 2422$ lần (tính theo chu kỳ) và $256 \sim 1858$ lần (tính theo lệnh thực thi) so với hệ thống không có tích hợp. Nguyên nhân là vì tác vụ tính toán này được giao cho bộ tăng tốc thực hiện và bộ tăng tốc tận dụng những đặc tính sau:

* Về tác vụ bộ nhớ: Cải thiện việc đọc dữ liệu khi cùng một thời điểm có thể lấy tối đa là 4 dữ liệu 8-bit.
* Về tác vụ tính toán:
  - Tận dụng việc tái sử dụng Kernel và tính cục bộ của Input Feature Map
  - Sử dụng mô hình tính toán bit-serial nên chỉ cần 8 clock cycles để hoàn thành tính toán và còn thêm khả năng song song hóa tính toán.
* Sử dụng phương pháp Pipeline gồm 3 giai đoạn gồm đọc dữ liệu – RDATA, tính toán – COMPS và ghi dữ liệu – WBACK.

– Ở Fully-Connected Layer,

Kiểm thử	**Chu kỳ**			**Lệnh thực thi**		
	PicoSoC	*AlPicoSoC*	*Hệ số*	*PicoSoC*	*AlPicoSoC*	*Hệ số*
Test case 0	45913	1333	34.44	7575	240	31.56
Test case 1	60889	1433	42.49	9918	260	38.15
Test case 2	106706	1508	70.76	12405	275	45.11
Test case 3	105847	1708	61.97	16967	315	53.86
Test case 4	141002	1883	74.88	22602	350	64.58
Test case 5	136948	1683	81.37	22591	310	72.87
Test case 6	316739	2758	114.84	50765	525	96.70

Bảng 6.26: Bảng so sánh tốc độ tính toán giữa PicoSoC và AlPicoSoC ở Fully-Connected Layer

trong đó, hệ số được tính bằng cách lấy số chu kỳ/lệnh thực thi ở PicoSoC chia cho AlPicoSoC.

Theo đó, hệ thống AlPicoSoC tăng tốc tốc độ tính toán ở Fully-Connected Layer khoảng $34 \sim 114$ lần (tính theo chu kỳ) và $31 \sim 96$ lần (tính theo lệnh thực thi) so với hệ thống nền tảng PicoSoC. Tương tự, nguyên nhân là vì tác vụ tính toán này được giao cho bộ tăng tốc thực hiện và bộ tăng tốc tận dụng những đặc tính sau ở Fully-Connected:

* Về tác vụ bộ nhớ: Cải thiện việc đọc dữ liệu khi cùng một thời điểm có thể lấy tối đa là 4 dữ liệu 8-bit.
* Về tác vụ tính toán:
  - Tận dụng việc tái sử dụng Input Activaion
  - Sử dụng mô hình tính toán bit-serial nên chỉ cần 8 clock cycles để hoàn thành tính toán và còn thêm khả năng song song hóa tính toán.
* Sử dụng phương pháp Pipeline gồm 3 giai đoạn gồm đọc dữ liệu – RDATA, tính toán – COMPS và ghi dữ liệu – WBACK.

– Ở Pooling Layer,

Kiểm thử	**Chu kỳ**			**Lệnh thực thi**		
	PicoSoC	*AlPicoSoC*	*Hệ số*	*PicoSoC*	*AlPicoSoC*	*Hệ số*
Test case 0	80570	1473	54.70	12245	262	46.74
Test case 1	80586	1423	56.63	12261	252	48.65
Test case 2	52996	1373	38.60	7983	242	32.99
Test case 3	160672	1748	91.92	24418	317	77.03
Test case 4	285035	1948	146.32	43314	357	121.33
Test case 5	105523	1548	68.17	15886	277	57.35
Test case 6	328074	2273	144.34	49141	422	116.45

Bảng 6.27: Bảng so sánh tốc độ tính toán giữa PicoSoC và AlPicoSoC ở Pooling Layer

trong đó, hệ số được tính bằng cách lấy số chu kỳ/lệnh thực thi ở PicoSoC chia cho AlPicoSoC.

Hệ thống AlPicoSoC tăng tốc tốc độ tính toán ở Pooling Layer khoảng $38 \sim 146$ lần (tính theo chu kỳ) và $32 \sim 121$ lần (tính theo lệnh thực thi) so với hệ thống nền tảng PicoSoC. Tương tự, nguyên nhân là vì tác vụ tính toán này được giao cho bộ tăng tốc thực hiện và bộ tăng tốc tận dụng những đặc tính sau ở Fully-Connected:

* Về tác vụ bộ nhớ: Cải thiện việc đọc dữ liệu khi cùng một thời điểm có thể lấy tối đa là 4 dữ liệu 8-bit.
* Về tác vụ tính toán: Tận dụng việc tái sử dụng Input Feature Map ở một số trường hợp đặc thù.
* Sử dụng phương pháp Pipeline gồm 2 giai đoạn gồm đọc dữ liệu – RDATA và ghi dữ liệu – WBACK.

- Về tốc độ tính toán ở cấp Model,

Kiểm thử	**Chu kỳ**			
	Processor System	*PicoSoC*	*AlPicoSoC*	*Hệ số tỷ lệ*
Đọc dữ liệu	56,830	2,409,948	2,426,795	1 : 42.41 : 42.70
Tính toán	267,934,142	7,277,166,368	6,601,950	40.58 : 1102.28 : 1
In kết quả	66,476	6,337,480	6,337,489	1 : 95.33 : 95.33
Tổng cộng:	268,057,448	7,285,913,796	15,366,234	17.44 : 474.15 : 1

Bảng 6.28: Bảng so sánh tốc độ tính toán mô hình CNN MNIST ở Ultra96-V2 Processor System, PicoSoC và AlPicoSoC

Hệ thống AlPicoSoC tăng tốc tốc độ một cách rõ rệt ở tác vụ tính toán của mô hình CNN MNIST, khi mà nó cải thiện tới 1102.28 lần so với hệ thống nền tảng PicoSoC và 40.58 lần so với Ultra96-V2 Processor System. Từ đây, có thể thấy, việc tích hợp thêm Alpha Accelerator mang lại hiệu quả tăng tốc một cách đáng ngạc nhiên cho hệ thống AlPicoSoC.

Tuy nhiên, với tác vụ đọc dữ liệu và in kết quả, tốc độ của AlPicoSoC và PicoSoC là như nhau và cao hơn lần lượt là khoảng 42 và 96 lần so với Ultra96-V2 Processor System. Nguyên nhân của việc này là do sự khác nhau trong việc xây dựng mô hình đánh giá. Ở AlPicoSoC và PicoSoC, dữ liệu đọc dữ liệu và địa chỉ ghi kết quả là bộ nhớ ngoài, mà bộ nhớ ngoài này chính là từ Ultra96-V2 Processor System. Ở Ultra96-V2 Processor System, tất cả việc đọc dữ liệu và in kết quả đều được thực hiện trên bộ nhớ của mình.

Tổng kết lại, hệ thống AlPicoSoC tích hợp bộ tăng tốc Alpha Accelerator đánh đổi bằng tài nguyên và năng lượng của bộ tăng tốc để gia tăng tốc độ tính toán ở cấp Layer của mô hình Deep Learning. Cụ thể là bằng việc sử dụng gấp hơn 8 lần tài nguyên (LUT và FF) và tiêu hao gấp hơn 2 lần năng lượng, Alpha Accelerator mang lại khả năng tính toán vượt trội cho PicoSoC khi mà tốc độ tính toán tăng gấp $272 \sim 2422$ lần ở Convolution Layer, $34 \sim 114$ lần ở Fully-Connected layer và $38 \sim 146$ lần ở Pooling Layer. Sở dĩ việc tăng tốc có hệ số rất lớn ở Convolution Layer là vì ở đây tác vụ tính toán mới là tác vụ trọng yếu, mà Alpha Accelerator lại hỗ trợ cực kỳ mạnh mẽ cho quá trình tính toán của Deep Learning. Còn ở các Layer khác, tác vụ bộ nhớ mới là tác vụ trọng yếu, Alpha Accelerator chỉ hỗ trợ được một phần khi cố gắng sử dụng gấp 4 lần lượng dữ liệu (1 lần đọc RAM cho ra 32-bit data mà dữ liệu tính toán có kích thước là 8-bit). Thêm nữa, Alpha Accelerator mang lại khả năng tăng tốc đáng kinh ngạc khi mà tốc độ tính toán tăng gấp 474 lần đối với nền tảng gốc và 17 lần đối với cả Ultra96-V2 Processor System.

Có thể thấy rằng sự đánh đổi về tài nguyên và năng lượng tương đối xứng đáng khi mang tới tốc độ tính toán vượt trội cho hệ thống ở tác vụ tính toán đặc thù của Deep Learning. Tuy nhiên, khi một mô hình Deep Learning chạy trên thiết bị biên, không chỉ có mỗi tác vụ tính toán trong quá trình suy luận mà còn nhiều tác vụ khác thực hiện. Và có thể, một tác vụ nào khác tác vụ tính toán mới là nút thắc cổ chai trong quá trình thực thi. Do đó, khi đánh giá tổng quát trên toàn hệ thống, kết quả tăng tốc sẽ không được vượt trội như vậy.

Chương 7

KẾT LUẬN

7.1 Tổng kết

Trong dự án này, tôi đã hiện thực hệ thống tích hợp AI trên thiết bị biên dựa trên RISC-V cho các ứng dụng IoT bao gồm 2 thành phần chính: phần cứng là hệ thống AlPicoSoC dựa trên RISC-V và phần mềm bao gồm các chương trình C phục vụ huấn luyện và suy luận mô hình Deep Learning trên phần cứng tương ứng.

Cụ thể,

- Thứ nhất, đề xuất hệ thống phần cứng AlPicoSoC dựa trên mô hình PicoSoC đã tinh chỉnh về bộ nhớ lệnh và được tích hợp thêm bộ tăng tốc Alpha Accelerator. Bộ tăng tốc Alpha Accelerator được thiết kế để tăng tốc quá trình tính toán cho mô hình Deep Learning dựa trên việc tái sử dụng dữ liệu, sử dụng mô hình bit-serial cho phép song song hóa tính toán, sử dụng hiệu quả dữ liệu được đọc từ bộ nhớ và sử dụng cơ chế Pipeline gồm 3 giai đoạn.
- Thứ hai, hiện thực và sử dụng tập phần mềm cho phần cứng AlPicoSoC. Tập phần mềm này bao gồm 4 loại: phần mềm phục vụ huấn luyện mô hình ở dạng ngôn ngữ Python và sử dụng TensorFlow Lite, phần mềm phục vụ suy luận mô hình ở dạng ngôn ngữ C, phần mềm đặc tả đặc thù cho SoC ở dạng ngôn ngữ C và tập phần mềm mã nguồn chương trình biên dịch RISC-V [24].
- Thứ ba, triển khai hệ thống phần cứng là AlPicoSoC và tập phần mềm đi kèm lên trên Ultra96-V2 FPGA. Hiện tại, hệ thống kết hợp này đã được kiểm thử với việc đánh đổi về tài nguyên và năng lượng để mang lại độ tăng tốc vượt trội ở tác vụ suy luận trên một Layer.

Đặc tính quan trọng của AlPicoSoC được nhắm vào:

- *Tính đơn giản và chi phí thấp*. Hệ thống AlPicoSoC có cấu trúc đơn giản, dễ hiện thực do đó chi phí cần cho hệ thống tương đối thấp.
- *Khả năng tính toán*. Bộ tăng tốc Alpha Accelerator được thiết kế với mục đích cải thiện tốc độ tính toán ở mô hình Deep Learning.

- *Khả năng mở rộng*. Hệ thống AlPicoSoC có khả năng mở rộng thêm các thành phần tính toán khác và thiết bị ngoại vi một cách dễ dàng.

7.2 Tiềm năng phát triển

Hệ thống AlPicoSoC đặc biệt là bộ tăng tốc Alpha Accelerator có rất nhiều tiềm năng để phát triển trong tương lai. Dưới dây là một vài ý tưởng về việc tiếp tục phát triển hệ thống sử dụng AlPicoSoC làm nền tảng.

7.2.1 Mở rộng về thiết bị

Hiện tại, AlPicoSoC rất nghèo nàn về ngoại vi khi chỉ có duy nhất một thiết bị ngoại vi là UART. Tuy nhiên, với tính chất đơn giản và dễ hiểu của mình, việc tích hợp thêm các thiết bị ngoại vi vào hệ thống phù hợp với yêu cầu của ứng dụng hướng tới trở nên cực kỳ đơn giản và dễ dàng.

Không chỉ dừng lại ở ngoại vi, tích hợp thêm các thiết bị quan trọng khác lên hệ thống cũng cực kỳ dễ dàng. Một trong những thiết bị ưu tiên hàng đầu chính là DMA khi nó cho phép việc trực tiếp sử dụng bộ nhớ của bộ tăng tốc Alpha Accelerator góp phần hỗ trợ tăng tốc độ tính toán của mình.

7.2.2 Đa lõi – đa nhân

Với xu hướng phát triển hiện tại, việc sử dụng tích hợp đa lõi – đa nhân xử lý trở nên ngày càng phổ biến và phát triển. Hệ thống AlPicoSoC có thể thực hiện việc này một cách không quá khó khăn khi mà lõi PicoRV32 có cấu trúc tương đối dễ hiểu.

7.2.3 Huấn luyện mô hình

Tiến trình triển khai trí tuệ nhân tạo trên thiết bị biên bao gồm tác vụ huấn luyện mô hình được thực hiện ở Server/Cloud và tác vụ suy luận mô hình được thực hiện ở thiết bị biên. Tuy nhiên, vấn đề của triển khai trên là mô hình không thể tinh chỉnh và thay đổi theo sự biến đổi của môi trường. Do đó, dần dần tác vụ huấn luyện mô hình đang được dời ra thiết bị biên.

Theo sự phát triển đó, bộ tăng tốc Alpha Accelerator có thể được phát triển để đảm nhận việc tính toán trong tác vụ huấn luyện mô hình trên thiết bị biên.

7.2.4 Chuyển đổi nền tảng

Nếu nền tảng PicoSoC sử dụng lõi PicoRV32 không phù hợp với mục đích sử dụng, người dùng có thể chuyển đổi nền tảng khác bằng cách tích hợp bộ tăng tốc vào nền tảng đó. Việc này cực kỳ dễ dàng bởi vì bộ tăng tốc cung cấp giao diện giao tiếp đơn giản và dễ hiểu.

Bibliography

[1] HireOtter, *The future of iot: Emerging trends and applications*. [Online]. Available: https://www.linkedin.com/pulse/future-iot-emerging-trends-applications-hireotter#:~:text=According%5C%20to%5C%20a%5C%20report%5C%20by,monitor%5C%20crops%5C%20and%5C%20improve%5C%20yield..

[2] A. Marchisio, M. A. Hanif, F. Khalid, *et al.*, "Deep learning for edge computing: Current trends, cross-layer optimizations, and open research challenges," *2019 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2019. DOI: 10.1109/isvlsi.2019.00105.

[3] P. Dhilleswararao, S. Boppu, M. S. Manikandan, and L. R. Cenkeramaddi, "Efficient hardware architectures for accelerating deep neural networks: Survey," *IEEE Access*, vol. 10, pp. 131 788–131 828, 2022. DOI: 10.1109/access.2022.3229767.

[4] *Risc-v international – risc-v: The open standard risc instruction set architecture*. [Online]. Available: https://riscv.org/.

[5] D. . Patterson and A. . Waterman, *The RISC-V Reader: An Open Architecture Atlas*, 1st ed. Strawberry Canyon, Nov. 2017.

[6] B. Jacob, S. Kligys, B. Chen, *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2018. DOI: 10.1109/cvpr.2018.00286.

[7] *Picorv32 - a size-optimized risc-v cpu*. [Online]. Available: https://github.com/YosysHQ/picorv32.

[8] *Ibex risc-v core*. [Online]. Available: https://github.com/lowRISC/ibex.

[9] *Openhw group core-v cv32e40p risc-v ip*. [Online]. Available: https://github.com/openhwgroup/cv32e40p.

[10] *Cva6 risc-v cpu*. [Online]. Available: https://github.com/openhwgroup/cva6.

[11] *Picosoc - a simple example soc using picorv32*.

[12] *Pulpino*. [Online]. Available: https://github.com/pulp-platform/pulpino.

[13] *Pulpissimo*. [Online]. Available: https://github.com/pulp-platform/pulpissimo.

[14] *Pulp*. [Online]. Available: https://github.com/pulp-platform/pulp.

[15] N.-D. Nguyen, D.-H. Bui, and X.-T. Tran, "Tiny neuron network system based on risc-v processor: A decentralized approach for iot applications," *2022 International Conference on Advanced Technologies for Communications (ATC)*, 2022. DOI: 10.1109/atc55345.2022.9942990.

[16] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "Unpu: An energy-efficient deep neural network accelerator with fully variable weight bit precision," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 1, pp. 173–185, 2019. DOI: 10.1109/jssc.2018.2865489.

[17] Z. Du, R. Fasthuber, T. Chen, *et al.*, "Shidiannao," *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, 2015. DOI: 10.1145/2749469.2750389.

[18] D.-A. Nguyen, H.-H. Ho, D.-H. Bui, and X.-T. Tran, "An efficient hardware implementation of artificial neural network based on stochastic computing," *2018 5th NAFOSTED Conference on Information and Computer Science (NICS)*, 2018. DOI: 10.1109/nics.2018.8606843.

[19] K. . Ando, K. . Ueyoshi, K. . Orimo, *et al.*, "Brein memory: A single-chip binary/ternary reconfigurable in-memory deep neural network accelerator achieving 1.4 tops at 0.6 w," *IEEE Journal of Solid-State Circuits*, vol. 53, no. 4, pp. 983–994, Apr. 2018. DOI: 10.1109/jssc.2017.2778702.

[20] D. . Miyashita, E. H. Lee, and B. . Murmann, "Convolutional neural networks using logarithmic data representation," *arXiv: Neural and Evolutionary Computing*, Mar. 3, 2016.

[21] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017. DOI: 10.1109/jssc.2016.2616357.

[22] L. Cavigelli and L. Benini, "Origami: A 803-gop/s/w convolutional network accelerator," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 27, no. 11, pp. 2461–2475, 2017. DOI: 10.1109/tcsvt.2016.2592330.

[23] *Ml for mobile and edge devices - tensorflow lite*. [Online]. Available: https://www.tensorflow.org/lite.

[24] *Risc-v gnu compiler toolchain*. [Online]. Available: https://github.com/riscv-collab/riscv-gnu-toolchain.

[25] *Gemmlowp: A small self-contained low-precision gemm library*. [Online]. Available: https://github.com/google/gemmlowp.